



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

**Praktikum iz logičkog dizajna i
upravljanja zasnovan na FPGA razvojnom
sistemu Gowin 9k**

ZAVRŠNI RAD
- PRVI CIKLUS STUDIJA -

Student:
Tarik Ćatić

Mentor:
Vanr. prof. dr Nedim Osmić.

Sarajevo,
septembar 2026.

Sažetak

Ovaj završni rad se bavi problemom nedostatka strukturirane literature na bosanskom jeziku za učenje digitalnog dizajna i upravljanja pomoću FPGA tehnologije, koja predstavlja pristupačnu, ali pedagoški nedovoljno obrađenu platformu u odnosu na klasične mikrokontrolere. Kao rješenje, izrađen je praktikum zasnovan na FPGA razvojnom sistemu Gowin 9k (Tang Nano 9K, GW1NR-9 čip), koji kroz šest laboratorijskih vježbi rastuće složenosti postepeno uvodi studenta u koncepte digitalnog dizajna korištenjem SystemVerilog jezika i Gowin EDA razvojnog okruženja. Vježbe obuhvataju implementaciju kombinacione logike, strukturnu sintezu sekvencijalnih Mealyevog i Mooreovog automata, generisanje video signala u realnom vremenu putem HDMI/TMDS protokola uz interaktivnu aplikaciju, integraciju analogne periferije (potenciometar i ADS1115 analogno-digitalni konvertor) putem I2C komunikacije, proširenje interaktivne aplikacije analognim džojstikom uz rad sa više taktnih domena, te implementaciju i simulacionu verifikaciju diskretnog PI regulatora. Svaka vježba je praćena odgovarajućom metodologijom testiranja – fizičkom realizacijom na razvojnom sistemu i/ili simulacijom putem alata Verilator i GTKWave – čime je omogućena samostalna provjera ispravnosti implementacije prije, ili umjesto, rada na stvarnom hardveru. Rezultat rada je kompletan, testiran i dokumentovan praktikum koji povezuje osnovne principe digitalnog dizajna sa naprednijim temama poput obrade analognih signala i implementacije algoritama upravljanja, spreman za korištenje u edukativne svrhe.

Ključne riječi: FPGA, SystemVerilog, Gowin 9k, Tang Nano 9K, digitalni dizajn

Abstract

This bachelor's thesis addresses the lack of structured, locally available learning material for digital design and control using FPGA technology, a platform that offers an attractive balance of performance and cost but lacks pedagogically oriented literature compared to classical microcontrollers. As a solution, a practicum was developed based on the Gowin 9k FPGA development system (Tang Nano 9K board, GW1NR-9 chip), guiding the student through six laboratory exercises of increasing complexity using the SystemVerilog language and the Gowin EDA development environment. The exercises cover the implementation of combinational logic, the structural synthesis of sequential Mealy and Moore finite-state machines, real-time video generation via the HDMI/TMDS protocol combined with an interactive application, the integration of an analog peripheral (a potentiometer and an ADS1115 analog-to-digital converter) via I2C communication, the extension of the interactive application with an analog joystick involving multi-clock-domain design, and the implementation and simulation-based verification of a discrete PI controller. Each exercise is accompanied by an appropriate testing methodology – physical realization on the development board and/or simulation using the Verilator and GTKWave tools – allowing independent verification of the implementation's correctness prior to, or instead of, work on real hardware. The result of this work is a complete, tested, and documented practicum that connects fundamental digital design principles with more advanced topics such as analog signal processing and control algorithm implementation, ready to be used for educational purposes.

Keywords: FPGA, SystemVerilog, Gowin 9k, Tang Nano 9K, digital design

Postavka zadatka završnog rada I ciklusa:
Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA
razvojnem sistemu Gowin 9k

U okviru rada je potrebno razviti praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnem sistemu Gowin 9k. U radu je potrebno:

- upoznati se sa FPGA tehnologijom – arhitekturom programabilnih logičkih kola, principom rada konfigurabilnih logičkih blokova i prednostima u odnosu na klasične mikrokontrolere,
- upoznati se sa Gowin EDA razvojnim okruženjem i izraditi uputstvo za instalaciju i konfiguraciju svih potrebnih softverskih alata (Gowin EDA, Verilator, GTKWave) za pisanje, simulaciju i sintezu SystemVerilog koda,
- kroz laboratorijske vježbe i mini-projekte pokriti osnovne funkcionalnosti Gowin 9k FPGA uređaja – implementaciju kombinacione logike, projektovanje konačnih automata (Mealy i Moore) i uspostavljanje HDMI komunikacije sa monitorom.

Polazna literatura:

[1] Carpinelli, J.D., "An Animated Introduction to Digital Logic Design", New Jersey Institute of Technology Library, 2023.

[2] Nelson, B.E., "Designing Digital Systems With SystemVerilog", Brigham Young University, Provo, 2018.

Vanr. prof. dr Nedim Osmić, dipl. ing. el.

Izjava o autentičnosti radova

Završni rad I ciklusa studija

Ime i prezime: Tarik Čatić

Naslov rada: Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnom sistemu Gowin 9k

Vrsta rada: Završni rad Prvog ciklusa studija

Broj stranica: 105

Potvrđujem:

- da sam pročitao dokumente koji se odnose na plagijarizam, kako je to definirano Statutom Univerziteta u Sarajevu, Etičkim kodeksom Univerziteta u Sarajevu i pravilima studiranja koja se odnose na I i II ciklus studija, integrirani studijski program I i II ciklusa i III ciklus studija na Univerzitetu u Sarajevu, kao i uputama o plagijarizmu navedenim na web stranici Univerziteta u Sarajevu;
- da sam svjestan univerzitetskih disciplinskih pravila koja se tiču plagijarizma;
- da je rad koji predajem potpuno moj, samostalni rad, osim u dijelovima gdje je to naznačeno;
- da rad nije predat, u cjelini ili djelimično, za stjecanje zvanja na Univerzitetu u Sarajevu ili nekoj drugoj visokoškolskoj ustanovi;
- da sam jasno naznačio prisustvo citiranog ili parafraziranog materijala i da sam se referirao na sve izvore;
- da sam dosljedno naveo korištene i citirane izvore ili bibliografiju po nekom od preporučenih stilova citiranja, sa navođenjem potpune reference koja obuhvata potpuni bibliografski opis korištenog i citiranog izvora;
- da sam odgovarajuće naznačio svaku pomoć koju sam dobio pored pomoći mentora i akademskih tutora/ica.

Sarajevo, 21.09.2026.god

Potpis:

Tarik Čatić

Sadržaj

Popis slika	vii
Popis tabela	viii
1 Uvod	1
1.1 Obrazloženje teme	1
1.2 Korištena tehnologija	2
1.2.1 Struktura i princip rada LUT-a	2
1.2.2 Konfigurabilni logički blok (CLB)	3
1.2.3 Konfiguracija FPGA uređaja	4
1.3 Struktura rada	4
2 Razvojno okruženje i metodologija rada	6
2.1 Softversko okruženje	6
2.1.1 Instalacija i konfiguracija Gowin EDA softvera	6
2.1.2 Korištenje softvera i njegove osnovne funkcije	11
2.1.3 Instalacija programa za simulaciju	16
2.1.4 Konfiguracija programa za simulaciju	20
2.1.5 Programiranje u system verilogu	22
2.2 Korišteni hardver	31
2.2.1 Tang Nano 9K razvojni sistem	31
2.2.2 Tehničke specifikacije i Pinout	31
3 Laboratorijske vježbe i praktična implementacija	33
3.1 Laboratorijska vježba broj 1	33
3.1.1 Zadatak 1 – Implementacija kombinacione logičke funkcije	33
3.1.2 Zadatak 2 – Implementacija kombinacione logičke funkcije sa pet promjenjivih	38
3.2 Laboratorijska vježba 2	48
3.2.1 Zadatak 1 – Mealyev automat	48
3.2.2 Zadatak 2 – Mooreov automat	52
3.3 Laboratorijska vježba 3	57
3.3.1 Teorijski uvod – HDMI komunikacija	57
3.3.2 Struktura startnog projekta	58
3.3.3 Prilagođavanje HDMI predloška odabranom FPGA uređaju i monitoru	59
3.3.4 Implementacija interaktivne igre – pokretna loptica	60
3.3.5 Dodavanje pokretne prepreke	62
3.3.6 Signalizacija sudara pomoću dvije nezavisne LED diode	65
3.4 Laboratorijska vježba 4	67

3.4.1	Teorijski uvod – potencijometar i ADS1115 analogno-digitalni konvertor	67
3.4.2	Struktura projekta i način spajanja	70
3.4.3	Zadatak 1 – LED bar-graf upravljan potencijometrom	71
3.4.4	Zadatak 2 – Upravljanje jačinom svjetla LED diode pomoću PWM-a .	76
3.5	Laboratorijska vježba 5	80
3.5.1	Teorijski uvod – analogni džojstik i proširenje interaktivne HDMI igre .	80
3.5.2	Struktura projekta i način spajanja	82
3.5.3	Zadatak – upravljanje igrom putem Y ose analognog džojstika	84
3.6	Laboratorijska vježba 6	93
3.6.1	Teorijski uvod – diskretni PID regulator	93
3.6.2	Struktura projekta	94
3.6.3	Zadatak – implementacija i simulaciona verifikacija diskretnog PI regulatora	94
Lista skraćenica		103
Literatura		104

Popis slika

1.1	Detaljan prikaz island-style FPGA arhitekture sa logičkim blokovima, spojnim blokovima i matricom prekidača [1].	2
1.2	Unutrašnja struktura LUT4 elementa – 16 SRAM ćelija i stablo multipleksera upravljano ulaznim signalima.	3
1.3	Tipična struktura konfigurabilnog logičkog bloka (CLB) sa LUT-om, flip-flopom i izlaznim multiplekserom.	4
2.1	Stranica za instalaciju Gowin Eda softvera.	6
2.2	Odabir softvera shodno uređaju.	7
2.3	Pokretanje instalacije.	7
2.4	Prihvatanje uslova korištenja.	8
2.5	Odabir komponenti za instalaciju.	8
2.6	Odabir lokacije za instalaciju.	9
2.7	Kraj instalacije.	9
2.8	Instalacija drivera.	10
2.9	Kraj instalacije drivera.	10
2.10	Ekstrakcija drivera.	11
2.11	Izgled programa.	12
2.12	Kreiranje novog projekta.	13
2.13	Odabir verzije FPGA uređaja.	13
2.14	Sažetak novog kreiranog projekta.	14
2.15	Kreiranje system verilog file-a.	15
2.16	Kreiranje system verilog file-a.	15
2.17	Kreiranje system verilog file-a.	16
2.18	Stranica za slikadnje Verilator softverskog alata.	17
2.19	Pokretanje instalacije MSYS2 softverskog alata.	17
2.20	Odabir lokacije za instalaciju.	18
2.21	Odabir kratice za Start Menu Folder.	18
2.22	Pokretanje instalacije.	19
2.23	Završetak instalacije	19
2.24	Unos komandi u MSYS2 terminal.	20
2.25	Raspakivanje Test foldera.	20
2.26	Pokretanje programa za simulaciju.	21
2.27	Program za simulaciju.	21
2.28	Najbitnija polja programa za simulaciju.	22
2.29	Korisnički interfejs FloorPlanner alata.	26
2.30	Uspješno izvršena sinteza, pozicioniranje i rutiranje.	27
2.31	Gowin Programmer – prozor za konfiguraciju programiranja FPGA čipa.	28

2.32	Hardverska arhitektura Tang Nano 9K razvojnog sistema.	31
2.33	Dijagram rasporeda pinova (Pinout diagram) Tang Nano 9K.	32
3.1	Simulacija za zadatak 1 iz laboratorijske vježbe broj 1.	38
3.2	Simulacija za zadatak 2 iz laboratorijske vježbe jedan	47
3.3	Graf Mealyevog automata (zadatak 1).	48
3.4	Ulazni i izlazni signali Mealyevog automata koristeći tablicu prelaza i izlaza.	52
3.5	Ulazni i izlazni signali Mealyevog automata koristeći strukturnu shemu.	52
3.6	Graf Mooreovog automata (zadatak 2).	53
3.7	Ulazni i izlazni signali Mooreovog automata.	56
3.8	Raspored pinova HDMI konektora tipa A.	57
3.9	Princip TMDS diferencijalnog prijenosa signala.	57
3.10	Tang Nano 9K razvojni sistem sa HDMI konektorom.	59
3.11	Crvena optica prikazana na monitoru putem HDMI izlaza.	62
3.12	Crvena optica i prepreka prikazani na monitoru putem HDMI izlaza.	67
3.13	Potenciometar korišten u laboratorijskoj vježbi.	68
3.14	ADS1115 modul korišten u laboratorijskoj vježbi.	68
3.15	Blok dijagram sistema korištenog u laboratorijskoj vježbi 4.	70
3.16	Šema spajanja potenciometra, ADS1115 modula i Tang Nano 9K razvojnog sistema.	71
3.17	Broj upaljenih LED dioda kontinualno prati položaj potenciometra.	76
3.18	Princip PWM upravljanja jačinom svjetla LED diode za tri različita radna ciklusa.	77
3.19	Jačina svjetla LED dioda kontinualno prati položaj potenciometra.	80
3.20	Analogni džojstik modul korišten u laboratorijskoj vježbi.	81
3.21	Blok dijagram sistema korištenog u laboratorijskoj vježbi 5.	83
3.22	Šema spajanja džojstika, ADS1115 modula i Tang Nano 9K razvojnog sistema.	83
3.23	Izgled ekrana igre – optica se kreće isključivo po Y osi, između granica BALL_Y_MIN i BALL_Y_MAX.	85
3.24	Preslikavanje otklona džojstika od centra u brzinu pomjeranja loptice, sa označenom mrtvom zonom i pragom prelaska na veću brzinu.	92
3.25	Blok dijagram modula PID_regulacija.	94
3.26	GTKWave prikaz rezultata simulacije modula PID_regulacija: signali areset, clk, e_in[15:0] i u_out[14:0].	98

Popis tabela

2.1	Tehničke karakteristike GW1NR-9 FPGA čipa.	31
3.1	Tablica istine za logičku funkciju $Y = \bar{A}BC \vee \overline{A \vee \bar{C}}$	34
3.2	Tablica istine za logičku funkciju $Y = \bar{A}BCD \vee \overline{A \vee \bar{E}} \vee B\bar{C}D\bar{E}$	39
3.3	Tablica prelaza i izlaza (zadatak 1).	48
3.4	Kodirana stanja.	49
3.5	Kodirani ulaz.	49
3.6	Kodirani izlazi.	49
3.7	Proširena kodirana tablica (zadatak 1).	49
3.8	Tablica prelaza i izlaza (zadatak 2).	53
3.9	Kodirana stanja.	54
3.10	Kodirani ulaz.	54
3.11	Kodirani izlazi.	54
3.12	Proširena kodirana tablica (zadatak 2).	54
3.13	Polja konfiguracionog registra ADS1115 korištena u ovoj vježbi [2].	69
3.14	Pragovi paljenja LED dioda (zadatak 1).	72
3.15	Parametri preslikavanja otklona džojstika u brzinu pomjeranja loptice.	91
3.16	Odziv modula PID_regulacija po fazama testa.	99

Poglavlje 1

Uvod

Tehnologije programabilnih logičkih mreža (engl. Field Programmable Gate Arrays – FPGA) [1, 3] značajno su transformisale savremeni inženjerski pristup projektovanju digitalnih sistema, omogućavajući implementaciju kompleksnih hardverskih rješenja sa neviđenom fleksibilnošću [4]. Za razliku od tradicionalnih integrisanih kola specifične namjene (ASIC), FPGA tehnologija pruža mogućnost rekonfiguracije hardvera nakon same proizvodnje, što je ključno za razvoj prototipova i sistema visokih performansi u stvarnom vremenu [5]. Ključne prednosti FPGA sistema uključuju hardverski paralelizam, visoku propusnost podataka i determinističko ponašanje signala, što ih čini superiornim u odnosu na klasične mikrokontrolere u zahtjevnim industrijskim aplikacijama. Upravo zbog te fleksibilnosti, FPGA čipovi su postali osnova za razvoj modernih sistema u oblastima digitalne obrade signala, mrežne infrastrukture i automatizacije [6].

U fokusu završnog rada je izrada edukacijskog praktikuma zasnovanog na Gowin 9k razvojnom sistemu. Cilj je sistematizacija teorijskog i praktičnog znanja kroz seriju laboratorijskih vježbi koje vode studenta od osnovnih logičkih funkcija do naprednih upravljačkih algoritama. Kroz ovaj praktikum, akcenat je stavljen na savremene inženjerske procese, koristeći SystemVerilog [7] kao jezik za opis hardvera i Gowin EDA softversko okruženje, čime se osigurava relevantnost znanja u skladu sa trenutnim industrijskim standardima.

1.1 Obrazloženje teme

FPGA predstavljaju digitalna integrisana kola čija se unutrašnja arhitektura sastoji od konfigurabilnih logičkih blokova i fleksibilnih interkonekcija. Za razliku od procesora opšte namjene, FPGA omogućavaju korisniku da direktno na hardverskom nivou definiše funkcionalnost uređaja, što ih čini idealnim za zadatke koji zahtijevaju masivni paralelizam i rad u stvarnom vremenu.

Poseban značaj ove tehnologije leži u mogućnosti programiranja unutar sistema (engl.: ISP – In-System Programmable), što dozvoljava modifikaciju hardvera bez fizičkih intervencija na samoj ploči. Upravo ta fleksibilnost čini FPGA platforme ključnim obrazovnim alatom u modernom digitalnom dizajnu [8].

Iako su FPGA uređaji prisutni decenijama, tradicionalne platforme (poput onih firmi Xilinx ili Intel) često su preskupe ili previše kompleksne za početni nivo učenja. **Gowin 9k** sistem [9] nudi izuzetan balans između performansi i cijene, ali mu nedostaje strukturirana literatura na lokalnom jeziku koja bi vodila studenta kroz proces dizajna.

Cilj ovog završnog rada je izrada praktikuma koji će obuhvatiti:

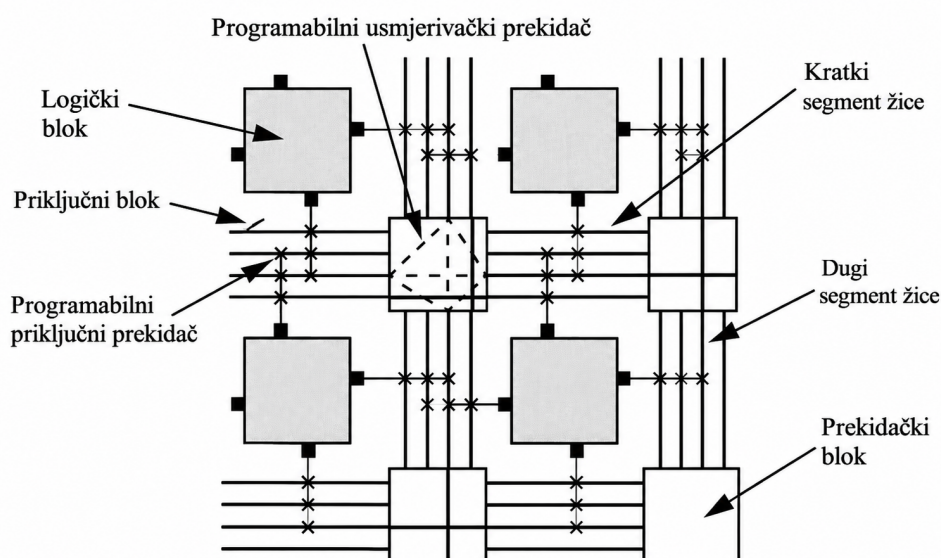
- Upoznavanje sa arhitekturom **Gowin 9k** čipa;

- Rad u razvojnom okruženju **Gowin EDA**;
- Implementaciju praktičnih primjera (od osnovnih logičkih kola do složenijih kontrolera).

Završni rad pruža metodološki okvir za učenje modernog digitalnog dizajna, koristeći pristupačnu tehnologiju koja je trenutno u velikom usponu u industriji ugradbenih sistema.

1.2 Korištena tehnologija

Savremeni FPGA uređaji, uključujući i seriju Gowin LittleBee, zasnivaju se na takozvanoj island-style (ostrvskoj) arhitekturi. Prema ovom modelu, unutrašnjost čipa je organizovana kao dvodimenzionalna matrica konfigurabilnih logičkih blokova (engl.: CLB - Configurable Logic Block) koji su „potopljeni” u mrežu programabilnih kanala za rutiranje. Osnovu svakog logičkog bloka čini Look-Up Table (LUT), najčešće četveroulazni (LUT4), koji omogućava implementaciju bilo koje kombinacione logičke funkcije putem tabele istinitosti upisane u SRAM (engl.: Static Random-Access Memory, bos.: statička memorija sa slučajnim pristupom) ćelije. Pored LUT-a, CLB sadrži i registre (D-flip-flopove) za sekvencijalnu logiku, te namjenske multipleksere za brzi prenos signala (engl. carry logic). Interkonekcijska mreža, koja okružuje logičke blokove, sastoji se od horizontalnih i vertikalnih kanala. Ključni elementi ove mreže su Connection Boxes (matrice konekcije), koji povezuju ulaze i izlaze CLB-a sa žicama u kanalima, te Switch Boxes (matrice prekidača) koje se nalaze na presjecima kanala. Ove matrice omogućavaju fleksibilno preusmjeravanje signala kroz čip, čime se definiše konačna hardverska funkcija sistema bez potrebe za fizičkom izmjenom komponenti [1].



Slika 1.1: Detaljan prikaz island-style FPGA arhitekture sa logičkim blokovima, spojnim blokovima i matricom prekidača [1].

1.2.1 Struktura i princip rada LUT-a

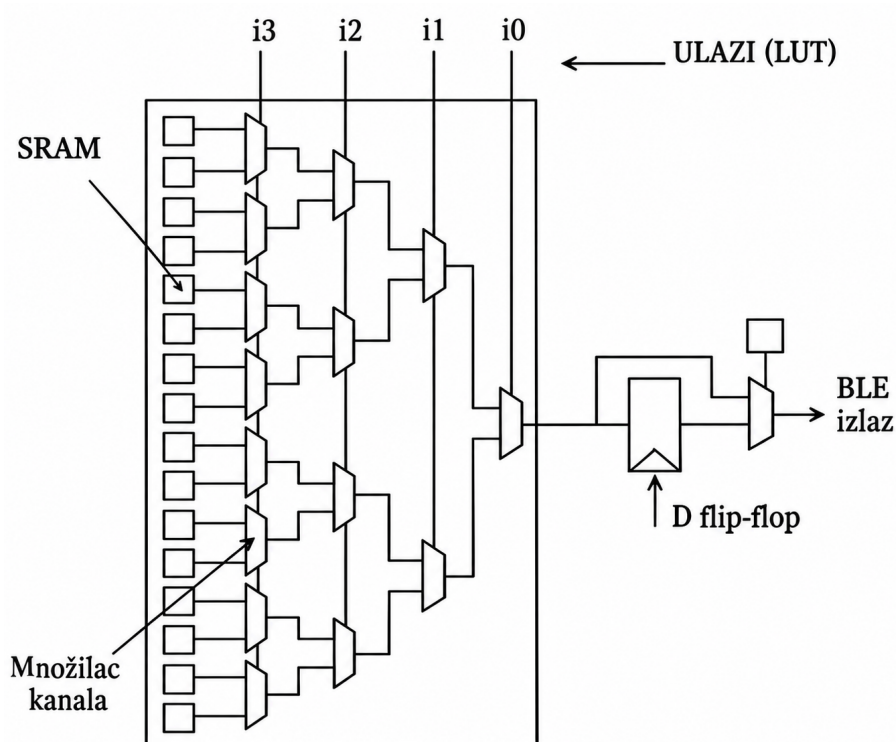
Look-Up Table predstavlja osnovnu logičku ćeliju FPGA uređaja i, za razliku od klasičnih logičkih kapija koje su fizički ožičene da izvode tačno jednu logičku funkciju, LUT ne izvodi

logičku operaciju u klasičnom smislu – umjesto toga, unaprijed izračunatu tablicu istinitosti neke proizvoljne logičke funkcije jednostavno čita iz male interne memorije [10]. Za n -ulazni LUT, postoji 2^n mogućih kombinacija ulaznih vrijednosti, pa se u SRAM ćelije LUT-a upisuju tačno 2^n bita – po jedan bit za svaku kombinaciju ulaza, koji predstavlja izlaznu vrijednost funkcije za tu kombinaciju [11]. Kod uobičajenog LUT4 elementa (4 ulaza), to iznosi $2^4 = 16$ bita, dovoljno da se predstavi *bilo koja* moguća logička funkcija četiri promjenjive.

Sama implementacija LUT-a sastoji se od dvije cjeline [10]:

- niza od 2^n jednobitnih SRAM ćelija, u koje se prilikom konfiguracije (programiranja) FPGA uređaja upisuju vrijednosti tablice istinitosti željene funkcije;
- multipleksera (ili stabla manjih multipleksera, npr. niza 2-na-1 multipleksera) čije upravljačke (selekcione) linije čine upravo ulazni signali LUT-a, a čiji podatkovni ulazi dolaze sa izlaza SRAM ćelija.

Princip rada je, prema tome, jednostavan: ulazne vrijednosti A, B, C, D (u slučaju LUT4 elementa) djeluju kao adresa koja bira tačno jednu od 16 upisanih SRAM vrijednosti, koja se zatim provodi na izlaz LUT-a [11]. Pošto je ovo suštinski operacija čitanja iz memorije, a ne fizičko preklapanje tranzistora u lancu logičkih kapija, kašnjenje signala kroz LUT je predvidivo i nezavisno od toga koja je konkretna logička funkcija u njemu programirana. Slika 1.2 [10] prikazuje unutrašnju strukturu jednog LUT4 elementa.

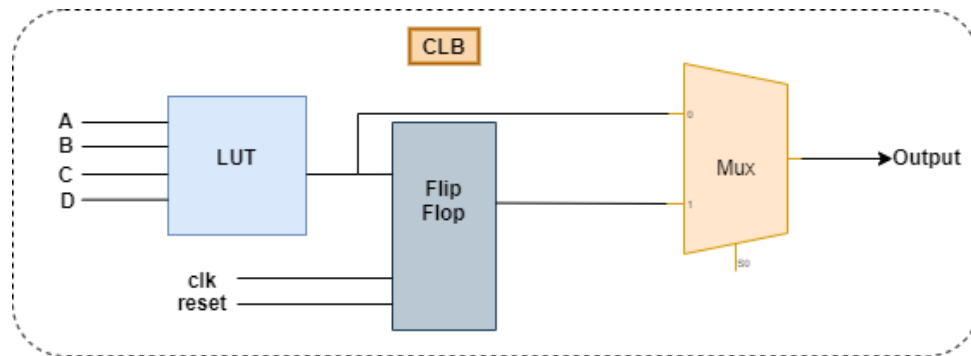


Slika 1.2: Unutrašnja struktura LUT4 elementa – 16 SRAM ćelija i stablo multipleksera upravljano ulaznim signalima.

1.2.2 Konfigurabilni logički blok (CLB)

Jedan CLB obično objedinjuje više LUT elemenata zajedno sa pripadajućim D-flip-flopovima i multiplekserima za odabir izlaza (npr. odabir između kombinacione vrijednosti direktno sa

LUT-a ili registrovane vrijednosti sa izlaza flip-flopa) [1]. Ovakva struktura omogućava da se u istom bloku implementira kako čisto kombinaciona logika (LUT bez korištenja flip-flopa), tako i sekvencijalna logika (LUT koji generiše sljedeće stanje, koje se registruje u flip-flop na ivicu takta) – što odgovara upravo dva proceduralna bloka koja se koriste u SystemVerilogu, `always_comb` i `always_ff`, opisana u poglavlju 2.1.5. Slika 1.3 prikazuje tipičnu unutrašnju organizaciju jednog CLB-a.



Slika 1.3: Tipična struktura konfigurabilnog logičkog bloka (CLB) sa LUT-om, flip-flopom i izlaznim multiplekserom.

1.2.3 Konfiguracija FPGA uređaja

Vrijednosti upisane u SRAM ćelije LUT-ova, kao i stanje svih programabilnih prekidača u interkonekcijskoj mreži, zajedno čine takozvani *bitstream* – binarni fajl koji u potpunosti definiše hardversku funkciju čipa, dobijen kao rezultat sinteze i place-and-route procesa opisanog u prethodnim poglavljima. Pošto se radi o SRAM tehnologiji, ova konfiguracija je isparljiva (*volatile*) i gubi se prilikom gubitka napajanja, pa GW1NR-9 čip pri svakom uključivanju mora ponovo učitati bitstream iz vanjske neisparljive (*non-volatile*) SPI flash memorije smještene na samoj Tang Nano 9K pločici, što traje tek nekoliko milisekundi [1].

1.3 Struktura rada

Završni rad je organizovan kroz sistematski međusobno povezana poglavlja koja se sastoje od teorijskih osnova logičkog dizajna do praktične primjene na savremenom FPGA hardveru. Struktura je prilagođena formi praktikuma kako bi se omogućilo postepeno usvajanje znanja.

- **Prvo poglavlje** donosi uvod u svijet programabilne logike. Predstavljena je motivacija za izradu praktikuma, sa posebnim osvrtom na prednosti FPGA tehnologije u odnosu na tradicionalne edukacijske metode. Definisani su ključni ciljevi rada i uloga Gowin 9k sistema u modernom inženjerskom obrazovanju.
- **Drugo poglavlje** detaljno elaborira arhitekturu Gowin 9k razvojnog sistema. Predstavljene su specifikacije FPGA čipa, unutrašnji resursi (logički elementi, memorija, DSP blokovi) te prateći hardver na ploči koji je neophodan za realizaciju vježbi, poput LED dioda, tastera i komunikacijskih interfejsa. Poglavlje takođe opisuje instalaciju i konfiguraciju Gowin EDA razvojnog okruženja, kao i alata za simulaciju (Verilator, GTKWave) korištenih kroz sve laboratorijske vježbe.

- **Treće poglavlje** čini srž praktikuma, kroz šest laboratorijskih vježbi rastućeg stepena složenosti. Prva vježba uvodi osnove kombinacione logike, kroz implementaciju i simulaciju logičkih funkcija sa tri, odnosno pet ulaznih promjenjivih. Druga vježba prelazi na sekvencijalnu logiku, kroz strukturnu sintezu Mealyevog i Mooreovog konačnog automata, sa implementacijom na dva nezavisna načina (tablica prelaza i izlaza, te strukturna shema sa T flip-flopovima). Treća vježba uvodi HDMI komunikaciju, od teorijskih osnova TMDS prenosa signala, preko prilagođavanja startnog projekta odabranom FPGA uređaju i monitoru, do implementacije interaktivne aplikacije na samom razvojnom sistemu. Četvrta vježba uvodi analogne periferije kroz potencijometar i ADS1115 analogno-digitalni konvertor povezan I2C protokolom, sa dva zadatka – upravljanjem LED bar-grafom i regulacijom jačine svjetla LED diode putem PWM-a. Peta vježba proširuje HDMI igru iz treće vježbe analognim džojstikom, uvodeći koncept mrtve zone pri obradi analognog signala te sinhronizaciju signala između različitih taktnih domena. Šesta vježba zaokružuje praktikum implementacijom diskretnog PI regulatora, uz simulacionu verifikaciju ispravnosti hardverske realizacije naspram teorijski izvedene formule.
- **Četvrto poglavlje** donosi zaključna razmatranja rada. Sumiraju se ostvareni rezultati i doprinosi izrađenog praktikuma, te se daju smjernice za moguć buduć razvojni, poput fizičke realizacije šeste vježbe na stvarnom izvršnom elementu.

*

*

*

U ovom poglavlju predstavljena je motivacija za izradu praktikuma zasnovanog na Gowin 9k FPGA sistemu, kao i teorijske osnove neophodne za razumijevanje unutrašnje arhitekture FPGA uređaja – od konfigurabilnih logičkih blokova (CLB) i njihove osnovne gradivne jedinice, LUT-a, preko interkonekcijske mreže, do samog procesa konfigurisanja čipa putem bitstream-a. Time je čitalac upoznat sa time *šta* FPGA tehnologija predstavlja i *zašto* je odabrana kao platforma za izradu ovog praktikuma.

Naredno poglavlje prelazi sa teorijskih osnova na praktičnu pripremu za rad – opisuje se instalacija i konfiguracija Gowin EDA razvojnog okruženja, osnovni alati potrebni za pisanje, simulaciju i sintezu SystemVerilog koda, te se detaljno predstavlja Tang Nano 9K razvojni sistem koji će se koristiti kroz sve laboratorijske vježbe u nastavku rada.

Poglavlje 2

Razvojno okruženje i metodologija rada

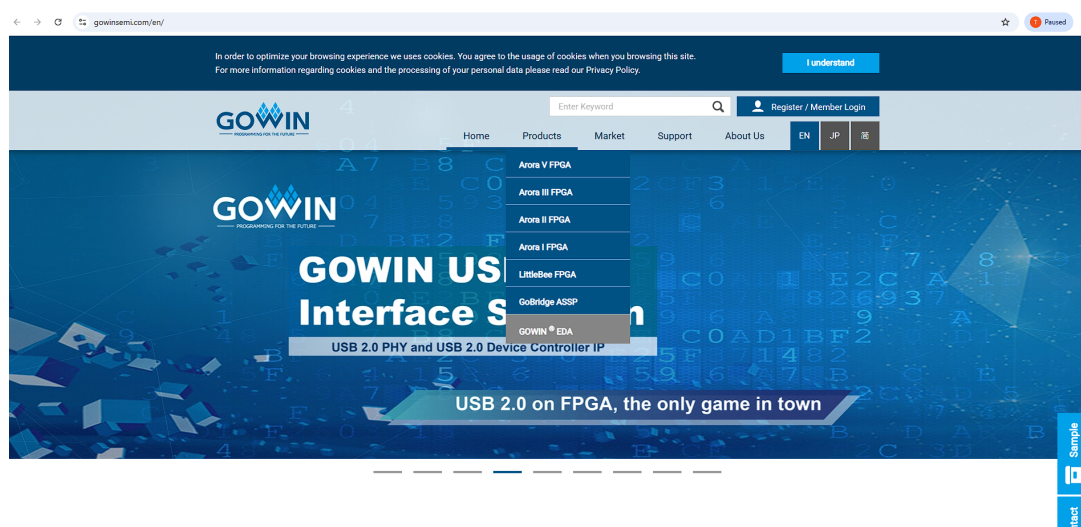
2.1 Softversko okruženje

Ovo poglavlje opisuje proces postavljanja razvojnog okruženja i osnovna znanja potrebna za rad u njemu, kao i pregled korištenog hardvera. Fokus je na upoznavanju sa Gowin EDA okruženjem i praktičnoj primjeni SystemVerilog jezika, te na predstavljanju Tang Nano 9K razvojnog sistema koji se koristi za realizaciju svih laboratorijskih vježbi u nastavku rada.

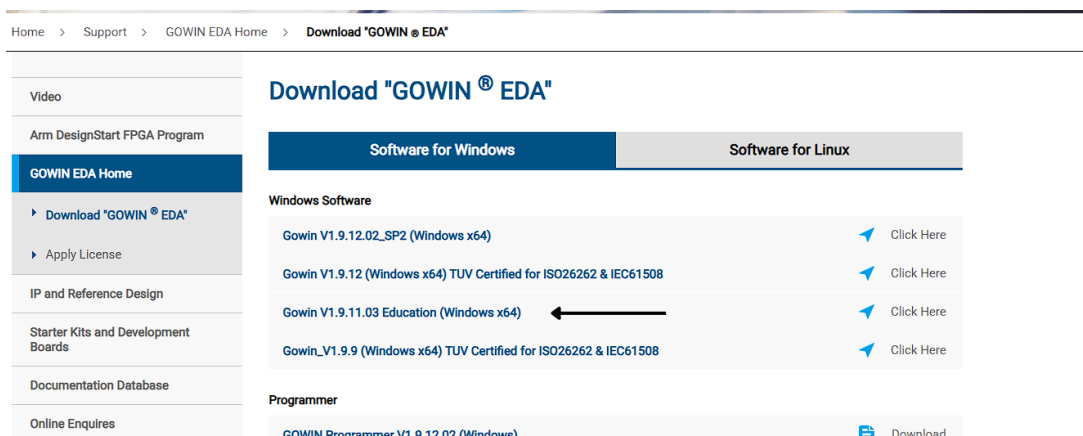
2.1.1 Instalacija i konfiguracija Gowin EDA softvera

Kako Tang Nano 9K razvojni sistem koristi GW1NR-9 FPGA čip, za izradu projekata i pisanje koda koristiti se Gowin EDA softver, razvojno okruženje koje služi za pisanje koda korištenjem VHDL-a, Verilog-a ili SystemVeriloga. Instalacija softvera se može izvršiti praćenjem sljedećih koraka:

1. Prvi korak jeste posjetiti stranicu <https://www.gowinsemi.com/en/>. Potrebno je napraviti registraciju sa fakultetskim mailom kako bi se mogao instalirati program. Nakon toga treba otići na opciju Products u navigation baru i izabrati opciju GOWIN EDA kao na slici 2.1. Nakon toga treba izabrati opciju Download Gowin EDA i izabrati verziju shodno uređaju koji korisnik koristi 2.2. Potrebno je izabrati Education verziju softvera. Ona je sasvim dovoljna za izradu zadataka i za nju nije potrebna licenca.

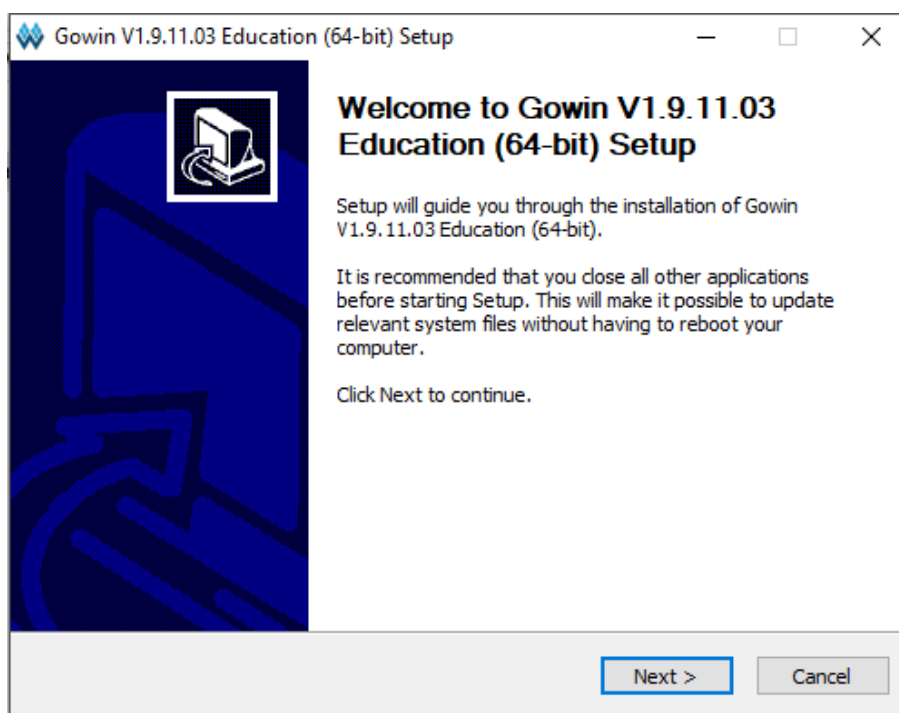


Slika 2.1: Stranica za instalaciju Gowin EDA softvera.

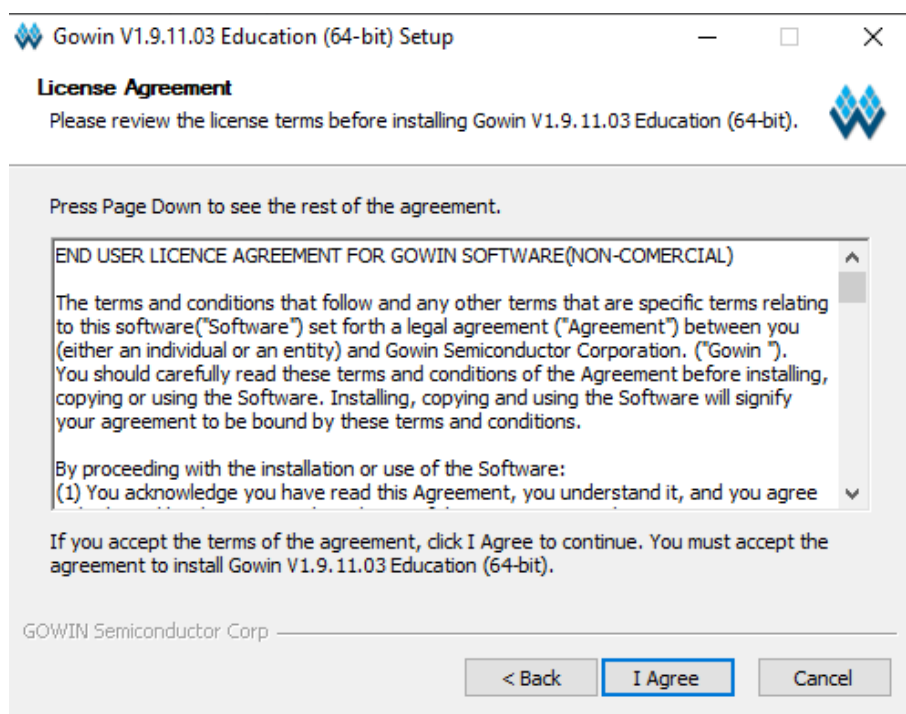


Slika 2.2: Odabir softvera shodno uređaju.

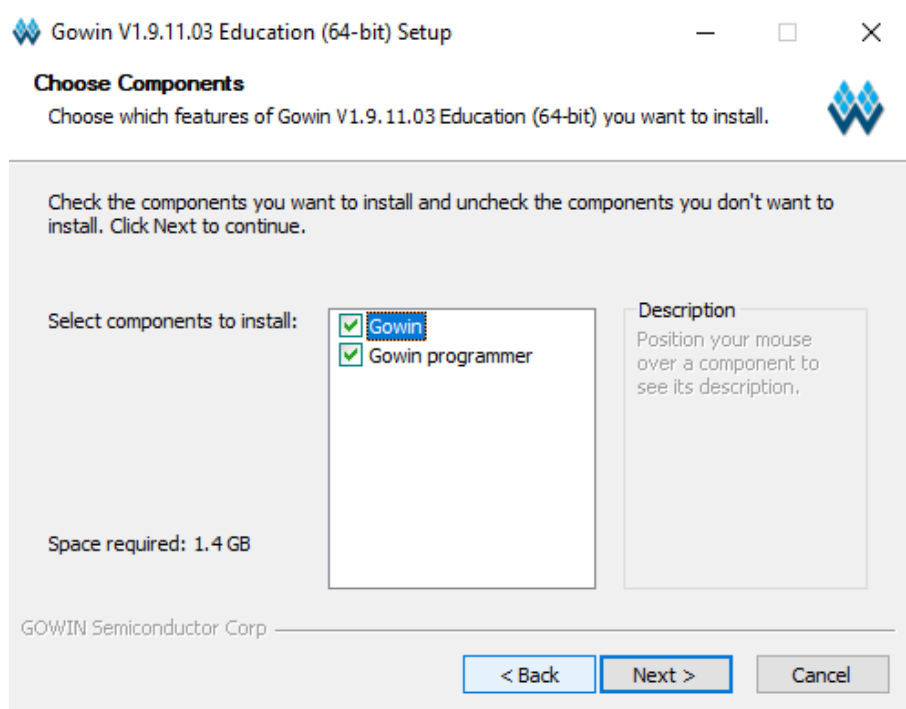
2. Drugi korak jeste pokretanje instalacije (slika 2.3). Nakon pokretanja instalacije potrebno je prihvatiti License Agreement (slika 2.4), izabrati komponente koje instaliramo (a to su Gowin i Gowin programmer 2.5), izabrati lokaciju na koju želimo pohraniti program (slika 2.6) i potrebno je označiti instalaciju drivera koji su neophodni za povezivanje FPGA pločice sa programom (pokazano je na slici 2.7). Pritiskom na dugme Finish završena je instalacija Gowin EDA razvojnog softvera.



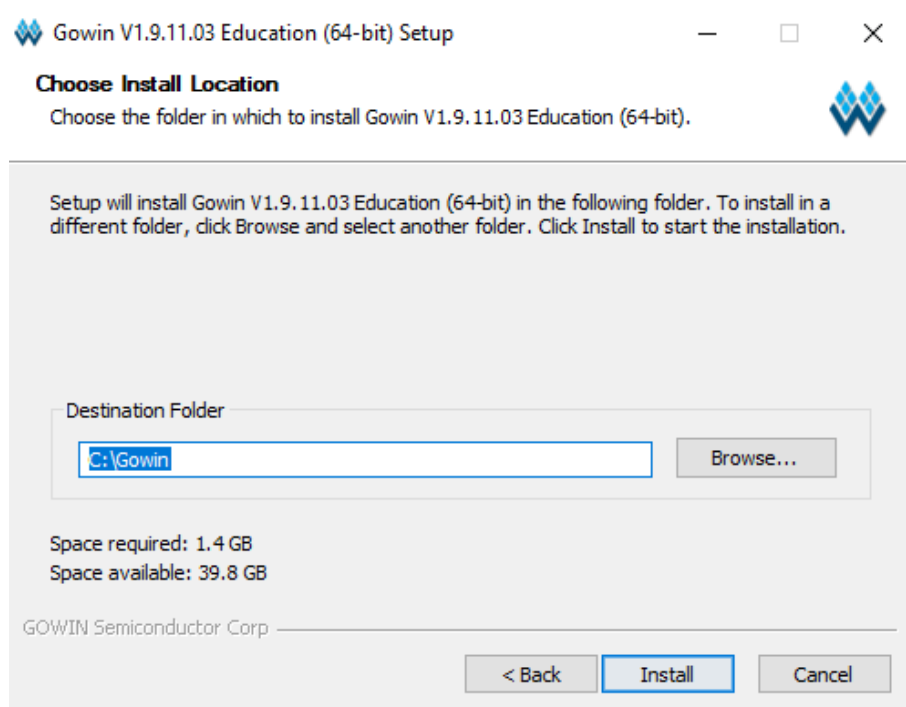
Slika 2.3: Pokretanje instalacije.



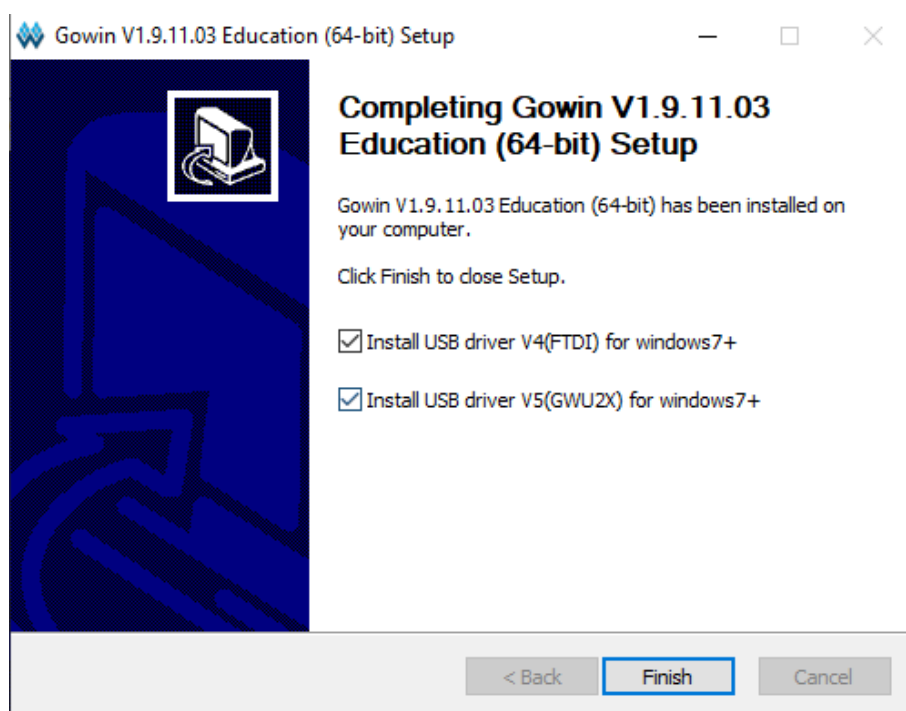
Slika 2.4: Pihvatanje uslova korištenja.



Slika 2.5: Odabir komponenti za instalaciju.

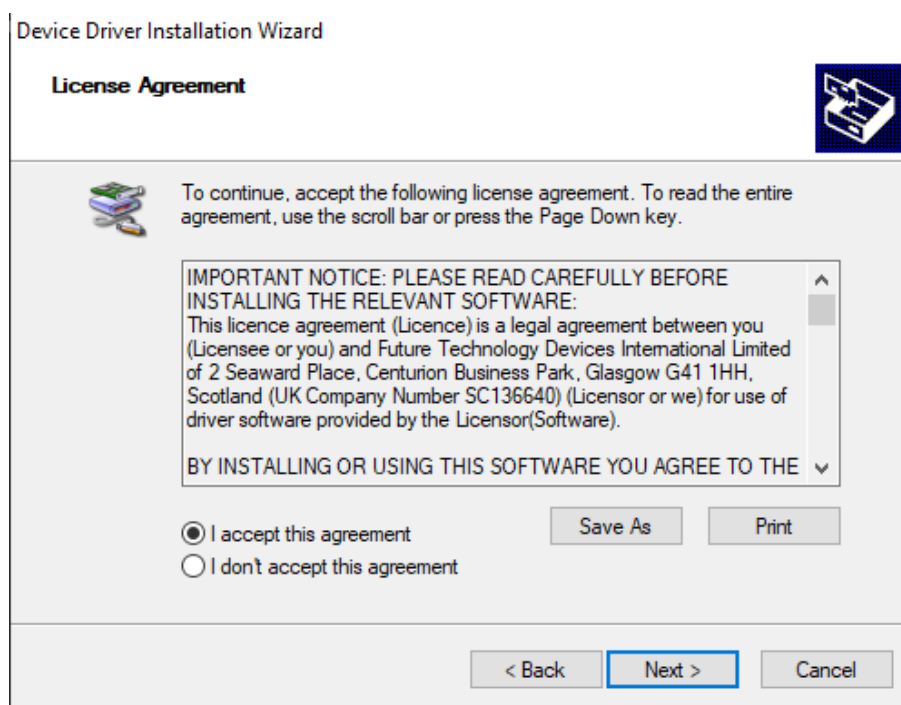


Slika 2.6: Odabir lokacije za instalaciju.



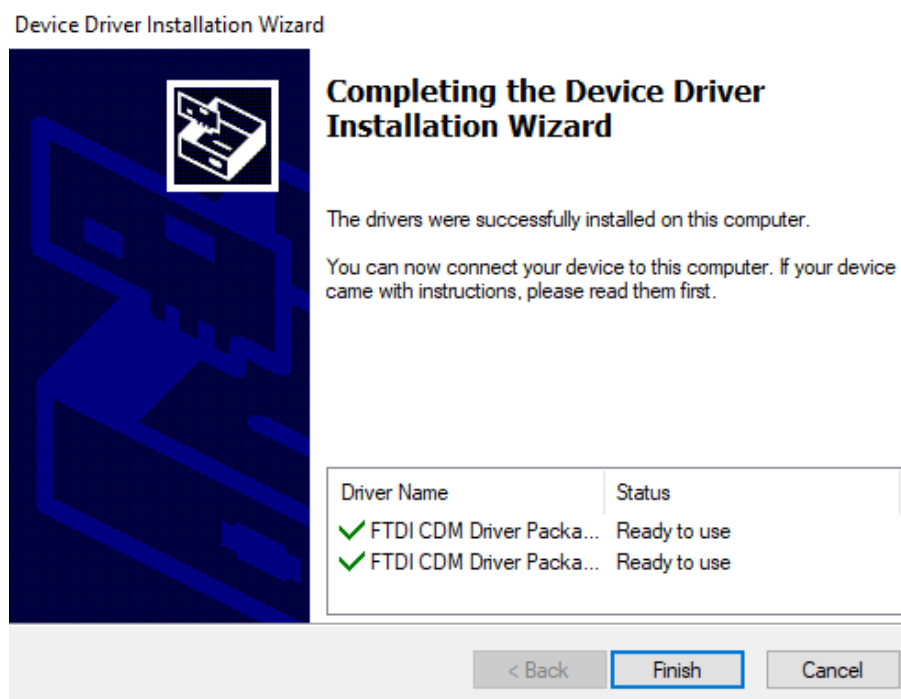
Slika 2.7: Kraj instalacije.

3. Nakon završetka instalacije Gowin Eda razvojnog softvera pokrenuti će se instalacija driver-a štrahiranih na slici 2.7, koji su potrebni za ispravan rad okruženja i korištene FPGA tehnologije.

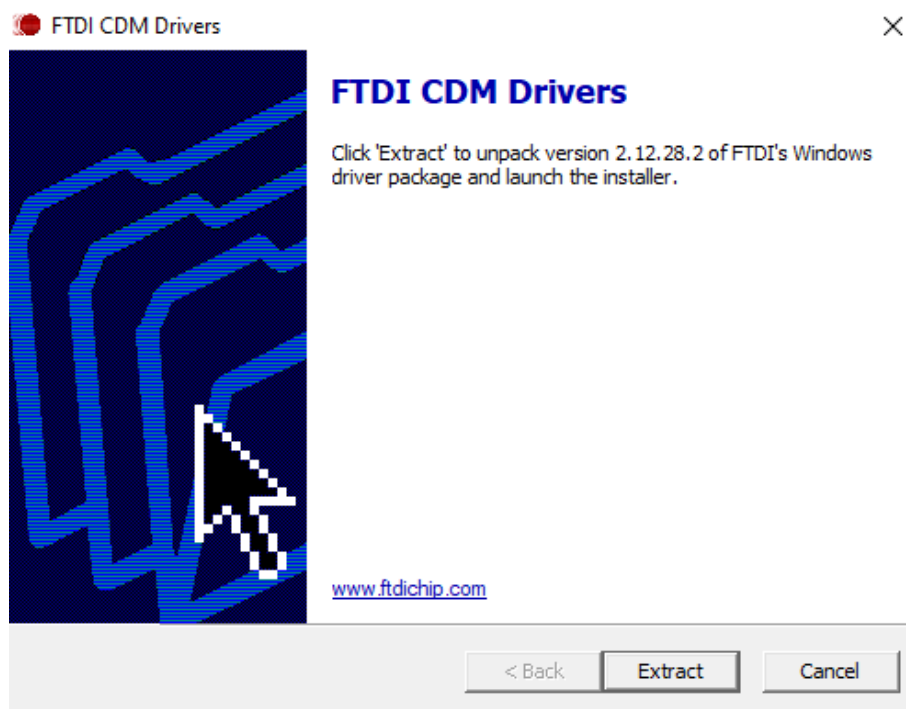


Slika 2.8: Instalacija drivera.

Ako se nakon instalacije drivera (slika 2.8) pojavi prozor kao na slici 2.9, onda je instalacija uspješno obavljena. Nakon klika na dugme Finish otvoriti će se još jedan prozor za ekstrakciju driver-a (kao na slici 2.10). Izvršavanjem ovog posljednjeg koraka razvojni softver je spreman za korištenje.



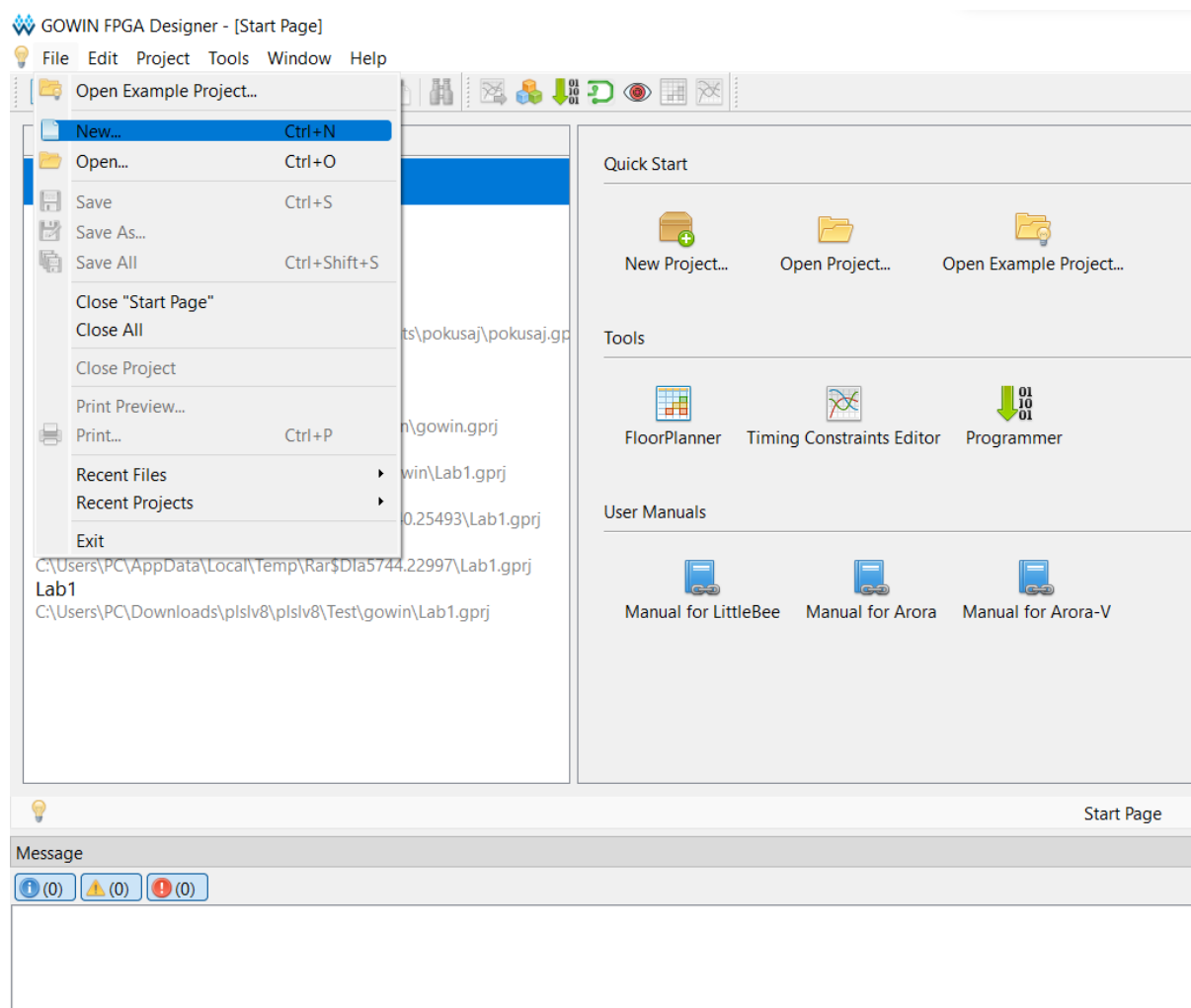
Slika 2.9: Kraj instalacije drivera.



Slika 2.10: Ekstrakcija drivera.

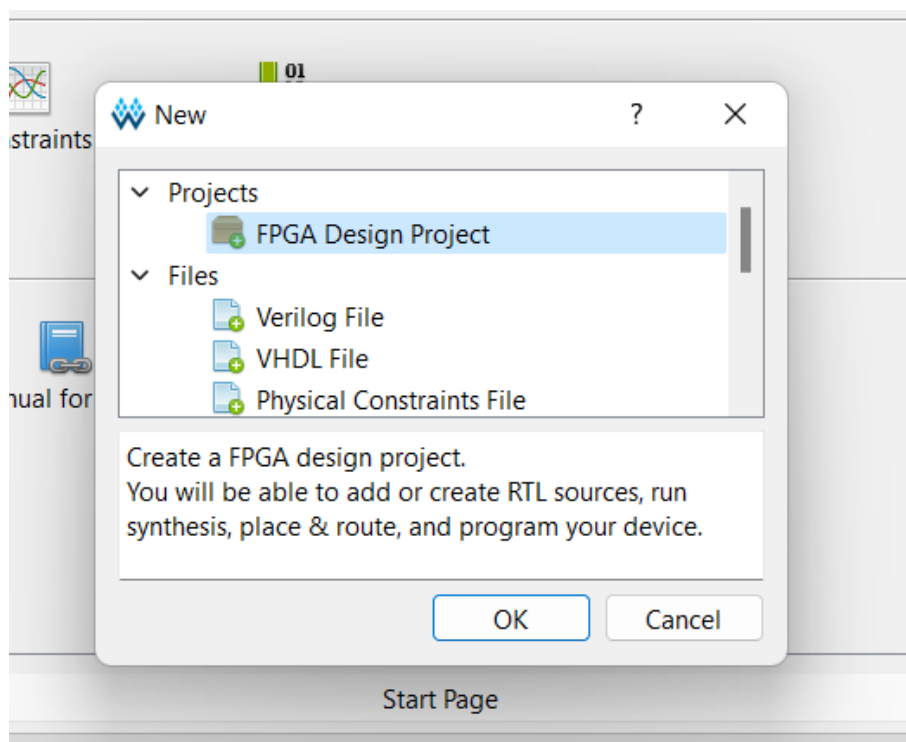
2.1.2 Korištenje softvera i njegove osnovne funkcije

Pokretanjem Gowin FPGA Designera treba se otvoriti prozor kao na slici 2.11. Za kreiranje novog projekta potrebno je odabrati opciju File, pa New file.

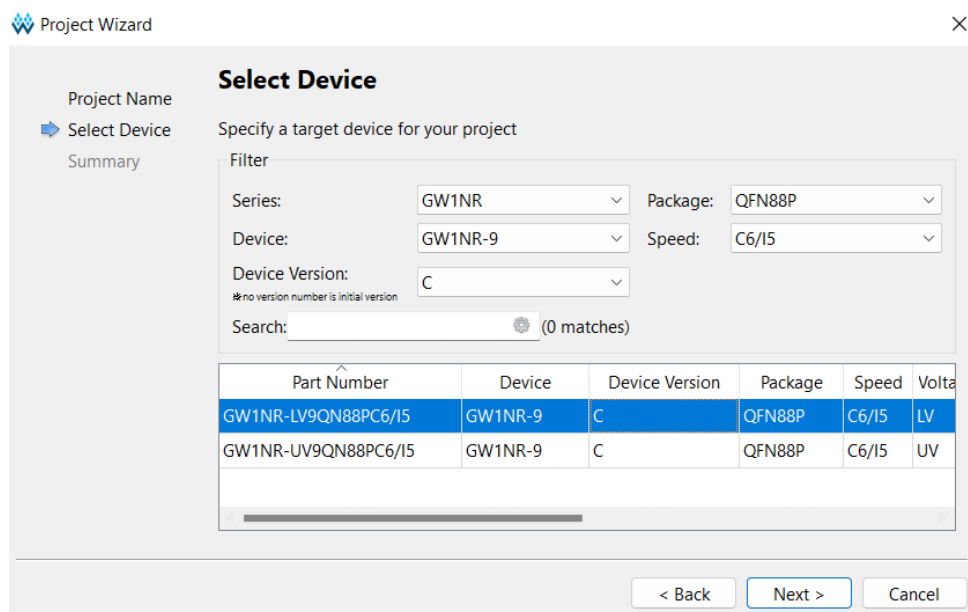


Slika 2.11: Izgled programa.

Dalje bi se trebao otvoriti prozor kao na slici 2.12. Tu se treba izabrati FPGA design project jer je prvo potrebno napraviti projekat a tek kasnije mozemo praviti file-ove unutar projekta. Pritiskom na dugme OK otvara se prozor sa slike 2.13. Tu se treba odabrati FPGA uređaj koji koristimo za izradu projekta. Ovaj korak je jako bitan jer ukoliko se ne odabere tačna verzija uređaja, program koji se napiše u ovom razvojnom softveru neće se moći prebaciti na uređaj. Tačna verzija uređaja se može pročitati sa samog uređaja ili se može pronaći u njegovoj dokumentaciji.

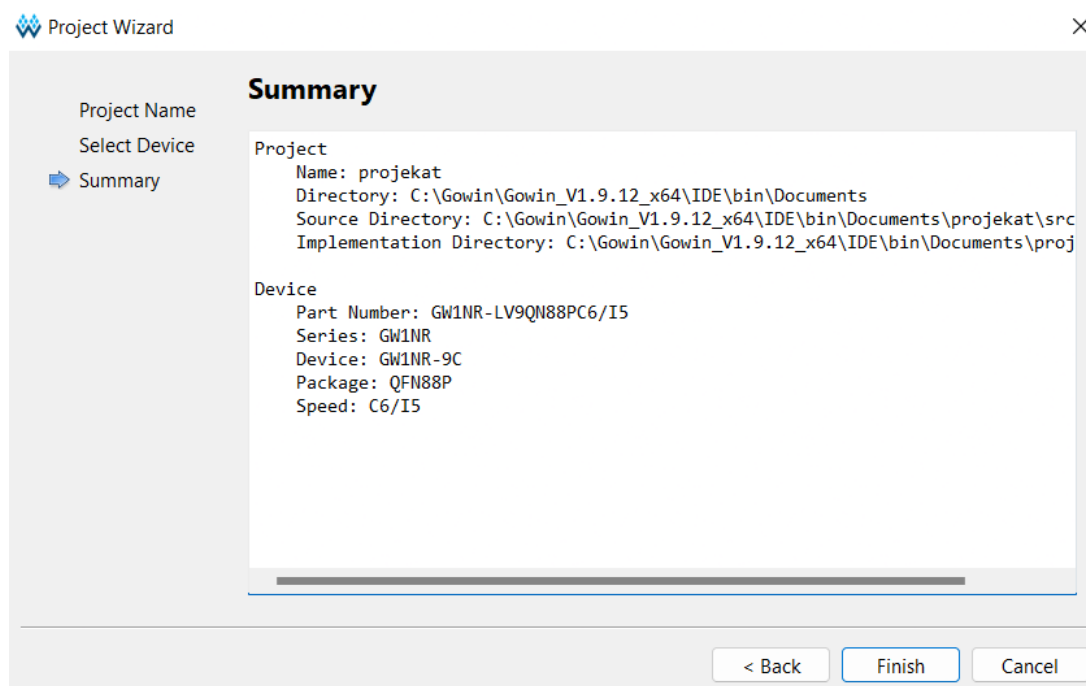


Slika 2.12: Kreiranje novog projekta.



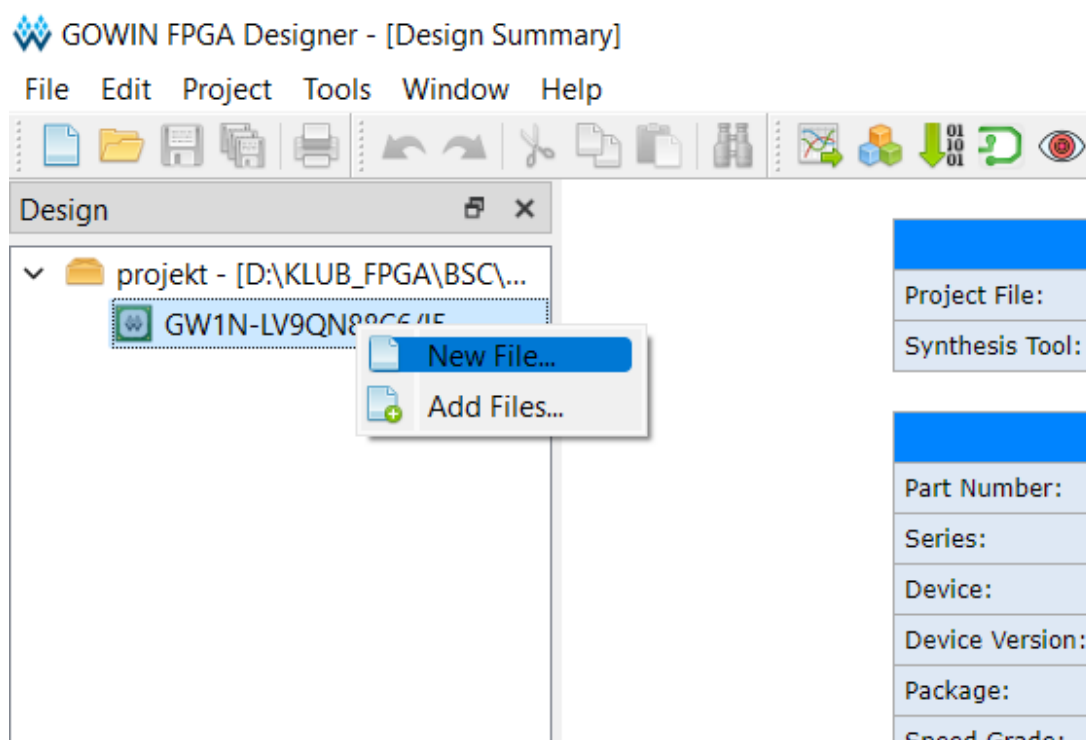
Slika 2.13: Odabir verzije FPGA uređaja.

Nakon što se odabere uređaj trebao bi se pojaviti prozor kao na slici 2.14 sa kratkim sažetkom hardvera na kojem se radi projekat. Tu se može pregledati ispravnost novog kreiranog projekta i ako je sve kako treba ovaj korak završavamo pritiskom na opciju Finish.

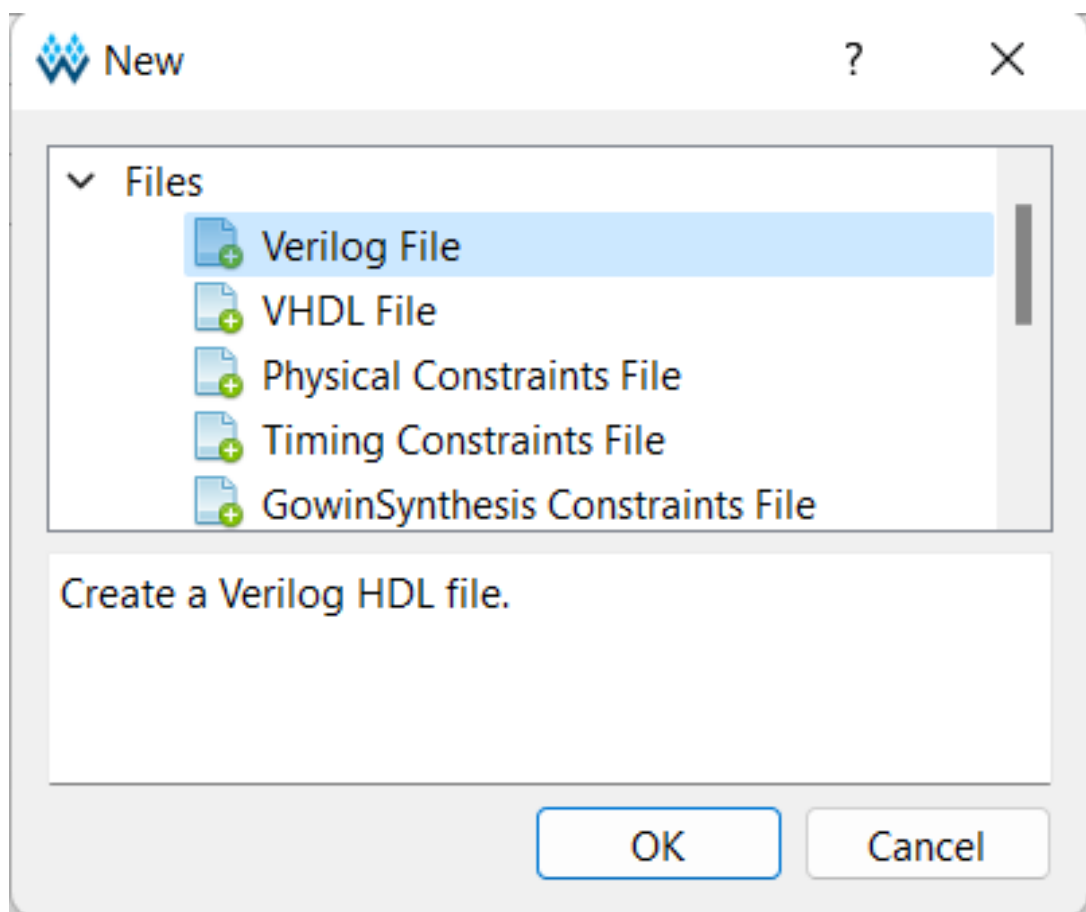


Slika 2.14: Sažetak novog kreiranog projekta.

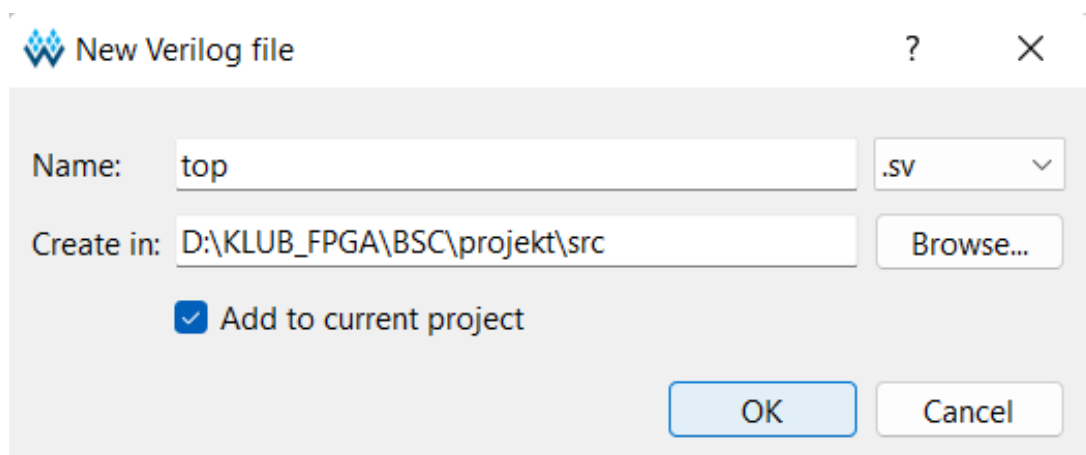
Sada je potrebno kreirati .sv file u kojem se piše logika zadatka koji treba izvršavati FPGA. Ako kliknemo desnim klikom na projekat trebale bi se pojaviti opcije kao na slici 2.15. Odabirom opcije New file trebao bi se pojaviti prozor kao na slici 2.16. Tu se treba odabrati Verilog file i pritiskom na dugme OK treba se otvoriti prozor kao na slici 2.17. Sada je bitno izabrati sa lijeve strane .sv jer će ta opcija kreirati system verilog file što u ovom slučaju i treba. Verilog i system verilog su dva različita jezika kao što su C i C++ dva različita jezika. Na ovom praktikumu svi zadaci će biti rađeni u system verilogu.



Slika 2.15: Kreiranje system verilog file-a.



Slika 2.16: Kreiranje system verilog file-a.



Slika 2.17: Kreiranje system verilog file-a.

Poželjno je prvi .sv file koji se napravi nazvati top jer se on najčešće koristi kao top modul. Više o tome će biti objašnjeno u narednom dijelu poglavlja.

2.1.3 Instalacija programa za simulaciju

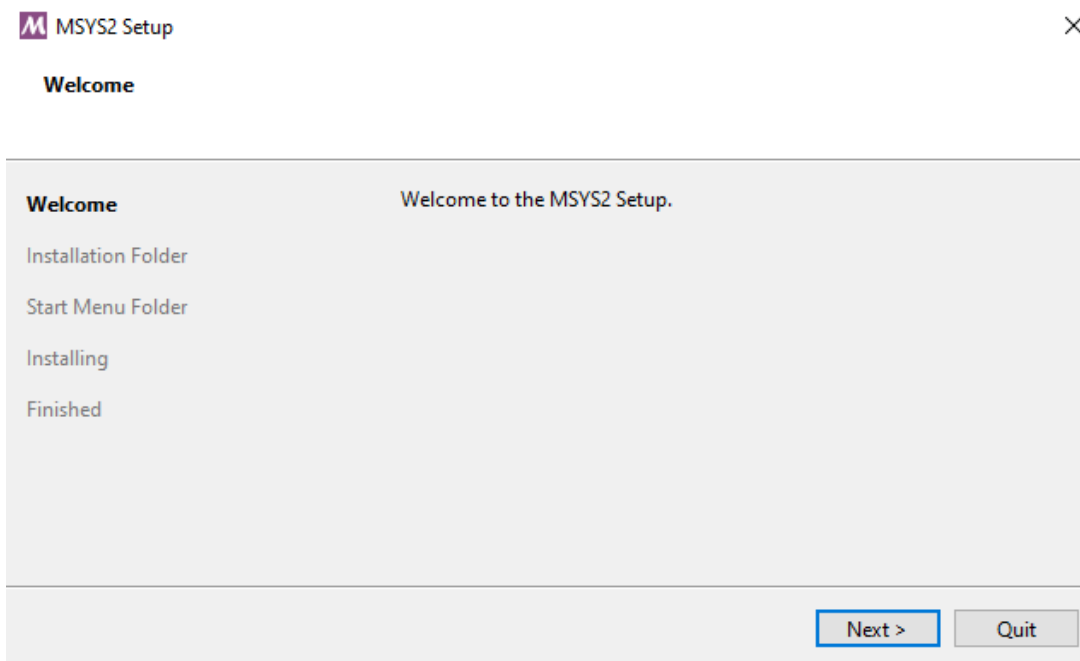
Jedan od osnovnih nedostataka Gowin EDA softvera je što ne podržava simulaciju izrađenih projekata. Dakle, izrađeni projekt se mora prebaciti na FPGA pločicu i moraju se spojiti svi korišteni ulazi i izlazi kako bi se testirala ispravnost napisanog koda. Međutim, postoji alternativa koja olakšava proceduru. Za testiranje projekata može se koristiti Verilator softverski alat. To je open source softverski alat koji služi za pretvaranje Verilog/SystemVerilog koda u C++ kod. Njegova instalacija se može izvršiti praćenjem sljedećih koraka:

- Na sljedećem linku može se preuzeti file za instalaciju ovog softverskog alata https://repo.msys2.org/distrib/x86_64/. Potrebno je instalirati MSYS2 okruženje pomoću kojeg se može instalirati Verilator program za testiranje Gowin projekata jer je Verilator program napravljen za rad sa Linux operativnim sistemom. Za instalaciju je najbolje izabrati posljednju verziju koja je izašla, kao na slici 2.18, ali ni par prethodnih verzija ne bi trebale predstavljati problem jer je MSYS2 "rolling release" sistem koji će se prilikom prvog ažuriranja sam nadograditi na najnoviju verziju.

msys2-base-x86_64-20260322.tar.xz	22-Mar-2026 15:40	51M
msys2-base-x86_64-20260322.tar.xz.sig	22-Mar-2026 15:40	566
msys2-base-x86_64-20260322.tar.zst	22-Mar-2026 15:40	52M
msys2-base-x86_64-20260322.tar.zst.sig	22-Mar-2026 15:40	566
msys2-x86_64-20240507.exe	07-May-2024 06:05	80M
msys2-x86_64-20240507.exe.sig	07-May-2024 06:05	566
msys2-x86_64-20240727.exe	27-Jul-2024 13:17	79M
msys2-x86_64-20240727.exe.sig	27-Jul-2024 13:17	566
msys2-x86_64-20241116.exe	16-Nov-2024 18:50	80M
msys2-x86_64-20241116.exe.sig	16-Nov-2024 18:50	566
msys2-x86_64-20241208.exe	08-Dec-2024 11:32	80M
msys2-x86_64-20241208.exe.sig	08-Dec-2024 11:32	566
msys2-x86_64-20250221.exe	21-Feb-2025 11:32	82M
msys2-x86_64-20250221.exe.sig	21-Feb-2025 11:32	566
msys2-x86_64-20250622.exe	22-Jun-2025 18:03	88M
msys2-x86_64-20250622.exe.sig	22-Jun-2025 18:03	566
msys2-x86_64-20250830.exe	30-Aug-2025 15:35	89M
msys2-x86_64-20250830.exe.sig	30-Aug-2025 15:35	566
msys2-x86_64-20251213.exe	13-Dec-2025 17:12	89M
msys2-x86_64-20251213.exe.sig	13-Dec-2025 17:12	566
msys2-x86_64-20260322.exe	22-Mar-2026 15:40	90M
msys2-x86_64-20260322.exe.sig	22-Mar-2026 15:40	566

Slika 2.18: Stranica za slikadnje Verilator softverskog alata.

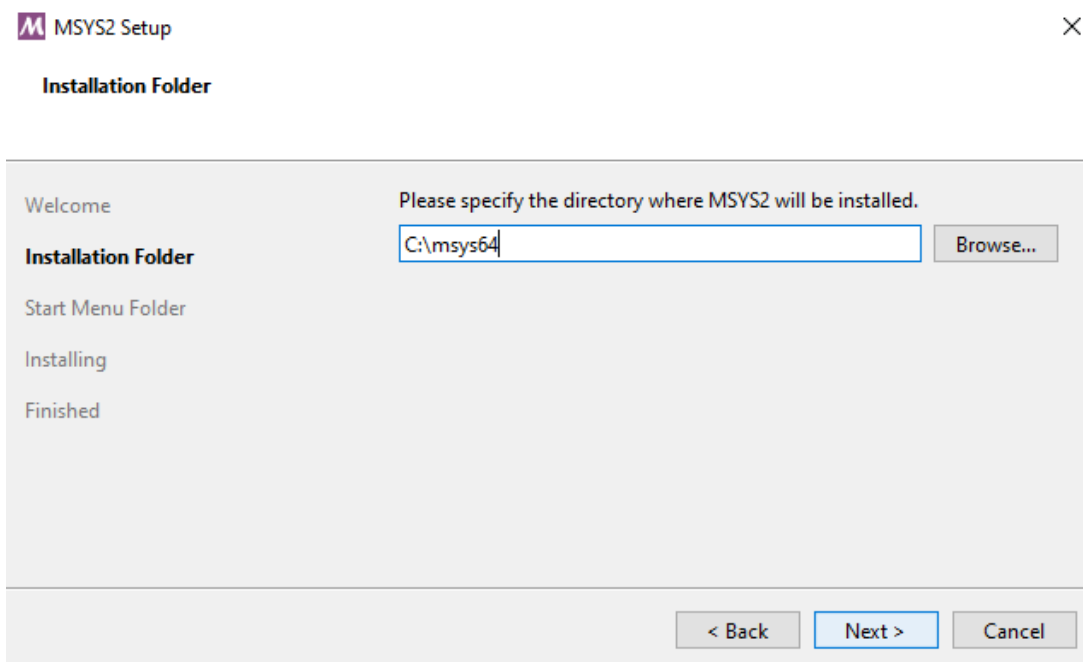
- Nakon preuzimanja exe file-a sa stranice neophodno je izvršiti instalaciju programa na računaru. Pokretanjem instalacijskog file-a otvoriti će se prozor kao na slici 2.19. Klikom na dugme Next pokreće se instalacija.



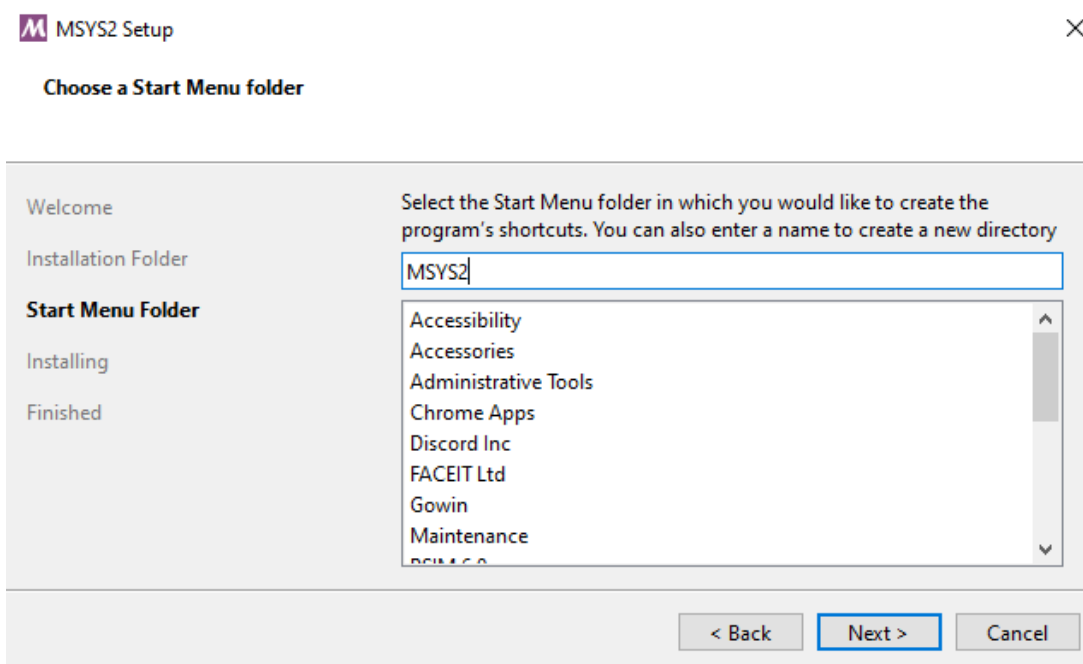
Slika 2.19: Pokretanje instalacije MSYS2 softverskog alata.

- Klikom na dugme Next instalacija prelazi na idući korak u kojem je potrebno izabrati lokaciju gdje će se izvršiti instalacija (slika 2.20). Nakon toga treba odabrati kraticu za Start Menu Folder kao na slici 2.21. Odrađivanjem prethodnih koraka pokreće se instalacija (slika 2.22). Nakon uspješne instalacije pojaviti će se prozor kao na slici 2.23. Pritiskom na dugme Finish završena je instalacija. Ako je označen checkbox za pokretanje MSYS2

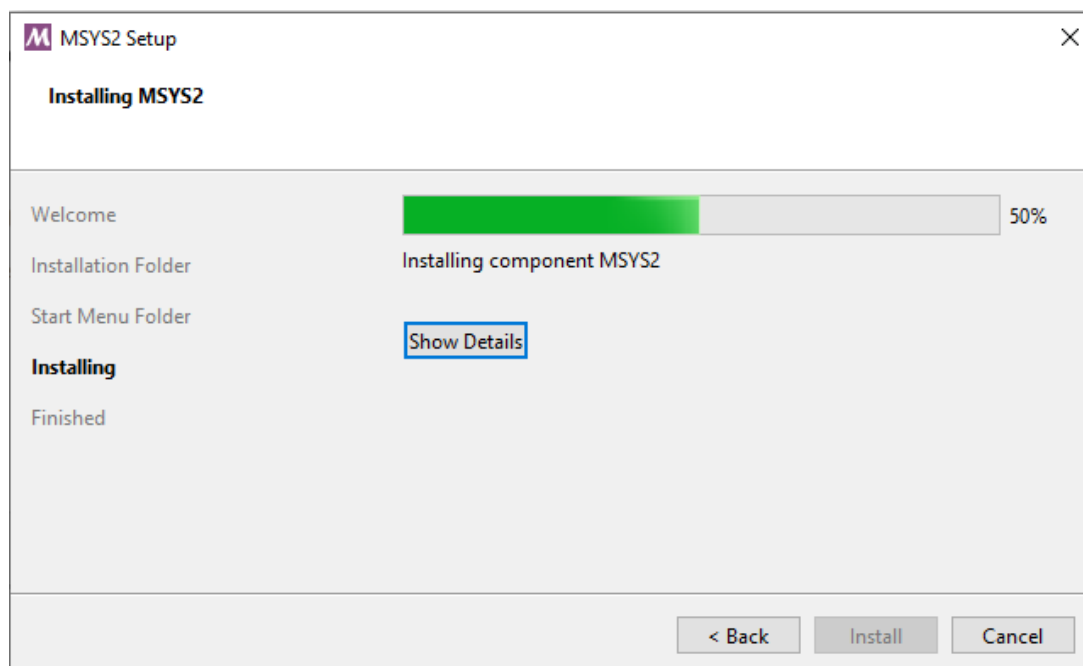
onda će se program pokrenuti pritiskom na dugme Finish, a ako nije mora se ponovno otvoriti.



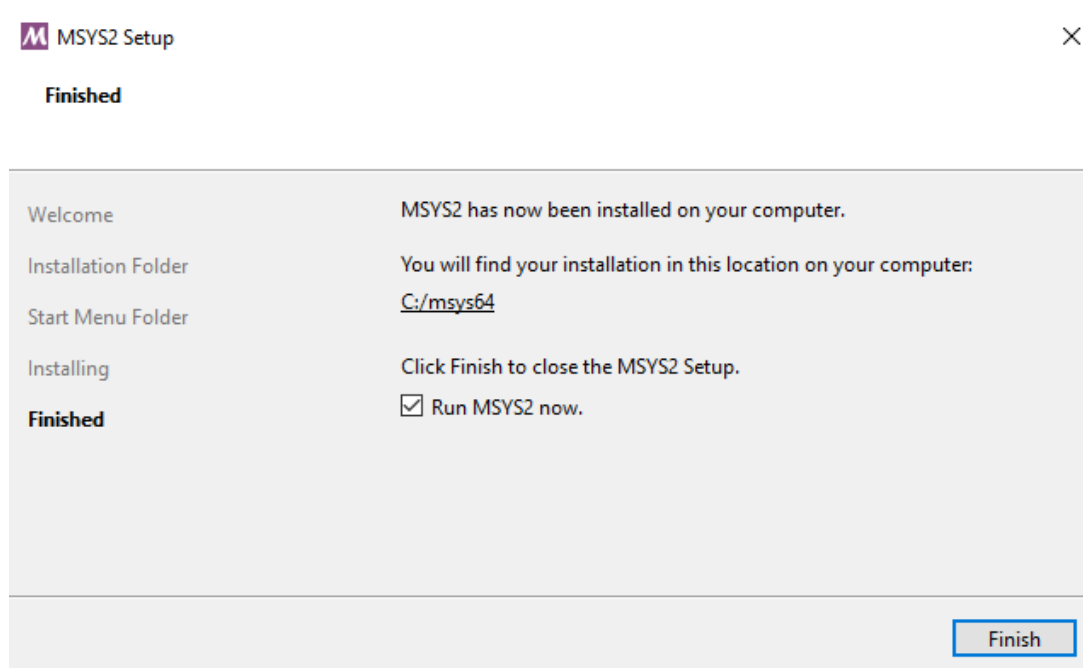
Slika 2.20: Odabir lokacije za instalaciju.



Slika 2.21: Odabir kratice za Start Menu Folder.



Slika 2.22: Pokretanje instalacije.



Slika 2.23: Završetak instalacije

- Nakon pokretanja otvoriti će se MSYS2 terminal kao na slici 2.24 u koji se trebaju upisati sljedeće komande. Komande se mogu unositi jedna po jedna ili sve odjednom. Nakon toga je potrebno pritiskati Y kada to terminal traži da bi se instalacija nastavila. Nakon završetka ovih komandi instalacija je završena i terminal se može zatvoriti. Time je završena instalacija svih softverskih alata koji će služiti za rad sa prethodno opisanim hardverom.

Kod 1 Instalacija potrebnih MSYS2 paketa.

```
$ pacman -S mingw-w64-x86_64-gcc
$ pacman -S make
$ pacman -S mingw-w64-x86_64-verilator
$ pacman -S mingw-w64-x86_64-gtkwave
```

```

User@DESKTOP-3UUJAE5 UCRT64 ~
$ pacman -S mingw-w64-x86_64-gcc
resolving dependencies...
looking for conflicting packages...

Packages (17) mingw-w64-x86_64-binutils-2.46-2 mingw-w64-x86_64-crt-13.0.0.r560.g3197fc7d6-1
mingw-w64-x86_64-gcc-libs-15.2.0-13 mingw-w64-x86_64-gettext-runtime-1.0-1
mingw-w64-x86_64-gmp-6.3.0-2 mingw-w64-x86_64-headers-13.0.0.r560.g3197fc7d6-1
mingw-w64-x86_64-isl-0.27-1 mingw-w64-x86_64-libiconv-1.19-1
mingw-w64-x86_64-libwinpthread-13.0.0.r560.g3197fc7d6-1 mingw-w64-x86_64-mpc-1.3.1-2
mingw-w64-x86_64-mpfr-4.2.2-1 mingw-w64-x86_64-tzdata-2026a-1
mingw-w64-x86_64-windows-default-manifest-6.4-4
mingw-w64-x86_64-winpthread-13.0.0.r560.g3197fc7d6-1 mingw-w64-x86_64-zlib-1.3.2-2
mingw-w64-x86_64-zstd-1.5.7-1 mingw-w64-x86_64-gcc-15.2.0-13

Total Download Size: 69.82 MiB
Total Installed Size: 550.19 MiB

:: Proceed with installation? [Y/n] y
:: Retrieving packages...
mingw-w64-x86_64-isl-0.27-... 1456.4 KiB 278 KiB/s 00:05 [#####] 100%
mingw-w64-x86_64-crt-13.0-... 4.6 MiB 291 KiB/s 00:16 [#####] 100%

```

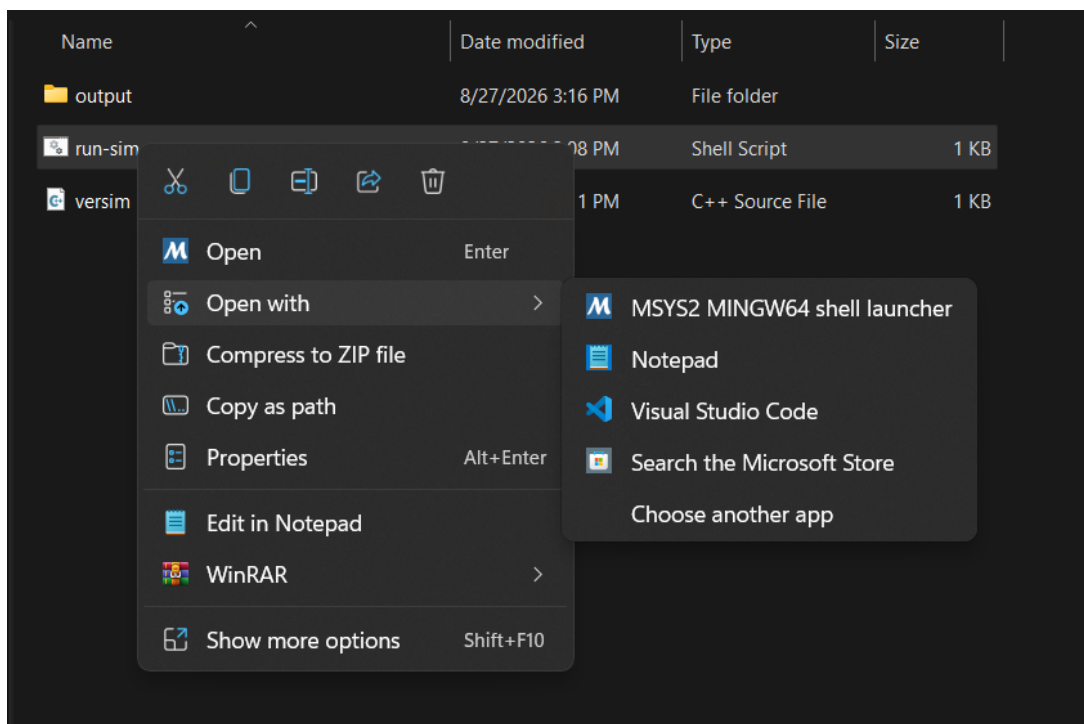
Slika 2.24: Unos komandi u MSYS2 terminal.

2.1.4 Konfiguracija programa za simulaciju

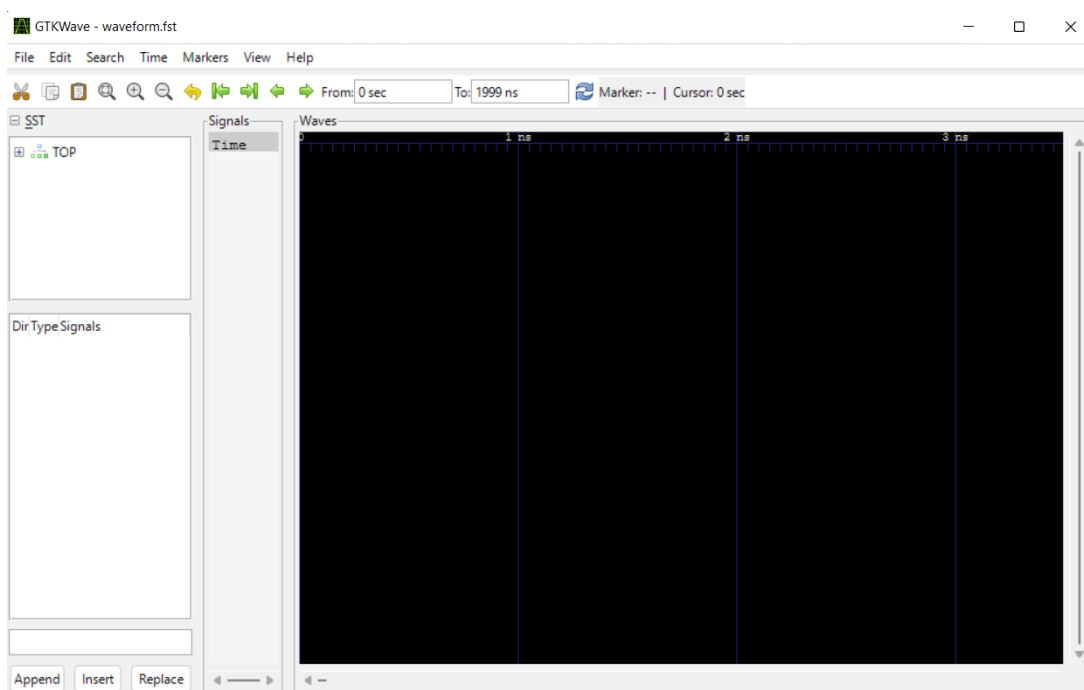
Nakon uspješne instalacije potrebno je instalirati Test file koji se nalazi na github repozitoriju. Raspakivanjem testnog file-a dobiju se dva foldera kao na slici 2.25. U gowin folderu se nalazi gowin program u kojem se piše kod logičke funkcije, a u sim folderu se nalazi bitan file pod nazivom run-sim. Desnim klikom na njega dobije se opcija da se pokrene pomoću MSYS2 terminala što se treba i uraditi kako bi se pokrenuo program za simulaciju (slika 2.26). Ukoliko je sve ispravno urađeno dobije se prozor kao na slici 2.27.

Name	Date modified	Type
gowin	8/27/2026 3:08 PM	File folder
sim	8/27/2026 3:16 PM	File folder

Slika 2.25: Raspakivanje Test foldera.



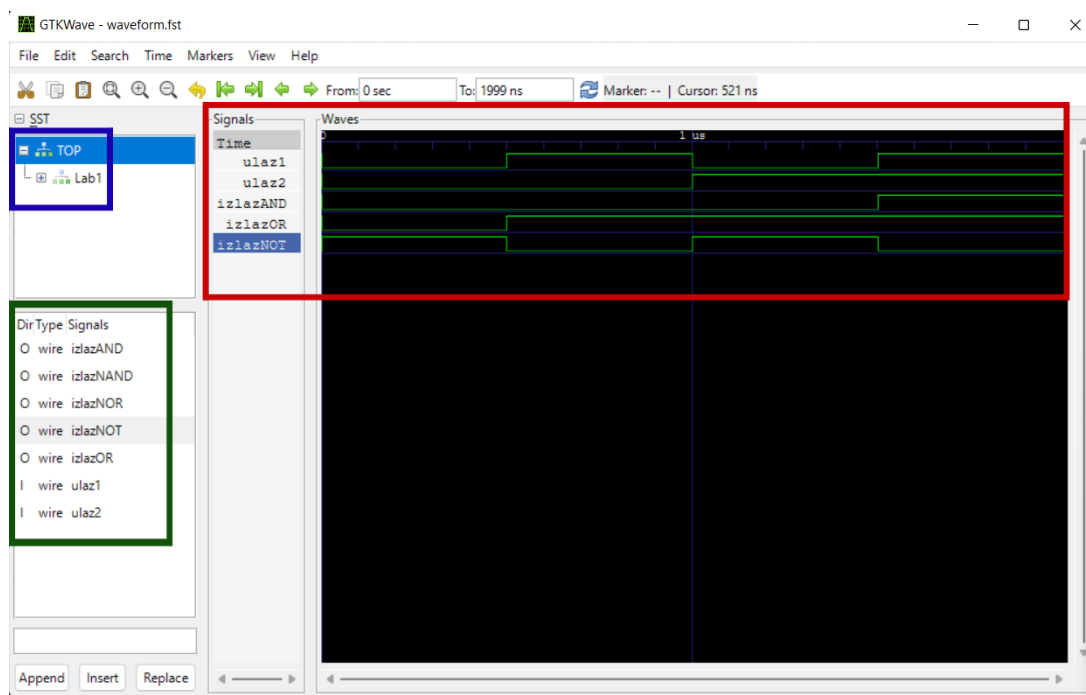
Slika 2.26: Pokretanje programa za simulaciju.



Slika 2.27: Program za simulaciju.

Na slici 2.28 se nalaze zaokružena tri najbitnija polja u programu za simulaciju. U plavom polju, klikom na znak plus pored naziva top modula u program se učitavaju sve ulazne i izlazne varijable koje su korištene u kodu i one će nakon učitavanja biti vidljive u zelenom polju. Sada pritiskom na neku od varijabli u zelenom polju njene vrijednosti će biti ispisane u crvenom polju, odnosno biti će ispisani grafici promjene te varijable u vremenu, a trajanje simulacije se

podešava u run-sim file-u, tako što se on otvori pomoću nekog tekst editora kao što je notepad ili VS code i promijeni se dužina trajanja na period koji je potreban za program koji se testira.



Slika 2.28: Najbitnija polja programa za simulaciju.

2.1.5 Programiranje u system verilogu

SystemVerilog je hardversko opisni jezik (engl.: *Hardware Description Language*, HDL) koji predstavlja proširenje klasičnog Verilog jezika, uveden kao standard IEEE 1800 [7]. Osim mogućnosti opisivanja digitalnih kola, SystemVerilog uvodi i napredne konstrukte za verifikaciju, objektno orijentisano programiranje i pisanje testbencheva, čime spaja projektovanje (*design*) i verifikaciju (*verification*) u jedan jezik [7].

Moduli i portovi

Osnovna gradivna jedinica u SystemVerilogu je `module`. Svaki modul predstavlja zaokruženu cjelinu digitalnog kola, sa jasno definisanim ulazima i izlazima:

Kod 2 SystemVerilog implementacija brojača.

```

1 module brojac (
2     input logic clk,
3     input logic reset,
4     output logic [3:0] izlaz
5 );
6
7     always_ff @(posedge clk or posedge reset) begin
8         if (reset)
9             izlaz <= 4'b0000;
10        else

```

```
11         izlaz <= izlaz + 1;  
12     end  
13  
14 endmodule
```

Moduli se mogu instancirati unutar drugih modula, čime se omogućava hijerarhijsko projektovanje složenijih sistema.

Tipovi podataka

SystemVerilog uvodi noviji i intuitivniji tip `logic`. Pored njega, dostupni su i:

- `bit` – dvovrijednosni tip (0 ili 1), bez stanja visoke impedanse ili nepoznatog stanja,
- `int`, `shortint`, `longint` – tipovi za rad sa cjelobrojnim vrijednostima,
- `struct` i `union` – za grupisanje više polja u jednu strukturu,
- dinamički nizovi i redovi (*queues*) – za fleksibilnije rukovanje podacima, posebno korisno pri pisanju testbencheva.

Proceduralni blokovi

SystemVerilog nudi tri specijalizovana proceduralna bloka koja jasno opisuju namjeru projektanta: `always_comb` za kombinacionu logiku, `always_ff` za sekvencijalnu logiku i `always_latch` za logiku zasnovanu na riglovima [7]. Za razumijevanje njihove uloge, ključno je poći od same prirode hardvera koji se njima opisuje.

`always_comb` Blok `always_comb` opisuje kombinacionu logiku – kolo čiji izlaz u svakom trenutku zavisi isključivo od trenutnih vrijednosti ulaza, bez ikakve memorije prethodnog stanja. Simulator automatski prepoznaje sve signale koji se čitaju unutar bloka i formira listu osjetljivosti (engl.: *sensitivity list*), tako da se blok ponovo izvršava svaki put kada se bilo koji od tih signala promijeni:

Kod 3 Primjer kombinacione logike korištenjem `always_comb` bloka.

```
1 always_comb begin  
2     if (selektor)  
3         izlaz = ulaz_a;  
4     else  
5         izlaz = ulaz_b;  
6 end
```

Pošto se lista osjetljivosti generiše automatski, izbjegava se čest izvor grešaka – zaboravljanje signala u listi, što bi u simulaciji moglo prouzrokovati da blok ne reaguje na promjenu ulaza iako bi u stvarnom (sintetizovanom) kolu reagovao [12]. Dodatno, alat za sintezu prilikom prevođenja `always_comb` bloka može provjeriti da li su svi izlazi definisani za svaku kombinaciju ulaza; ako nisu, upozorava projektanta da bi moglo doći do nenamjernog stvaranja rigla [13].

always_ff Blok `always_ff` opisuje sekvencijalnu logiku – kolo čiji se izlaz mijenja isključivo u tačno određenim trenucima, najčešće na ivicu (rastuću ili opadajuću) taktnog signala (engl.: *clock*). Za razliku od kombinacione logike, ovaj blok posjeduje memoriju: vrijednost izlaza se čuva između dva takta, sve dok se eksplicitno ne promijeni novim taktnim impulsom.

Kod 4 Primjer sekvencijalne logike korištenjem `always_ff` bloka.

```
1 always_ff @(posedge clk or posedge reset) begin
2     if (reset)
3         stanje <= 0;
4     else
5         stanje <= sljedece_stanje;
6 end
```

Ovakav blok se koristi za opisivanje flip-floпова, registara i konačnih automata (*finite state machines*). Ključna razlika u odnosu na `always_comb` blok jeste operator dodjele: unutar `always_ff` bloka koristi se neblokirajuća dodjela (`<=`), čime se osigurava da svi flip-floповi unutar istog takta "vide" stare vrijednosti signala prije nego što se izvrši ažuriranje – što tačno odgovara fizičkom ponašanju flip-floпова u stvarnom hardveru, gdje se svi okidaju istovremeno na istu ivicu takta [7].

Paralelizam u FPGA arhitekturi

Bitno je naglasiti da izvršavanje proceduralnih blokova u SystemVerilogu ne treba posmatrati kao sekvencijalno izvršavanje instrukcija, kakvo poznajemo iz softverskih programskih jezika [13]. Svaki `always_comb` i `always_ff` blok u kodu predstavlja zaseban, nezavisan dio hardvera koji fizički postoji na FPGA čipu i radi paralelno sa svim ostalim blokovima – svi se "izvršavaju" istovremeno, jer svaki odgovara stvarnom kolu (mreži logičkih kapija, multiplekserima, flip-floповima) implementiranom u programabilnim logičkim blokovima (engl.: *Configurable Logic Blocks*, CLB) FPGA uređaja.

Ovaj paralelizam se konkretno ogleda u sljedećem:

- Ako se u kodu nalazi deset odvojenih `always_ff` blokova, na FPGA čipu će nakon sinteze postojati deset fizički odvojenih grupa flip-floпова, koje se sve ažuriraju u istom trenutku, na istu ivicu takta – ne jedna za drugom, kako bi se moglo pomisliti čitajući kod odozgo nadole.
- Kombinaciona logika opisana kroz `always_comb` blokove se implementira kao mreža logičkih kapija čiji se izlazi ustaljuju (engl.: *propagate*) kroz kolo praktično trenutno (u okviru fizičkog kašnjenja propagacije signala), bez obzira na to u kom redoslijedu su blokovi napisani u izvornom kodu.
- Upravo zbog ove paralelne prirode, redoslijed pisanja blokova u SystemVerilog kodu nema uticaja na redoslijed njihovog "izvršavanja" u konačnom kolu – za razliku od softvera, gdje redoslijed linija koda direktno određuje redoslijed izvršavanja.
- Ova osobina je i razlog zašto FPGA uređaji mogu ostvariti veliku propusnost i predvidivo, deterministički kašnjenje kod obrade signala u stvarnom vremenu – svi dijelovi kola rade istovremeno, umjesto da dijele jedan procesor koji izvršava instrukcije jednu po jednu.

Razumijevanje ove razlike – da **SystemVerilog kod opisuje strukturu paralelnog hardvera, a ne slijed instrukcija** – predstavlja jedan od ključnih koncepata koje je potrebno usvojiti pri prelasku sa softverskog na hardversko projektovanje.

Zapisivanje brojeva

Brojčane vrijednosti u SystemVerilogu se zapisuju u formatu `broj_bita'brojni_sistem vrijednost`, gdje oznaka brojnog sistema može biti `b` (binarni), `d` (dekadni), `h` (heksadekadni) ili `o` (oktalni):

Kod 5 Primjeri zapisivanja brojeva u SystemVerilogu.

```
1'b1    // broj 1 zapisan u binarnom brojnom sistemu (1 bit)
1'b0    // broj 0 zapisan u binarnom brojnom sistemu (1 bit)
2'd3    // broj 3 zapisan u dekadnom brojnom sistemu (2 bita)
3'd6    // broj 6 zapisan u dekadnom brojnom sistemu (3 bita)
3'd7    // broj 7 zapisan u dekadnom brojnom sistemu (3 bita)
```

Prvi broj označava širinu (broj bita) rezultujuće vrijednosti – bitno je da ta širina bude dovoljna da stane odabrana vrijednost u odabranom brojnom sistemu, inače dolazi do odsijecanja viših bitova.

Instanciranje modula

Umjesto pisanja cijelog dizajna unutar jednog `.sv` fajla, RTL dizajn se uobičajeno razdvaja na više manjih, međusobno povezanih modula. Ovakav pristup ima više prednosti: izbjegava se nepotrebno ponavljanje koda, omogućava se pojedinačno testiranje svakog modula, više osoba može paralelno raditi na različitim dijelovima dizajna, a ispravljanje grešaka je jednostavnije jer je svaki modul manja, izolovana cjelina. Modul se instancira (poziva unutar drugog modula) na sljedeći način:

Kod 6 Primjer instanciranja modula unutar drugog (top) modula.

```
1 module MojModul (
2     input  logic x1,
3     output logic x2
4 );
5     // Dizajn modula
6 endmodule
7
8 module TopModul (
9     input  logic ulaz,
10    output logic izlaz
11 );
12     logic prvi_signal;
13     logic drugi_signal;
14
15     MojModul instanca (
16         .x1 (prvi_signal),
```

```

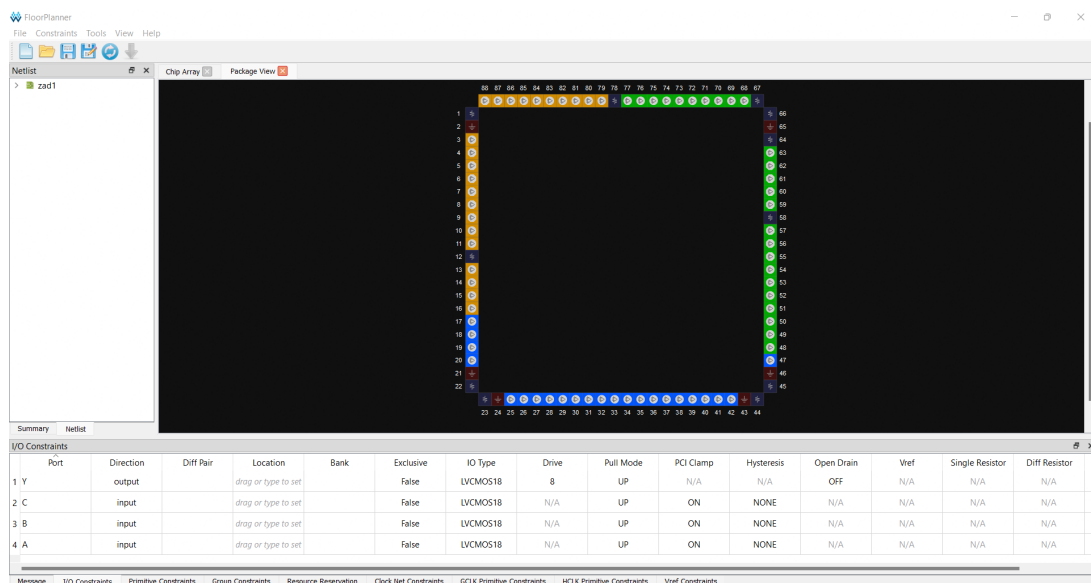
17         .x2 (drugi_signal)
18     );
19 endmodule

```

Pri instanciranju, svaki port modula se povezuje sa odgovarajućim signalom iz okružujućeg (nadređenog) modula preko notacije `.ime_porta(ime_signala)` – ovakav način povezivanja (engl.: *named port mapping*) je preporučen, jer redoslijed navođenja portova ne mora odgovarati redoslijedu u definiciji modula, čime se smanjuje mogućnost greške u odnosu na povezivanje portova isključivo prema pozicijskom redoslijedu. Ovaj princip je već korišten u prethodnim sekcijama, npr. prilikom instanciranja modula `T_ff` unutar strukturne implementacije Mealyevog i Mooreovog automata.

Dodjela fizičkih pinova (FloorPlanner)

Nakon uspješne sinteze RTL dizajna, potrebno je povezati ulazne i izlazne varijable definisane u top modulu sa konkretnim fizičkim pinovima na FPGA čipu. Ovo se u projektnom stablu (panel *Process*) prepoznaje po znaku upitnika ispred stavke *Place & Route* – sve dok se dodjela pinova ne izvrši, ovaj korak se ne može pokrenuti. Ulogu mapiranja ulaznih i izlaznih varijabli obavlja alat **FloorPlanner**, koji se pokreće klikom na istoimenu stavku u panelu *Process*.



Slika 2.29: Korisnički interfejs FloorPlanner alata.

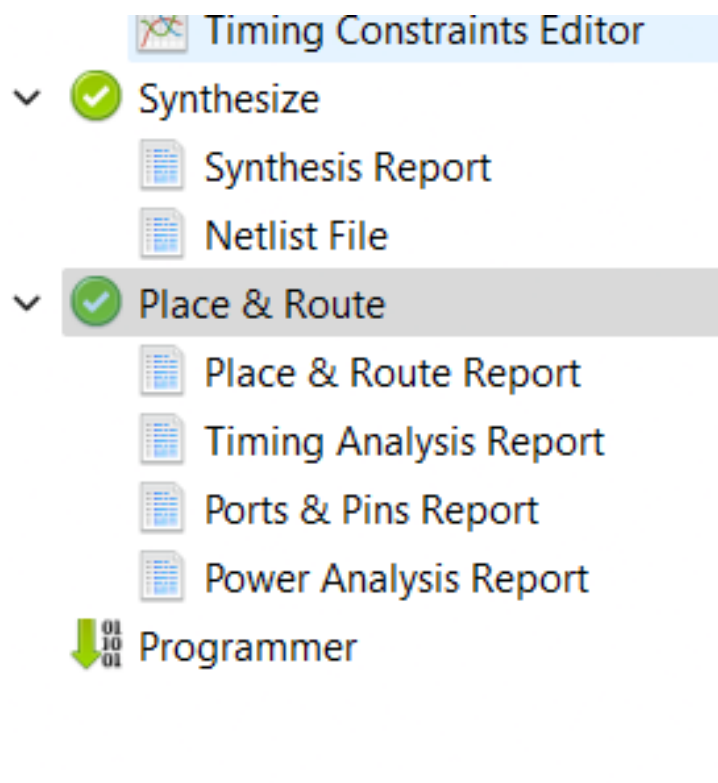
U FloorPlanner-u je za svaku ulaznu/izlaznu varijablu top modula potrebno u koloni **Port** prepoznati odgovarajuću varijablu, a zatim u koloni **Location** unijeti broj fizičkog pina na FPGA čipu na koji se ta varijabla povezuje (brojevi pinova preuzimaju se iz dokumentacije razvojne pločice, odnosno sa dijagrama rasporeda pinova prikazanog na slici 2.33). Pored fizičke lokacije, moguće je podesiti i dodatne parametre svakog pina:

- **Exclusive** – rezerviše određene resurse čipa (logičke blokove, memoriju i sl.) isključivo za tu namjenu, bez dijeljenja sa ostatkom dizajna;
- **I/O type** – određuje električni standard i naponski nivo pina (npr. LVCMOS33 odgovara naponu od 3.3V); vrijednost mora odgovarati naponu koji pločica dovodi u banku kojoj pin pripada;

- **Drive** – maksimalna izlazna struja pina, izražena u mA;
- **Pull Mode** – ponašanje pina kada nije aktivno vođen: *Pull-up* (povezuje se na logičku jedinicu), *Pull-down* (povezuje se na logičku nulu), ili *Keeper* (zadržava posljednje stanje);
- **Hysteresis** – određuje karakteristike prelaska ulaznog signala između naponskih nivoa (*H2L*, *L2H*, ili *none* za odsustvo histereze);
- **Open Drain** – konfiguracija izlaznog stepena bez internog pull-up tranzistora prema napajanju;
- **Slew Rate** – brzina promjene izlaznog napona (*FAST* ili *SLOW*).

Nakon što se sve postavke sačuvaju, FloorPlanner automatski generiše (ili ažurira) tekstualni fajl sa ekstenzijom `.cst` (engl.: *Physical Constraints File*), koji sadrži kompletnu dodjelu pinova u tekstualnom obliku. Ovaj fajl se može i naknadno ručno uređivati, bez potrebe za ponovnim pokretanjem grafičkog interfejsa FloorPlanner-a.

Kada su sva fizička ograničenja ispravno postavljena, pokretanje koraka *Place & Route* rezultira uspješnim završetkom cijelog procesa (sinteza, pozicioniranje i rutiranje), što se u panelu *Process* označava zelenim kvačicama pored svake stavke, umjesto prethodnog znaka upitnika.

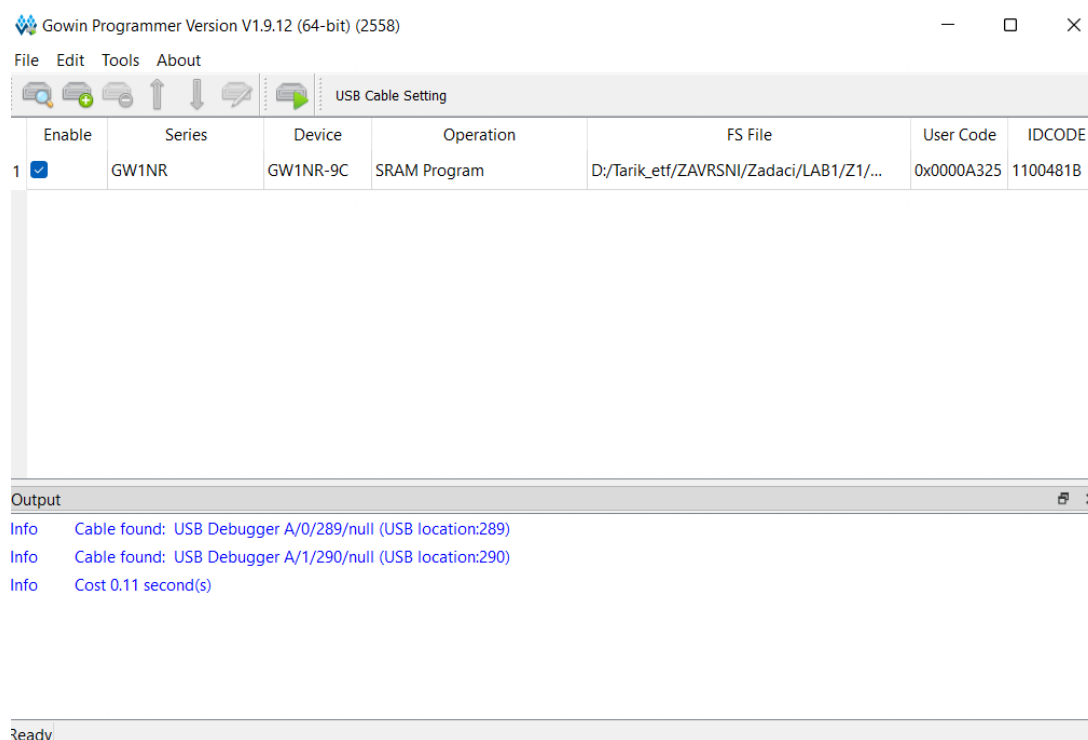


Slika 2.30: Uspješno izvršena sinteza, pozicioniranje i rutiranje.

Generisanje bitstream-a i programiranje FPGA čipa

Bitstream predstavlja binarni zapis koji u potpunosti definiše način na koji je FPGA čip konfigurisan da bi izvršavao željenu funkcionalnost – to je, drugim riječima, krajnji proizvod cijelog

procesa sinteze, pozicioniranja i rutiranja, spreman za upis u sam čip. Nakon uspješno završenog *Place & Route* koraka, generisanje bitstream-a i njegov prenos na fizički uređaj vrši se odabirom opcije **Program Device**, čime se otvara Gowin Programmer alat (slika 2.31).



Slika 2.31: Gowin Programmer – prozor za konfiguraciju programiranja FPGA čipa.

U ovom prozoru (slika 2.31), kolona **Enable** omogućava (ili onemogućava) programiranje odabranog čipa, kolone **Series** i **Device** prikazuju tačan model čipa na koji se vrši programiranje, dok kolona **Operation** definiše u koju memoriju čipa se bitstream upisuje. Bitna razlika postoji između dvije moguće opcije:

- **SRAM Program** – bitstream se upisuje direktno u isparljivu (engl.: *volatile*) SRAM konfiguracionu memoriju čipa. Ovakvo programiranje je brzo, ali se konfiguracija gubi čim čip izgubi napajanje;
- programiranje u **Flash memoriju** (ukoliko je čip posjeduje) – bitstream se trajno upisuje u neisparljivu memoriju, pa se pri sljedećem uključivanju napajanja čip automatski ponovo konfigurira bez potrebe za ponovnim povezivanjem na računar.

Posljednja kolona prikazuje apsolutnu putanju do generisanog bitstream fajla koji će biti upisan u odabranu memoriju čipa.

Struktura testbencha za simulaciju

Svaka simulacija u završnom radu oslanja se na dva fajla koja se nalaze u `sim` folderu svakog projekta: `versim.cpp` i `run-sim.sh`. Iako je konkretan sadržaj ova dva fajla prilagođen svakom pojedinačnom zadatku (vidjeti odgovarajuće laboratorijske vježbe), njihova opšta struktura i uloga ostaju iste kroz cijeli rad, pa se ovdje daje njihov opći pregled.

versim.cpp Ovaj fajl predstavlja C++ testbench u kojem se, korištenjem Verilator biblioteke, kreira instanca simuliranog SystemVerilog modula i definiše sekvenca vrijednosti koje se dodjeljuju njegovim ulazima tokom simulacije.

Kod 7 Opšta struktura versim.cpp fajla.

```
#include <stdlib.h>
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "VImeModula.h"
#include "VImeModula__024root.h"

#define MAX_SIM_TIME 2000
vluint64_t sim_time = 0;

int main(int argc, char** argv, char** env) {
    VImeModula *dut = new VImeModula;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;
    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");

    while (sim_time < MAX_SIM_TIME) {

        if (sim_time == 0) {
            dut->ulaz1 = 0;
            dut->ulaz2 = 0;
        }
        // ... dodatne promjene ulaza u odredjenim trenucima
        ...

        dut->eval();
        m_trace->dump(sim_time);
        sim_time++;
    }

    m_trace->close();
    delete dut;
    exit(EXIT_SUCCESS);
}
```

Nekoliko dijelova ovog koda je bitno napomenuti:

- `#include "VImeModula.h"` i `#include "VImeModula__024root.h"` – Verilator prilikom kompajliranja generiše C++ klasu čiji naziv odgovara imenu top modula, sa dodatim prefiksom V. Umjesto `ImeModula` potrebno je upisati tačno ime top

modula korištenog u konkretnom zadatku (vidljivo u *Hierarchy* panelu Gowin EDA softvera, gdje je top modul označen zelenom kvačicom);

- `MAX_SIM_TIME` – definiše ukupno trajanje simulacije u nanosekundama; `sim_time` predstavlja trenutno vrijeme simulacije, koje raste od 0 do `MAX_SIM_TIME`;
- `VImeModula *dut = new VImeModula;` – kreira pokazivač `dut` (*Device Under Test*) na instancu simuliranog modula. Dereferenciranjem ovog pokazivača (`dut->ime_ulaza`) moguće je direktno postaviti vrijednost bilo kojeg ulaznog porta modula;
- unutar petlje `while`, provjerom konkretnih vrijednosti `sim_time` definišu se trenuci u kojima se ulazne vrijednosti mijenjaju – izlazi modula se automatski ažuriraju u skladu sa napisanim SystemVerilog kodom, bez potrebe za eksplicitnim postavljanjem.

run-sim.sh Ova skripta pokreće Verilator, kompajlira SystemVerilog kod zajedno sa testbenchom, izvršava simulaciju, te automatski otvara GTKWave alat radi pregleda rezultata.

Kod 8 Opšta struktura run-sim.sh skripte.

```
LAB_NAME="ImeModula"

rm -rf output
mkdir output
cd output

verilator \
  --top-module ${LAB_NAME} \
  -Wall \
  +define+SIM_ONLY \
  --timescale 1ns/1ns \
  --trace-fst \
  --cc \
  ../../gowin/src/*.sv \
  --exe versim.cpp \
  --Mdir .

make -f V${LAB_NAME}.mk V${LAB_NAME}
./V${LAB_NAME}

gtkwave waveform.fst
```

Jedina izmjena koju je potrebno napraviti prilikom prelaska sa jednog zadatka na drugi jeste postavljanje varijable `LAB_NAME` na tačno ime top modula – ova vrijednost se koristi kroz cijelu skriptu za automatsko generisanje odgovarajućih Verilator naredbi i imena izlaznih fajlova, pa mora odgovarati imenu klase korištene u `versim.cpp` fajlu.

Nakon pokretanja skripte, Verilator kompajlira SystemVerilog kod u C++ model, izvršava simulaciju u trajanju definisanom testbenchom, generiše fajl `waveform.fst` sa zapisom svih signala tokom simulacije, te automatski otvara GTKWave alat u kojem se dobijeni rezultati

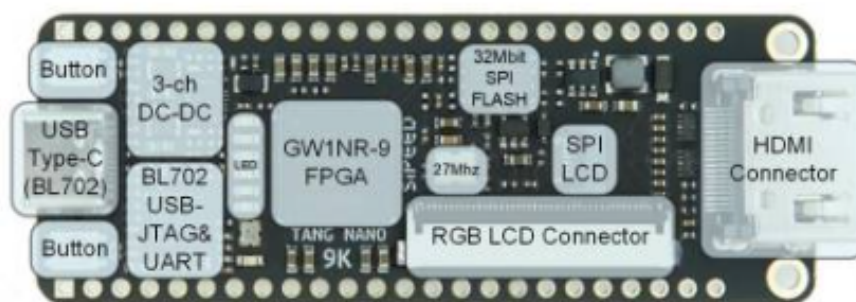
mogu vizuelno pregledati i uporediti sa očekivanim ponašanjem definisanim tablicom istine, odnosno tablicom prelaza i izlaza.

2.2 Korišteni hardver

U fokusu praktičnog dijela završnog rada nalazi se **Tang Nano 9K** razvojni sistem, baziran na Gowin LittleBee familiji FPGA čipova. Ovu platformu proizvodi kompanija Sipeed, a dizajnirana je da pruži kompaktan, ali efikasan okvir za razvoj digitalnih sistema, čineći je idealnom za edukacijske svrhe i izradu praktikuma.

2.2.1 Tang Nano 9K razvojni sistem

Osnovu ploče čini **GW1NR-9** FPGA čip koji integriše 8640 logičkih jedinica (LUT4) i dodatne resurse poput ugrađene statičke memorije (engl.: Block Static Random-Access Memory, BSRAM) i DSP blokova. Ono što Tang Nano 9K izdvaja od sličnih razvojnih sistema u ovom cjenovnom rangu je integracija bogate periferije na samoj ploči, uključujući HDMI konektor, slot za MicroSD karticu i podršku za RGB LCD ekrane. Na slici 2.32 prikazan je raspored ključnih komponenti, uključujući USB-C port koji služi za napajanje i JTAG programiranje, te BL702 čip koji obavlja funkciju USB-to-UART i USB-to-JTAG mosta.



Slika 2.32: Hardverska arhitektura Tang Nano 9K razvojnog sistema.

2.2.2 Tehničke specifikacije i Pinout

Za uspješnu implementaciju projekata unutar praktikuma, neophodno je poznavanje dostupnih resursa FPGA čipa, koji su sumirani u tabeli 2.1.

Tabela 2.1: Tehničke karakteristike GW1NR-9 FPGA čipa.

Resurs	Kapacitet / Opis
Logičke jedinice (LUT4)	8640
Flip-Flop (FF)	6480
Shadow RAM (SRAM)	17280 bita
Block RAM (BSRAM)	468 Kbits
Korisnička Flash memorija	608 Kbits
DSP blokovi	Integrisani (18x18 multiplikatori)
Radni takt	27 MHz (oscilator na ploči)

*

Poglavlje 3

Laboratorijske vježbe i praktična implementacija

U ovom poglavlju predstavljen je niz jasno postavljenih laboratorijskih vježbi, koncipiranih tako da studenta postepeno uvedu u praktičan rad sa FPGA razvojnim sistemima. Svaka vježba je zamišljena kao samostalna, ali logički povezana cjelina, pri čemu se znanje i vještine stečene u prethodnoj vježbi nadograđuju u narednoj – od najjednostavnijih kombinacionih kola, preko sekvencijalne logike, do složenijih upravljačkih modula.

Cilj ovog dijela praktikuma nije samo demonstracija pojedinačnih rješenja, već sistematsko usvajanje osnovnih vještina koje su preduslov za bilo kakav ozbiljniji, samostalni rad na FPGA projektima – bilo u akademskom, bilo u industrijskom okruženju. Kroz vježbe se obrađuju teme poput opisivanja kombinacione i sekvencijalne logike u SystemVerilogu, sinteze i implementacije dizajna na Tang Nano 9K razvojnom sistemu [9], verifikacije ponašanja kola putem simulacije, kao i rada sa osnovnim ulazno-izlaznim perifernim komponentama.

Namjera je da student, nakon što prođe kroz sve vježbe u ovom poglavlju, samostalno vlada kompletnim tokom rada sa FPGA sistemom – od pisanja i simulacije koda, preko sinteze i implementacije, do konačnog testiranja na fizičkom hardveru – i da na taj način stekne čvrstu osnovu neophodnu za suočavanje sa zahtjevnijim, realnim inženjerskim problemima u oblasti digitalnog dizajna.

3.1 Laboratorijska vježba broj 1

3.1.1 Zadatak 1 – Implementacija kombinacione logičke funkcije

U okviru prvog zadatka potrebno je implementirati kombinacionu logičku funkciju definisanu izrazom:

$$Y = \bar{A}BC \vee \overline{A \vee C} \quad (3.1)$$

gdje su A , B i C ulazne binarne varijable, a Y izlaz kola. Zadatak studenta je da na osnovu datog logičkog izraza izvede tablicu istine, implementira funkciju u SystemVerilogu, te provjeri ispravnost implementacije putem simulacije prije prebacivanja koda na FPGA razvojni sistem.

Primjenom De Morganovog zakona na drugi član izraza (3.1) dobija se $\overline{A \vee C} = \bar{A}\bar{C}$, čime se funkcija svodi na $Y = \bar{A}BC \vee \bar{A}\bar{C} = \bar{A}(B \vee \bar{C}) = \bar{A}$. Ovo pojednostavljenje studentu služi kao provjera ispravnosti tablice istine i simulacionih rezultata – očekuje se da je izlaz Y jednak jedinici isključivo u slučajevima kada je $A = 0$ i $C = 1$, nezavisno od vrijednosti varijable B .

Tabela 3.1: Tablica istine za logičku funkciju $Y = \bar{A}BC \vee A\bar{B}\bar{C}$.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Implementacija date logičke funkcije izvedena je korištenjem bloka `always_comb`, s obzirom na to da se radi o čisto kombinacionoj logici bez memorijskog elementa:

Kod 9 SystemVerilog implementacija logičke funkcije zadatka 1.

```
1 module zad1 (  
2     input logic A,  
3     input logic B,  
4     input logic C,  
5     output logic Y  
6 );  
7  
8     always_comb begin  
9         Y = (~A & B & C) | ~(A | ~C);  
10    end  
11  
12 endmodule
```

Nakon pisanja koda, potrebno je izvršiti simulaciju putem Verilator alata, uporediti dobijene rezultate sa tablicom istine 3.1, te tek nakon potvrde ispravnosti implementaciju prebaciti na Tang Nano 9K razvojni sistem radi konačnog testiranja na fizičkom hardveru.

Simulacija putem Verilator alata

Prije prebacivanja rješenja na fizički FPGA uređaj, potrebno je verifikovati ispravnost napisanog koda putem simulacije, korištenjem prethodno instaliranog Verilator alata. U tu svrhu se prvo kreira novi projekat na način opisan u poglavlju 2.1.2, pri čemu je ključno da se `.sv` fajl kreiran tokom otvaranja projekta nazove `zad1` – ime top modula u kodu i ime samog fajla se moraju poklapati, jer se to ime dalje koristi i pri konfiguraciji simulacionog okruženja. Za potrebe ovog zadatka dovoljan je jedan jedini modul, u koji se unosi kod koji opisuje zadatu logičku funkciju, nakon čega se fajl obavezno sprema.

Nakon što je `.sv` fajl sačuvan, iz ranije preuzetog test fajla se folder `sim` kopira u direktorij samog projekta. Konačna struktura foldera mora izgledati tako da se kreirani projekat nalazi u folderu `gowin`, dok se neposredno pored njega (na istom nivou) nalazi zalijepljeni folder `sim`, namijenjen simulaciji.

Unutar foldera `sim` potrebno je izmijeniti dva fajla. Prvi je skripta `run-sim`, koju je najlakše otvoriti u nekom code ili tekst editoru (preporučuje se VS Code). U njoj je potrebno promijeniti vrijednost varijable `LAB_NAME` tako da odgovara imenu top modula – u ovom slučaju `zad1` – kao što je prikazano u Kodu 10. Ova varijabla se dalje koristi kroz cijelu skriptu za automatsko generisanje odgovarajućih Verilator naredbi, imena izlaznih fajlova i pokretanje simulacije, pa je njena tačna vrijednost preduslov da bi skripta uopšte mogla pronaći i kompajlirati odgovarajući modul.

Drugi fajl koji je potrebno prilagoditi jeste testbench fajl `versim.cpp`, prikazan u Kodu 11. Prva i obavezna izmjena jeste zamjena svih instanci `VLab1` sa `Vzad1` – ovo je posljedica toga što Verilator prilikom kompajliranja generiše C++ klasu čiji naziv odgovara imenu top modula, sa prefiksom `V` (u ovom slučaju `Vzad1`), pa bez ove izmjene simulacija neće moći biti pokrenuta jer traženi tip klase ne bi postojao. Nakon toga je potrebno podesiti i vremenski tok simulacije tako da se jasno mogu razlučiti sve kombinacije ulaznih varijabli i pripadajući izlazi. Pošto se u ovom zadatku koriste samo tri ulazne varijable (A , B i C), svih osam mogućih kombinacija može se jednostavno definisati ručno, direktno unutar `versim.cpp` fajla, dodjeljivanjem odgovarajućih vrijednosti u unaprijed određenim vremenskim trenucima simulacije, kao što je vidljivo u kodu ispod.

Nakon pokretanja skripte `run-sim`, Verilator kompajlira SystemVerilog kod u C++ model, izvršava simulaciju u trajanju definisanom u `versim.cpp` fajlu, generiše `waveform.fst` fajl sa zapisom svih signala tokom simulacije, te automatski otvara GTKWave alat u kojem se dobijeni rezultati mogu vizuelno uporediti sa tablicom istine 3.1.

Kod 10 Skripta `run-sim` prilagođena za pokretanje simulacije zadatka 1.

```
LAB_NAME="zad1"

%-----

rm -rf output
mkdir output
cd output

verilator \
  --top-module ${LAB_NAME} \
  -Wall \
  +define+SIM_ONLY \
  --timescale 1ns/1ns \
  --trace-fst \
  --cc \
  ../../gowin/src/*.sv \
  --exe versim.cpp \
  --Mdir .

make -f V${LAB_NAME}.mk V${LAB_NAME}

./V${LAB_NAME}

%verilator ../../1.gowin/src/${LAB_NAME}.sv -xml-only --bbox-
  sys --Mdir .
```

```
%xml2stems V${LAB_NAME}.xml V${LAB_NAME}.stems
%gtkwave --stems V${LAB_NAME}.stems waveform.fst &

gtkwave waveform.fst
```

Kod 11 Testbench fajl `versim.cpp` sa definisanim testnim vektorima za sve kombinacije ulaza *A*, *B* i *C*.

```
#include <stdlib.h>
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "Vzad1.h"
#include "Vzad1__024root.h"

#define MAX_SIM_TIME 4000
vluint64_t sim_time = 0;

double sc_time_stamp() {
    return sim_time;
}

int main(int argc, char** argv, char** env) {
    Vzad1 *dut = new Vzad1;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;

    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");

    while (sim_time < MAX_SIM_TIME) {

        if(sim_time == 0)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
        }

        if(sim_time == 500)
        {
            dut->A = 1;
            dut->B = 0;
            dut->C = 0;
        }
    }
}
```

```
    if(sim_time == 1000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 0;
    }

    if(sim_time == 1500)
    {
        dut->A = 1;
        dut->B = 1;
        dut->C = 0;
    }

    if(sim_time == 2000)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
    }

    if(sim_time == 2500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
    }

    if(sim_time == 3000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
    }

    if(sim_time == 3500)
    {
        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
    }

    dut->eval();
    m_trace->dump(sim_time);
    sim_time++;
}

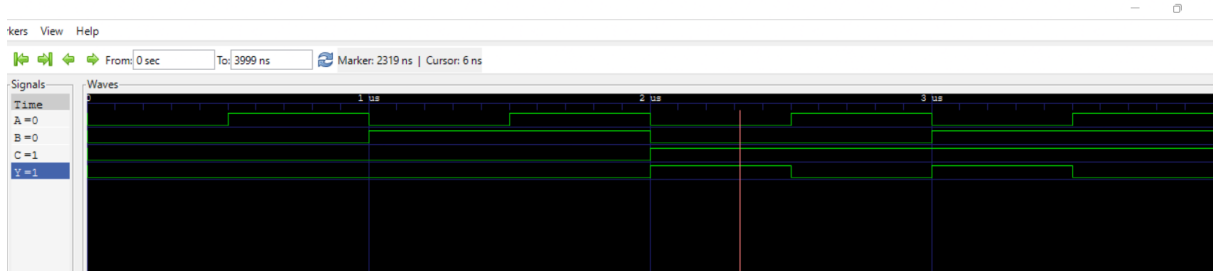
m_trace->close();
delete dut;
```

```

    exit(EXIT_SUCCESS);
}

```

Na slici 3.1 prikazani su rezultati simulacije prvog zadatka. Može se uočiti da izlaz logičke funkcije poprima vrijednost logičke jedinice isključivo kada je $C = 1$ i $A = 0$, nezavisno od vrijednosti ulaza B . Prema tome, postoje dvije kombinacije ulaznih vrijednosti za koje je izlaz jednak logičkoj jedinici, što je u potpunosti u skladu s tablicom istine prikazanom u tabeli 3.1.



Slika 3.1: Simulacija za zadatak 1 iz laboratorijske vježbe broj 1.

3.1.2 Zadatak 2 – Implementacija kombinacione logičke funkcije sa pet promjenjivih

U okviru drugog zadatka potrebno je implementirati kombinacionu logičku funkciju sa pet ulaznih promjenjivih, definisanu izrazom:

$$Y = \bar{A}BCD \vee A \vee \bar{E} \vee B\bar{C}\bar{D}\bar{E} \quad (3.2)$$

gdje su A, B, C, D i E ulazne binarne varijable, a Y izlaz kola. Kao i u prethodnom zadatku, cilj je da student na osnovu datog izraza izvede tablicu istine, implementira funkciju u SystemVerilogu, te potvrdi ispravnost implementacije putem simulacije prije prebacivanja koda na FPGA razvojni sistem. Povećanjem broja ulaznih varijabli sa tri na pet, broj mogućih kombinacija ulaza raste sa $2^3 = 8$ na $2^5 = 32$, čime se student dodatno uvježbava u sistematičnom pristupu izvođenju tablice istine za funkcije sa većim brojem ulaza.

Primjenom De Morganovog zakona na drugi član izraza (3.2) dobija se $\overline{A \vee \bar{E}} = \bar{A}E$, čime se funkcija svodi na:

$$Y = \bar{A}BCD \vee \bar{A}E \vee B\bar{C}\bar{D}\bar{E} \quad (3.3)$$

Iz pojednostavljenog izraza (3.3) se dodatno može uočiti da je izlaz Y jednak jedinici u sljedeća tri slučaja: kada je $A = 0$ i istovremeno $B = C = D = 1$; kada je $A = 0$ i $E = 1$ (nezavisno od preostalih varijabli); te kada je $B = 1, C = 0, D = 1$ i $E = 0$ (nezavisno od varijable A). Ovo zapažanje studentu služi kao brz i pouzdan način provjere ispravnosti tablice istine i kasnijih simulacionih rezultata, bez potrebe za provjerom svih 32 kombinacija ponasob.

Tabela 3.2: Tablica istine za logičku funkciju $Y = \bar{A}BCD \vee \overline{A \vee \bar{E}} \vee B\bar{C}\bar{D}\bar{E}$.

A	B	C	D	E	Y
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1

A	B	C	D	E	Y
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Implementacija date logičke funkcije u SystemVerilogu, kao i u prethodnom zadatku, izvedena je korištenjem bloka `always_comb`, s obzirom na to da se i ovdje radi o čisto kombina-
cionoj logici bez memorijskog elementa:

Kod 12 SystemVerilog implementacija logičke funkcije zadatka 2.

```

1 module zad2 (
2     input  logic A,
3     input  logic B,
4     input  logic C,
5     input  logic D,
6     input  logic E,
7     output logic Y
8 );
9
10    always_comb begin
11        Y = (~A & B & C & D) | ~(A | ~E) | (B & ~C & D & ~E);
12    end
13
14 endmodule

```

Nakon pisanja koda, potrebno je izvršiti simulaciju putem Verilator alata, uporediti dobijene rezultate sa tablicom istine 3.2, te tek nakon potvrde ispravnosti implementaciju prebaciti na Tang Nano 9K razvojni sistem radi konačnog testiranja na fizičkom hardveru.

Simulacija putem Verilator alata

Kao i u prethodnom zadatku, prije prebacivanja rješenja na fizički FPGA uređaj potrebno je verifikovati ispravnost napisanog koda putem simulacije. Postupak je identičan onom opisanom

u zadatku 1, s tim da je ovdje ključno da se `.sv` fajl kreiran tokom otvaranja projekta nazove `zad2`, u skladu sa imenom top modula. Nakon kreiranja projekta i pisanja koda koji opisuje logičku funkciju (3.2), folder `sim` se ponovo kopira iz test fajla u direktorij projekta, na isti način kao u zadatku 1.

Unutar foldera `sim` ponovo je potrebno izmijeniti dva fajla. U skripti `run-sim` varijabla `LAB_NAME` se mijenja u `zad2`, kao što je prikazano u Kodu 13. U testbench fajlu `versim.cpp`, prikazanom u Kodu 14, sve instance `Vzad1` (ili `VLab1`, u zavisnosti od toga sa koje se osnove kreće) potrebno je zamijeniti sa `Vzad2`, iz istog razloga kao i ranije – naziv generisane Verilator C++ klase mora odgovarati imenu top modula.

Dodatna razlika u odnosu na zadatak 1 proizilazi iz činjenice da funkcija sada ima pet ulaznih varijabli (*A*, *B*, *C*, *D* i *E*) umjesto tri, čime broj mogućih kombinacija raste sa 8 na 32. U skladu s tim, testbench fajl je proširen tako da definiše svih 32 kombinacija ulaznih vrijednosti, svaku u trajanju od 500 ns, čime je ukupno vrijeme trajanja simulacije (`MAX_SIM_TIME`) produženo na 16000 ns kako bi sve kombinacije stale u simulaciju. Redoslijed kombinacija u testbenchu odgovara redoslijedu kombinacija u tablici istine 3.2 (standardno binarno brojanje od 00000 do 11111), čime se rezultati simulacije mogu direktno, red po red, uporediti sa tablicom istine.

Kod 13 Skripta `run-sim` prilagođena za pokretanje simulacije zadatka 2.

```
LAB_NAME="zad2"

%-----

rm -rf output
mkdir output
cd output

verilator \
  --top-module ${LAB_NAME} \
  -Wall \
  +define+SIM_ONLY \
  --timescale 1ns/1ns \
  --trace-fst \
  --cc \
  ../../gowin/src/*.sv \
  --exe versim.cpp \
  --Mdir .

make -f V${LAB_NAME}.mk V${LAB_NAME}

./V${LAB_NAME}

gtkwave waveform.fst
```

Kod 14 Testbench fajl `versim.cpp` sa definisanim testnim vektorima za svih 32 kombinacije ulaza *A*, *B*, *C*, *D* i *E*.

```
#include <stdlib.h>
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "Vzad2.h"
#include "Vzad2___024root.h"

#define MAX_SIM_TIME 16000
vluint64_t sim_time = 0;

double sc_time_stamp() {
    return sim_time;
}

int main(int argc, char** argv, char** env) {
    Vzad2 *dut = new Vzad2;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;

    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");

    while (sim_time < MAX_SIM_TIME) {

        if(sim_time == 0)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
            dut->D = 0;
            dut->E = 0;
        }

        if(sim_time == 500)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
            dut->D = 0;
            dut->E = 1;
        }

        if(sim_time == 1000)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
        }
    }
}
```

```
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 1500)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 0;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 2000)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 2500)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 3000)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 3500)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }
}
```

```
    if(sim_time == 4000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 0;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 4500)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 0;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 5000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 0;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 5500)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 0;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 6000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 6500)
    {
```

```
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 7000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 7500)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 8000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 8500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 9000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
```

```
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 9500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 10000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 10500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 11000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 11500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }
}
```

```
if(sim_time == 12000)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 0;
    dut->E = 0;
}

if(sim_time == 12500)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 0;
    dut->E = 1;
}

if(sim_time == 13000)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 1;
    dut->E = 0;
}

if(sim_time == 13500)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 1;
    dut->E = 1;
}

if(sim_time == 14000)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 1;
    dut->D = 0;
    dut->E = 0;
}

if(sim_time == 14500)
{
```

```

        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 15000)
    {
        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

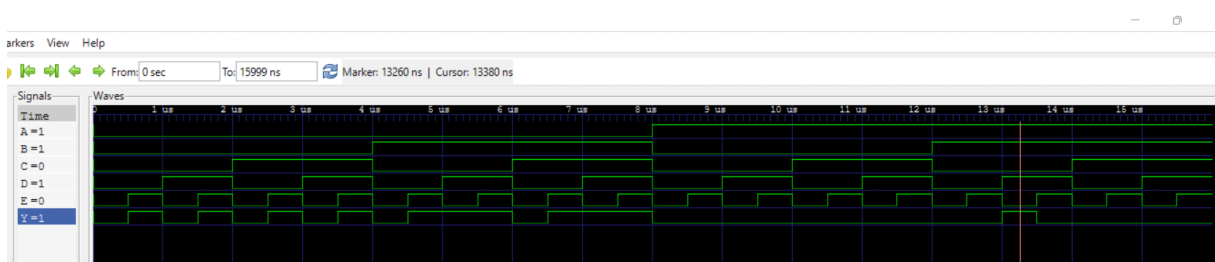
    if(sim_time == 15500)
    {
        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }

    dut->eval();
    m_trace->dump(sim_time);
    sim_time++;
}

m_trace->close();
delete dut;
exit(EXIT_SUCCESS);
}

```

Na slici 3.2 prikazani su rezultati simulacije drugog zadatka. Izlaz Y poprima vrijednost logičke jedinice tačno u onim vremenskim intervalima koji odgovaraju kombinacijama ulaza za koje je, prema pojednostavljenom izrazu (3.3), ispunjen bar jedan od uslova $\bar{A}BCD$, $\bar{A}E$ ili $B\bar{C}D\bar{E}$, što je u potpunosti u skladu s tablicom istine prikazanom u tabeli 3.2.



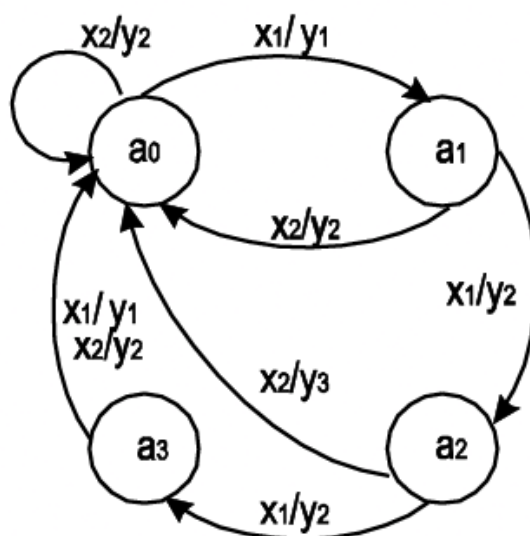
Slika 3.2: Simulacija za zadatak 2 iz laboratorijske vježbe jedan

3.2 Laboratorijska vježba 2

U okviru ove laboratorijske vježbe implementiraju se dva konačna automata – Mealyev i Mooreov. Za oba automata izvodi se tablica prelaza i izlaza na osnovu zadanog grafa, provjerava se mogućnost minimizacije broja unutrašnjih stanja, vrši se kodiranje stanja i signala, te se automat implementira na dva načina: korištenjem isključivo tablice prelaza i izlaza (bihevioralni opis), te korištenjem strukturne sheme automata zasnovane na T flip-flopovima.

3.2.1 Zadatak 1 – Mealyev automat

Graf koji opisuje funkcionalnost automata prikazan je na slici 3.3.



Slika 3.3: Graf Mealyevog automata (zadatak 1).

Automat ima četiri unutrašnja stanja (a_0, a_1, a_2, a_3), dva ulazna signala (x_1, x_2) i tri moguća izlazna signala (y_1, y_2, y_3). Na osnovu grafa se dobija tablica prelaza i izlaza, data u tabeli 3.3.

Tabela 3.3: Tablica prelaza i izlaza (zadatak 1).

$x_n a_r$	a_0	a_1	a_2	a_3
x_1	a_1/y_1	a_2/y_2	a_3/y_2	a_0/y_1
x_2	a_0/y_2	a_0/y_2	a_0/y_3	a_0/y_2

Provjera minimizacije broja stanja

Prije prelaska na strukturnu sintezu, potrebno je provjeriti postoje li ekvivalentna stanja koja bi se mogla objediniti. Stanja a_0 i a_3 imaju identičan izlazni par (y_1, y_2) za ulaze (x_1, x_2), pa predstavljaju kandidate za spajanje. Međutim, da bi $a_0 \equiv a_3$ vrijedilo, potrebno je da njihovi nasljednici po istom ulazu takođe budu ekvivalentni – za x_1 , a_0 prelazi u a_1 , a a_3 prelazi u a_0 , pa je uslov ekvivalencije $a_1 \equiv a_0$. Kako a_1 ima izlazni par (y_2, y_2), različit od izlaznog para stanja a_0 , uslov $a_1 \equiv a_0$ nije ispunjen, pa a_0 i a_3 nisu ekvivalentna stanja. Provjerom preostalih parova stanja utvrđuje se da automat ne sadrži ekvivalentna stanja, te je već zadan u minimalnom obliku sa četiri stanja.

Kodiranje signala

Pošto automat ima četiri stanja, za realizaciju registra stanja potrebna su dva T flip-flopa (Q_1 , Q_0). Kodiranje stanja, ulaza i izlaza dato je u tabelama 3.4–3.6.

Tabela 3.4: Kodirana stanja.

	Q_0	Q_1
a_0	0	0
a_1	0	1
a_2	1	0
a_3	1	1

Tabela 3.5: Kodirani ulaz.

	U
x_1	0
x_2	1

Tabela 3.6: Kodirani izlazi.

	Z_1	Z_2
y_1	0	1
y_2	1	0
y_3	1	1

Prošireno kodirana tablica i logičke funkcije

Na osnovu kodiranih tablica formira se proširena kodirana tablica prelaza i izlaza, data u tabeli 3.7, gdje se za svaku kombinaciju $U(t)$, $Q_0(t)$ i $Q_1(t)$ određuje sljedeće stanje $Q_0(t+1)$, $Q_1(t+1)$, potrebni pobudni signali T flip-flopora ($T_1 = Q_1(t) \oplus Q_1(t+1)$, $T_2 = Q_0(t) \oplus Q_0(t+1)$), te kodirani izlazi Z_1 , Z_2 .

Tabela 3.7: Proširena kodirana tablica (zadatak 1).

U	$Q_0(t)$	$Q_1(t)$	$Q_0(t+1)$	$Q_1(t+1)$	T_1	T_2	Z_1	Z_2
0	0	0	1	0	0	1	0	1
0	0	1	1	1	0	1	1	0
0	1	0	0	1	1	1	1	0
0	1	1	0	0	1	1	0	1
1	0	0	0	0	0	0	1	0
1	0	1	0	0	1	0	1	1
1	1	0	0	0	0	1	1	0
1	1	1	0	0	1	1	1	0

Minimizacijom logičkih funkcija iz proširene tablice (metodom Veitchovih dijagrama) dobijaju se sljedeće optimalne forme:

$$T_1 = Q_1 U \vee Q_0 \bar{U} \quad (3.4)$$

$$T_2 = Q_0 \vee \bar{U} \quad (3.5)$$

$$Z_1 = U \vee (Q_1 \oplus Q_0) \quad (3.6)$$

$$Z_2 = Q_1 Q_0 \bar{U} \vee Q_1 U \bar{Q}_0 \vee \bar{Q}_1 \bar{Q}_0 \bar{U} \quad (3.7)$$

Na osnovu ovih jednačina formira se strukturna shema automata, sastavljena od dva T flip-flopa i osnovnih logičkih kola (NE, I, ILI).

SystemVerilog implementacija

Implementacija korištenjem isključivo tablice prelaza i izlaza data je u Kodu 15:

Kod 15 Implementacija Mealyevog automata korištenjem tablice prelaza i izlaza.

```
1 module mealy_automat (  
2     input  logic      U,  
3     input  logic      Cp,  
4     input  logic      reset,  
5     output logic [1:0] Z,  
6     output logic [1:0] stanje  
7 );  
8  
9     logic [1:0] sljed_stanje;  
10  
11     always_ff @(posedge reset or posedge Cp) begin  
12         if (reset == 1'b1) begin  
13             stanje <= 2'd0;  
14         end  
15         else begin  
16             stanje <= sljed_stanje;  
17         end  
18     end  
19  
20     always_comb begin  
21         case (stanje)  
22             2'd0: begin  
23                 if (U == 1'b0) begin  
24                     Z = 2'b01;  
25                     sljed_stanje = 2'd1;  
26                 end  
27                 else begin  
28                     Z = 2'b10;  
29                     sljed_stanje = 2'd0;  
30                 end  
31             end  
32             2'd1: begin  
33                 if (U == 1'b0) begin  
34                     Z = 2'b10;  
35                     sljed_stanje = 2'd2;  
36                 end  
37                 else begin  
38                     Z = 2'b10;  
39                     sljed_stanje = 2'd0;  
40                 end  
41             end  
42             2'd2: begin  
43                 if (U == 1'b0) begin  
44                     Z = 2'b10;  
45                     sljed_stanje = 2'd3;  
46                 end  
47                 else begin  
48                     Z = 2'b11;  
49                     sljed_stanje = 2'd0;  
50                 end  
51             end  
52         end  
53     end  
54 end
```

```
52         2'd3: begin
53             if (U == 1'b0) begin
54                 Z = 2'b01;
55                 sljed_stanje = 2'd0;
56             end
57             else begin
58                 Z = 2'b10;           // y2
59                 sljed_stanje = 2'd0; // a0
60             end
61         end
62         default: begin
63             Z = 2'b00;
64             sljed_stanje = 2'd0;
65         end
66     endcase
67 end
68
69 endmodule : mealy_automat
```

Implementacija korištenjem strukturne sheme automata, zasnovana na izvedenim jednačinama T_1 , T_2 , Z_1 i Z_2 , data je u Kodu 16. Modul `T_ff` opisuje pojedinačni T flip-flop, dok modul `mealy_struktura` povezuje dva flip-flopa sa kombinacionom logikom sljedećeg stanja i izlaza.

Kod 16 Implementacija Mealyevog automata korištenjem strukturne sheme (T flip-flopi).

```
1 module T_ff (
2     input logic T,
3     input logic Cp,
4     input logic reset,
5     output logic Q
6 );
7
8     always_ff @(posedge reset or posedge Cp) begin
9         if (reset == 1'b1) begin
10             Q <= 1'b0;
11         end
12         else begin
13             Q <= T ^ Q;
14         end
15     end
16
17 endmodule : T_ff
18
19
20 module mealy_struktura (
21     input logic U,
22     input logic Cp,
23     input logic reset,
24     output logic [1:0] Z,
```

```

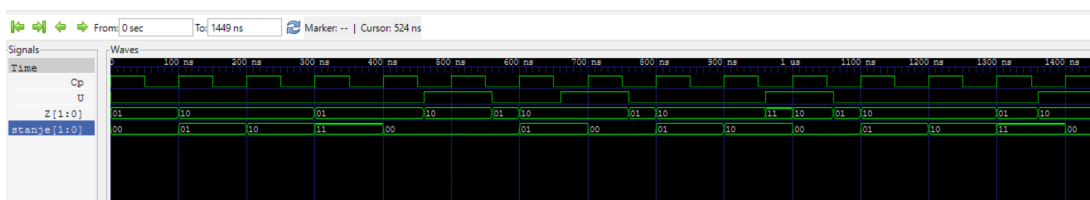
25     output logic [1:0] stanje
26 );
27
28     logic Q1, Q0;
29     logic T1, T2;
30
31     T_ff flipflop1 (.T(T1), .Cp(Cp), .reset(reset), .Q(Q1));
32     T_ff flipflop2 (.T(T2), .Cp(Cp), .reset(reset), .Q(Q0));
33
34     always_comb begin
35         T1 = (Q1 & U) | (Q0 & ~U);
36         T2 = Q0 | ~U;
37
38         Z[1] = U | (Q1 ^ Q0);
39         Z[0] = (Q1 & Q0 & ~U) | (Q1 & U & ~Q0) | (~Q1 & ~Q0 & ~U);
40
41         stanje = {Q1, Q0};
42     end
43
44 endmodule : mealy_struktura

```

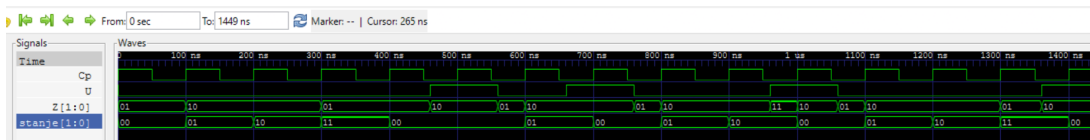
Simulacija putem Verilator alata

Fajlovi potrebni za simulaciju ovih primjera se mogu pronaći na github linku.

Prikaz ulaznih i izlaznih signala dobijenih simulacijom koda koji koristi tablicu prelaza i izlaza dat je na slici 3.4, a koristeći strukturnu shemu se nalazi na slici 3.5. Poređenjem dobijenih talasnih oblika uočava se da su nizovi vrijednosti signala *stanje* i *Z* potpuno identični kod oba pristupa, za istu sekvencu ulaznih vrijednosti *U* – čime je potvrđeno da je strukturna implementacija, zasnovana na izvedenim jednačinama T_1 , T_2 , Z_1 i Z_2 , funkcionalno ekvivalentna bihevioralnoj implementaciji zasnovanoj isključivo na tablici prelaza i izlaza.



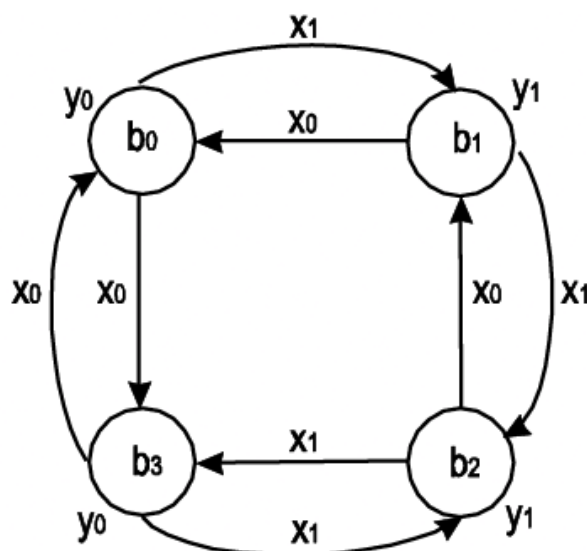
Slika 3.4: Ulazni i izlazni signali Mealyevog automata koristeći tablicu prelaza i izlaza.



Slika 3.5: Ulazni i izlazni signali Mealyevog automata koristeći strukturnu shemu.

3.2.2 Zadatak 2 – Mooreov automat

Graf koji opisuje funkcionalnost automata prikazan je na slici 3.6.



Slika 3.6: Graf Mooreovog automata (zadatak 2).

Automat ima četiri unutrašnja stanja (b_0, b_1, b_2, b_3), sa dva ulazna signala (x_0, x_1) i dva moguća izlazna signala (y_0, y_1). Za razliku od Mealyevog automata, izlaz Mooreovog automata zavisi isključivo od trenutnog stanja, a ne i od trenutnog ulaza. Prelazi automata ne formiraju jednostavan simetričan ciklus – iz stanja b_0, b_1 i b_2 ulaz x_1 redom vodi u sljedeće stanje ($b_0 \rightarrow b_1 \rightarrow b_2 \rightarrow b_3$), dok iz stanja b_3 ulaz x_1 vraća automat u b_2 (ne dalje ka b_0). Slično, ulaz x_0 iz stanja b_1, b_2 i b_3 redom vraća automat unazad ($b_1 \rightarrow b_0, b_2 \rightarrow b_1, b_3 \rightarrow b_0$), dok iz stanja b_0 ulaz x_0 vodi u b_3 . Tablica prelaza i izlaza data je u tabeli 3.8.

Tabela 3.8: Tablica prelaza i izlaza (zadatak 2).

$x_n b_r$	b_0	b_1	b_2	b_3
x_0	b_3	b_0	b_1	b_0
x_1	b_1	b_2	b_3	b_2
y_i	y_0	y_1	y_1	y_0

Provjera minimizacije broja stanja

Stanja b_0 i b_3 dijele isti izlaz y_0 , dok b_1 i b_2 dijele isti izlaz y_1 , pa predstavljaju kandidate za spajanje. Za $b_0 \equiv b_3$ bilo bi potrebno da za ulaz x_1 njihovi nasljednici (b_1 za b_0 , b_2 za b_3) budu ekvivalentni, odnosno $b_1 \equiv b_2$. Provjerom tog para: za ulaz x_0 , nasljednici b_1 i b_2 su redom b_0 i b_1 , pa bi uslov $b_1 \equiv b_2$ zahtijevao da vrijedi $b_0 \equiv b_1$ – što ne vrijedi, jer b_0 i b_1 imaju različite izlaze (y_0 naspram y_1). Prema tome, $b_1 \not\equiv b_2$, pa ni $b_0 \not\equiv b_3$. Automat, dakle, nema ekvivalentnih stanja i već je zadan u minimalnom obliku.

Kodiranje signala

Kao i u prethodnom zadatku, za realizaciju registra stanja potrebna su dva T flip-flopa. Pošto Mooreov izlaz zavisi isključivo od stanja, dovoljan je jedan izlazni bit Z. Kodiranje je dato u tabelama 3.9–3.11.

Tabela 3.9: Kodirana stanja.

	Q_0	Q_1
b_0	0	0
b_1	1	0
b_2	0	1
b_3	1	1

Tabela 3.10: Kodirani ulaz.

	U
x_0	0
x_1	1

Tabela 3.11: Kodirani izlazi.

	Z
y_0	0
y_1	1

Prošireno kodirana tablica i logičke funkcije

Proširena kodirana tablica prelaza data je u tabeli 3.12.

Tabela 3.12: Proširena kodirana tablica (zadatak 2).

U	$Q_0(t)$	$Q_1(t)$	$Q_0(t+1)$	$Q_1(t+1)$	T_1	T_2
0	0	0	1	1	1	1
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	1	0	0	1	1
1	0	0	1	0	0	1
1	0	1	1	1	0	1
1	1	0	0	1	1	1
1	1	1	0	1	0	1

Uočava se da je $T_2 = 1$ za svaku kombinaciju ulaza – flip-flop Q_0 mijenja stanje pri svakom taktu, nezavisno od ulaza. Minimizacijom preostalih funkcija (metodom Veitchovih dijagrama, provjereno i računski) dobija se:

$$T_1 = Q_1 \bar{U} \vee \bar{Q}_0 \bar{U} \vee Q_0 U \bar{Q}_1 \quad (3.8)$$

$$T_2 = 1 \quad (3.9)$$

$$Z = Q_1 \oplus Q_0 \quad (3.10)$$

SystemVerilog implementacija

Implementacija korištenjem isključivo tablice prelaza i izlaza data je u Kodu 17:

Kod 17 Implementacija Mooreovog automata korištenjem tablice prelaza i izlaza.

```

1 module moore_automat (
2     input  logic      U,
3     input  logic      Cp,
4     input  logic      reset,
5     output logic      Z,
6     output logic [1:0] stanje
7 );
8
9     logic [1:0] sljed_stanje;
10

```

```

11  always_ff @(posedge reset or posedge Cp) begin
12      if (reset == 1'b1) begin
13          stanje <= 2'd0;
14      end
15      else begin
16          stanje <= sljed_stanje;
17      end
18  end
19
20  always_comb begin
21      case (stanje)
22          2'd0: begin
23              Z = 1'b0;
24              sljed_stanje = (U == 1'b0) ? 2'd3 : 2'd1;
25          end
26          2'd1: begin
27              Z = 1'b1;
28              sljed_stanje = (U == 1'b0) ? 2'd0 : 2'd2;
29          end
30          2'd2: begin
31              Z = 1'b1;
32              sljed_stanje = (U == 1'b0) ? 2'd1 : 2'd3;
33          end
34          2'd3: begin
35              Z = 1'b0;
36              sljed_stanje = (U == 1'b0) ? 2'd0 : 2'd2;
37          end
38          default: begin
39              Z = 1'b0;
40              sljed_stanje = 2'd0;
41          end
42      endcase
43  end
44
45  endmodule : moore_automat

```

Implementacija korištenjem strukturne sheme automata, zasnovana na izvedenim jednačima T_1 , T_2 i Z , data je u Kodu 18. Iskorišten je isti modul `T_ff` definisan u zadatku 1.

Kod 18 Implementacija Mooreovog automata korištenjem strukturne sheme (T flip-flopovi).

```

1  module T_ff (
2      input logic T,
3      input logic Cp,
4      input logic reset,
5      output logic Q
6  );
7
8      always_ff @(posedge reset or posedge Cp) begin
9          if (reset == 1'b1) begin

```

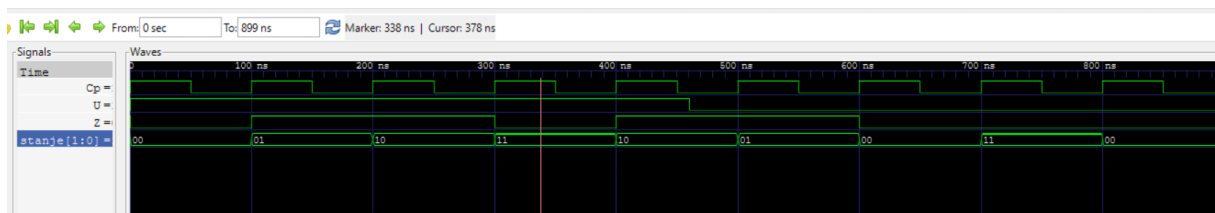
```

10         Q <= 1'b0;
11     end
12     else begin
13         Q <= T ^ Q;
14     end
15 end
16
17 endmodule : T_ff
18
19
20 module moore_struktura (
21     input logic      U,
22     input logic      Cp,
23     input logic      reset,
24     output logic      Z,
25     output logic [1:0] stanje // trenutno stanje automata (Q1,Q0
26                               ) - vidljivo u simulaciji
27 );
28
29     logic Q1, Q0;
30     logic T1, T2;
31
32     T_ff flipflop1 (.T(T1), .Cp(Cp), .reset(reset), .Q(Q1));
33     T_ff flipflop2 (.T(T2), .Cp(Cp), .reset(reset), .Q(Q0));
34
35     always_comb begin
36         T1 = (Q1 & ~U) | (~Q0 & ~U) | (Q0 & U & ~Q1);
37         T2 = 1'b1;
38
39         Z = Q1 ^ Q0;
40         stanje = {Q1, Q0};
41     end
42 endmodule : moore_struktura

```

Simulacija putem Verilator alata

Kod za fileove run-sim i versim se može naći na github repozitoriju. Prikaz ulaznih i izlaznih signala dobijenih simulacijom oba pristupa dat je na slici 3.7.

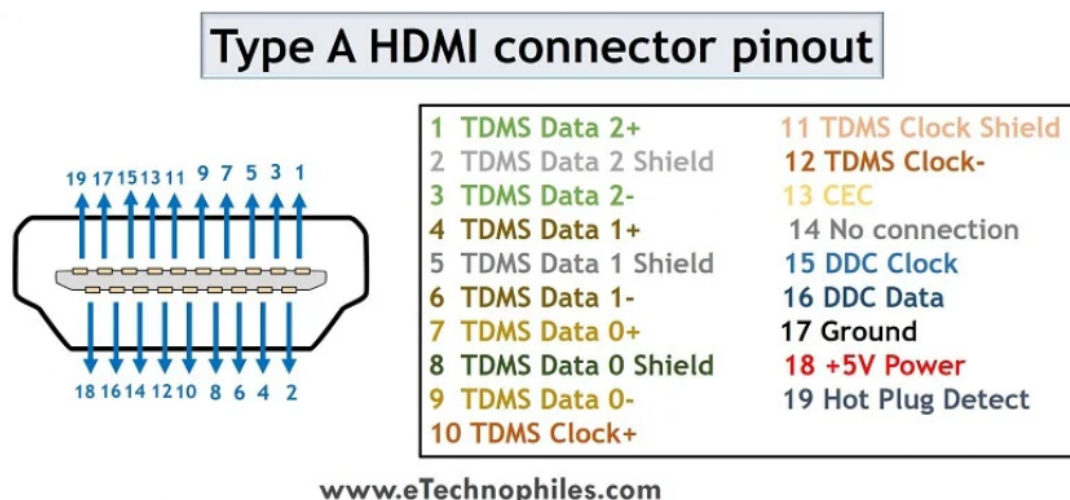


Slika 3.7: Ulazni i izlazni signali Mooreovog automata.

3.3 Laboratorijska vježba 3

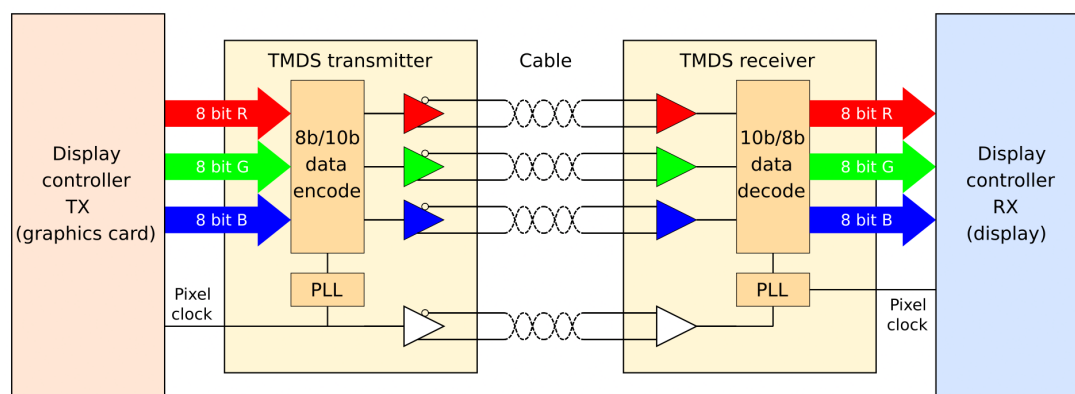
3.3.1 Teorijski uvod – HDMI komunikacija

HDMI (engl.: *High-Definition Multimedia Interface*) je standardizovani protokol za prijenos nekompresovanog digitalnog video i audio signala između uređaja, prvi put predstavljen 2002. godine, koji je od tada postao dominantan način povezivanja televizora, monitora i konzola sa izvorima signala poput računara i FPGA razvojnih sistema [14]. Fizički, standardni HDMI konektor tipa A sadrži 19 pinova: tri para pinova nose video podatke (po jedan par za crvenu, zelenu i plavu komponentu boje), jedan par nosi referentni takt, dok preostali pinovi služe za pomoćne funkcije poput DDC/EDID komunikacije (razmjena podataka o mogućnostima monitora) i CEC protokola [14]. Raspored pinova prikazan je na slici 3.8.



Slika 3.8: Raspored pinova HDMI konektora tipa A.

Prijenos podataka kroz sva četiri diferencijalna para zasniva se na TMDS (engl.: *Transition-Minimized Differential Signaling*) tehnologiji. Umjesto da se signal šalje kao jedna linija u odnosu na masu, TMDS koristi dvije komplementarne linije (diferencijalni par) po kojima se šalje isti signal u suprotnim fazama – prijemnik potom očitava razliku napona između te dvije linije, čime se značajno smanjuje osjetljivost na elektromagnetne smetnje u odnosu na jednostruki (engl.: *single-ended*) prijenos signala [15]. Prikaz principa diferencijalnog prijenosa dat je na slici 3.9.



Slika 3.9: Princip TMDS diferencijalnog prijenosa signala.

Dodatno, prije samog slanja, svakih 8 bita video podataka (po jednoj boji, po jednom pikselu) enkodira se u 10-bitnu TMDS riječ, pomoću algoritma koji ima dva cilja: minimizirati broj promjena logičkog stanja na liniji (radi smanjenja elektromagnetnih smetnji) i istovremeno održati dovoljan broj tranzicija da bi prijemnik mogao pouzdano rekonstruisati takt iz same podatkovne linije (engl.: *clock recovery*) [15]. TMDS takt (jedan od četiri diferencijalna para) odgovara frekvenciji piksela (engl.: *pixel clock*) i služi prijemniku kao vremenska referenca za očitavanje preostala tri podatkovna para.

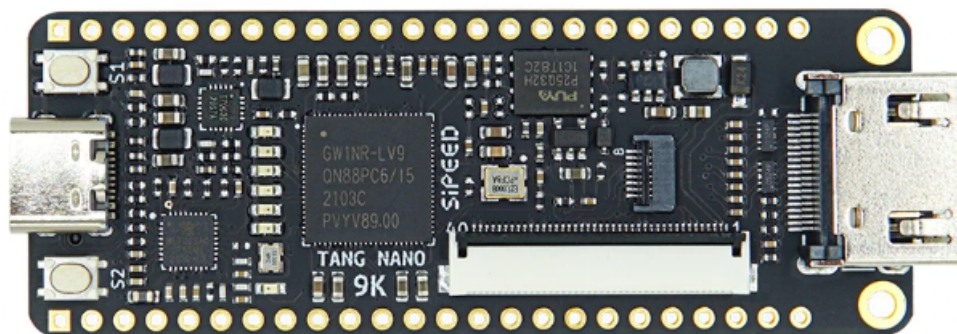
Period slanja jednog kompletnog videa okvira dijeli se na tri vrste perioda: period aktivnog video signala (kada se prenose stvarni pikseli slike), period sinhronizacije (kada se prenose horizontalni/vertikalni sinhronizacioni impulsi, van vidljivog dijela ekrana) i, opcionalno, period pomoćnih podataka (engl.: *data island*) za prijenos audio signala i dodatnih informacionih paketa [15]. Za potrebe ove laboratorijske vježbe, koristi se isključivo video i sinhronizacioni period – audio prijenos nije implementiran.

3.3.2 Struktura startnog projekta

Za realizaciju HDMI komunikacije korišten je pripremljeni startni projekat, prilagođen tačno za Tang Nano 9K razvojni sistem (GW1NR-9C čip), čija se struktura sastoji od sljedećih fajlova:

- `top.sv` – najviši nivo hijerarhije. Sadrži logiku za generisanje boje svakog piksela na osnovu njegovih *X* i *Y* koordinata (koje dobija od HDMI kontrolera), te instancira glavni HDMI modul.
- `hdmi_pkg.sv` – SystemVerilog paket sa zajedničkim tipovima podataka i parametrima vremenskog rasporeda ekrana (broj piksela po redu/koloni, pozicija i trajanje sinhronizacionih impulsa) za odabranu rezoluciju.
- `hdmi_backend.sv` – centralni modul koji generiše taktove (preko PLL-a), broji trenutnu *X, Y* poziciju piksela, generiše horizontalni i vertikalni sinhronizacioni signal, te prosljeđuje pikselske podatke na TMDS enkodiranje i serializaciju.
- `hdmi_tdmcs_enc.sv` – implementacija TMDS 8b/10b algoritma opisanog u prethodnoj sekciji, za svaki od tri kolor kanala posebno.
- `src/fpgatech_lib/GOWIN/` – folder sa tehnologijom-specifičnim omotačima (*wrapper* modulima) oko Gowin primitiva: `fpga_pll.sv` (generisanje piksel takta iz 27 MHz oscilatora ploče), `fpga_oser10.sv` (serializacija 10-bitne TMDS riječi), i `fpga_olvds.sv` (izlazni LVDS baferi koji fizički pokreću diferencijalne parove).
- `constraints/pinout.cst` – fizičko mapiranje signala na konkretne pinove FPGA čipa.

Razvojni sistem korišten u ovoj vježbi prikazan je na slici 3.10.



Slika 3.10: Tang Nano 9K razvojni sistem sa HDMI konektorom.

3.3.3 Prilagođavanje HDMI predloška odabranom FPGA uređaju i monitoru

Startni HDMI projekat je generički napisan tako da se lako prenosi (*portuje*) na različite FPGA platforme, ali prije upotrebe ga je potrebno prilagoditi konkretnoj kombinaciji FPGA uređaja i monitora koji se koriste. U ovom radu, to su:

- **FPGA uređaj:** Tang Nano 9K razvojni sistem (proizvođač Sipeed), sa GW1NR-9C čipom, tačnog part numbera GW1NR-LV9QN88PC6/I5, taktovan eksternim oscilatorom od 27 MHz;
- **Monitor:** standardni HDMI monitor rezolucije 1280×720 piksela pri frekvenciji osvježavanja od 60 Hz (*VideoMode=4* prema HDMI/CEA-861 standardu).

Prilagođavanje predloška ovoj kombinaciji uređaja obuhvata sljedeće korake:

1. **Odabir FPGA uređaja u projektu** – u postavkama Gowin EDA projekta potrebno je odabrati tačan model čipa (GW1NR-9C, part number GW1NR-LV9QN88PC6/I5), jer se prema tom modelu generišu parametri PLL bloka u fajlu `fpga_pll sv`.
2. **Parametri rezolucije ekrana** – u fajlu `hdmi_pkg sv` definisani su parametri vremenskog rasporeda (broj piksela po redu i koloni, pozicija i trajanje sinhronizacionih impulsa) tačno za rezoluciju $1280 \times 720@60\text{Hz}$. Za ovu kombinaciju uređaja ovi parametri se koriste bez izmjene; ukoliko bi se koristio monitor druge rezolucije, ove vrijednosti bi bilo potrebno zamijeniti odgovarajućim standardnim vrijednostima za tu rezoluciju.

3. **Podešavanje PLL bloka** – fajl `fpga_pll.sv` iz ulaznog takta od 27 MHz (frekvencija oscilatora na Tang Nano 9K pločici) generiše dva izlazna takta potrebna za odabranu rezoluciju: piksel takt od 74.25 MHz i $5\times$ serijski takt od 371.25 MHz. Množitelji/djelitelji PLL-a (`IDIV_SEL`, `FBDIV_SEL`, `ODIV_SEL`) su izračunati tačno za ovu kombinaciju ulaznog takta i ciljane rezolucije, i već su podešeni za GW1NR-9C čip.
4. **Fizička dodjela pinova** – fajl `pinout.cst` sadrži mapiranje HDMI signala (takt i tri podatkovna diferencijalna para) na tačne fizičke pinove Tang Nano 9K pločice na kojima je fizički izveden HDMI konektor, kao i mapiranje dva tastera i eksternog oscilatora. Za drugu razvojnu pločicu ove lokacije bi bilo potrebno prilagoditi prema šemi te pločice.
5. **Sinteza, generisanje bitstream-a i programiranje** – nakon što su sva prethodna podešavanja usklađena sa korištenim uređajima, projekat se sintetizuje na uobičajen način, a generisani bitstream se prenosi na Tang Nano 9K putem USB-C porta.

3.3.4 Implementacija interaktivne igre – pokretna loptica

Na osnovu prilagođenog predloška, implementirana je jednostavna interaktivna igra: crvena loptica prikazana na ekranu, čija se vertikalna pozicija mijenja u zavisnosti od toga koje je dugme na razvojnoj pločici pritisnuto.

Crtanje loptice

Boja svakog piksela određuje se na osnovu njegove udaljenosti od centra loptice. Piksel na poziciji (x, y) boji se crveno ako se nalazi unutar kruga poluprečnika r centriranog u tački (x_0, y_0) , odnosno ako vrijedi:

$$(x - x_0)^2 + (y - y_0)^2 \leq r^2 \quad (3.11)$$

Horizontalna koordinata centra x_0 je fiksna i postavljena na 640 (sredina ekrana rezolucije 1280×720), dok se vertikalna koordinata y_0 čuva u registru `brojac` i mijenja pritiskom na dugmad. Prag $r^2 = 250$ odgovara poluprečniku od približno 16 piksela.

Kod 19 Uslov za crtanje loptice unutar `_my_game` bloka.

```

1 always_comb begin: _my_game
2   // Crvena loptica poluprecnika ~sqrt(250)=15.8px, centrirana u
   // (640, brojac).
3   // Van kruga, pozadina je jednobojna plava.
4   if ((x-640)*(x-640)+(y-brojac)*(y-brojac)<=250) begin
5     pix.R = '1;
6     pix.G = '0;
7     pix.B = '0;
8   end
9   else begin
10    pix.R = '0;
11    pix.G = '0;
12    pix.B = '1;
13  end
14 end: _my_game

```

Upravljanje pozicijom loptice

Pozicija loptice se ažurira unutar istog sekvencijalnog bloka koji generiše i signal za LED diodu, koji se aktivira jednom u svakih MFRAM_CNT_MAX (30) prikazanih okvira – što odgovara ažuriranju otprilike svakih pola sekunde, dovoljno sporo da je kretanje jasno vidljivo. Dugme butnL_n pomjera lopticu prema gore, a butnR_n prema dole (oba dugmeta su aktivniska).

Kretanje je ograničeno na opseg [BALL_Y_MIN, BALL_Y_MAX] = [20, 700], čime loptica (poluprečnika ≈ 16 piksela) uvijek ostaje potpuno unutar vidljivog dijela ekrana (0–719), sa mekim zaustavljanjem na gornjoj i donjoj granici. Početna pozicija nakon reseta je $y_0 = 360$, tačno na vertikalnoj sredini ekrana.

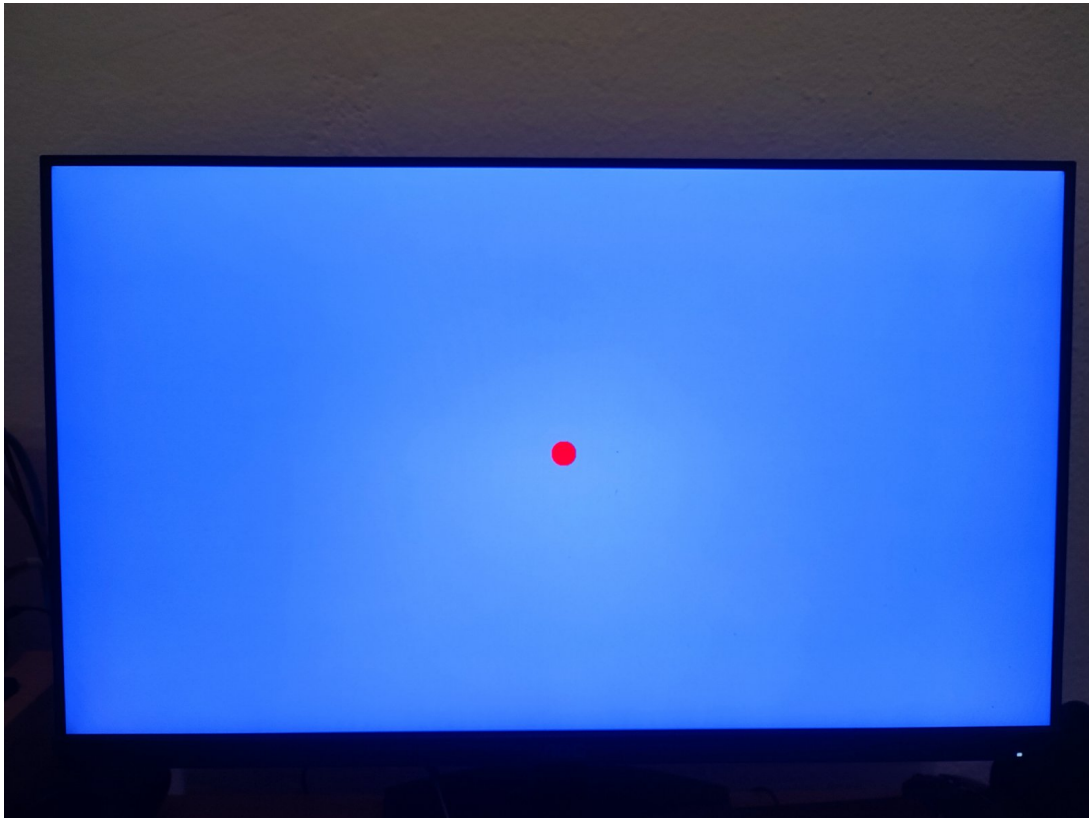
Kod 20 Logika resetovanja i pomjeranja loptice.

```
1  localparam bus12_t BALL_Y_MIN  = 12'd20;    // gornja granica
   kretanja
2  localparam bus12_t BALL_Y_MAX  = 12'd700;    // donja granica
   kretanja
3  localparam bus12_t BALL_Y_INIT = 12'd360;    // pocetna pozicija -
   sredina ekrana
4  localparam bus12_t BALL_STEP   = 12'd10;    // korak pomjeranja po
   jednom updateu
5
6  bus12_t brojac;
7
8  always_ff @(negedge srst_n or posedge clk_pix) begin: _led_mon
9      if (srst_n == LO) begin
10         led_n  <= HI;
11         brojac <= BALL_Y_INIT;
12     end
13     else begin
14         if (mfram_done == HI) begin
15             led_n <= ~led_n;
16
17             if (butnL_n == LO) begin
18                 if (brojac > BALL_Y_MIN + BALL_STEP)
19                     brojac <= brojac - BALL_STEP;    // gore
20                 else
21                     brojac <= BALL_Y_MIN;
22             end
23             else if (butnR_n == LO) begin
24                 if (brojac < BALL_Y_MAX - BALL_STEP)
25                     brojac <= brojac + BALL_STEP;    // dole
26                 else
27                     brojac <= BALL_Y_MAX;
28             end
29         end
30     end
```

```
31 end: _led_mon
```

Rezultat

Pritiskom na dugme S1 (butnL_n) loptica se pomjera prema gore, a pritiskom na S2 (butnR_n) prema dole, sa glatkim zaustavljanjem na gornjoj i donjoj granici ekrana. Rezultat prikazan na monitoru dat je na slici 3.11.



Slika 3.11: Crvena loptica prikazana na monitoru putem HDMI izlaza.

3.3.5 Dodavanje pokretne prepreke

Nakon osnovne implementacije loptice čija se vertikalna pozicija upravlja dugmadima, projekat je proširen dodavanjem pokretne prepreke koju je potrebno izbjegavati – koncept sličan poznatoj igri *Flappy Bird*, prilagođen tako da je horizontalna pozicija loptice fiksna, a igrač je pomjera isključivo vertikalno kako bi prošao kroz otvor na prepreci koja se kreće zdesna nalijevo.

Struktura prepreke

Prepreka je predstavljena kao vertikalni zid širine `OBST_WIDTH` (40 piksela), sa pravougaonim otvorom (prorezom) visine `GAP_HEIGHT` (200 piksela). Umjesto praćenja više prepreka istovremeno, korištena je najjednostavnija moguća šema: u svakom trenutku postoji tačno jedna prepreka na ekranu, predstavljena registrom `obst_x` koji čuva njenu trenutnu horizontalnu poziciju.

Kod 21 Parametri i signali prepreke.

```
1 localparam bus12_t OBST_WIDTH  = 12'd40;
2 localparam bus12_t OBST_SPEED  = 12'd6;
3 localparam bus12_t GAP_HEIGHT  = 12'd200;
4
5 bus12_t      obst_x;    // trenutna X pozicija lijeve ivice prepreke
6 bus12_t      gap_y;     // trenutna Y pozicija vrha proreza
7 logic [1:0] gap_sel;    // redni broj trenutne pozicije proreza (0-3)
```

Kretanje prepreke

Pozicija prepreke se ažurira jednom po svakom prikazanom okviru (signal `frame_done`, 60 puta u sekundi), pri čemu se `obst_x` smanjuje za `OBST_SPEED` piksela. Kada prepreka izađe van lijeve ivice ekrana, umjesto da se dozvoli neograničeno opadanje registra (što bi, s obzirom na to da je `obst_x` bez predznaka, dovelo do nekontrolisanog prekoračenja), eksplicitno se provjerava granica i prepreka se vraća na početnu poziciju van desne ivice ekrana:

Kod 22 Sekvencijalna logika kretanja prepreke.

```
1 always_ff @(negedge srst_n or posedge clk_pix) begin: _obstacle_mov
2     if (srst_n == LO) begin
3         obst_x  <= 12'(HSCREEN) + OBST_WIDTH; // pocinje van desne
4         gap_y   <= gap_preset(2'd0);
5         gap_sel <= 2'd0;
6     end
7     else if (frame_done == HI) begin
8         if (obst_x <= OBST_SPEED) begin
9             gap_sel <= gap_sel + 2'd1;
10            gap_y   <= gap_preset(gap_sel + 2'd1);
11            obst_x  <= 12'(HSCREEN) + OBST_WIDTH;
12        end
13        else begin
14            obst_x <= obst_x - OBST_SPEED;
15        end
16    end
17 end: _obstacle_mov
```

Pozicija proreza

Kako bi svaki prolazak prepreke djelovao drugačije, umjesto stvarnog generatora slučajnih brojeva (koji bi unio nepotrebnu složenost za ovako jednostavnu igru), korištene su četiri unaprijed pripremljene pozicije proreza, kroz koje se ciklično prolazi pomoću brojača `gap_sel` svaki put kada prepreka krene ispočetka:

Kod 23 Funkcija za odabir pozicije proreza.

```
1 function automatic bus12_t gap_preset(input logic [1:0] sel);
2     case (sel)
3         2'd0:    gap_preset = 12'd60;
4         2'd1:    gap_preset = 12'd220;
5         2'd2:    gap_preset = 12'd380;
6         default: gap_preset = 12'd500;
7     endcase
8 endfunction
```

Iscrtavanje prepreke

Prepreka se iscrtava zelenom bojom na svim pikselima koji se nalaze unutar njene širine, osim na mjestu proreza. Loptica ima prioritet iscrtavanja – ukoliko se piksel nalazi i unutar loptice i unutar prepreke, prikazuje se boja loptice, čime igrač uvijek jasno vidi tačnu poziciju loptice u odnosu na prepreku:

Kod 24 Prošireni `_my_game` blok sa iscrtavanjem prepreke.

```
1 always_comb begin: _my_game
2     obst_here = (x >= obst_x) && (x < obst_x + OBST_WIDTH);
3     in_gap     = (y >= gap_y) && (y < gap_y + GAP_HEIGHT);
4
5     if ((x-BALL_X)*(x-BALL_X)+(y-brojac)*(y-brojac) <= 250) begin
6         pix.R = '1; // loptica - crvena
7         pix.G = '0;
8         pix.B = '0;
9     end
10    else if (obst_here && !in_gap) begin
11        pix.R = '0; // prepreka - zelena
12        pix.G = '1;
13        pix.B = '0;
14    end
15    else begin
16        pix.R = '0; // pozadina - plava
17        pix.G = '0;
18        pix.B = '1;
19    end
20 end: _my_game
```

Detekcija sudara

Umjesto piksel-po-piksel provjere preklapanja kružnice loptice i pravougaonika prepreke (što bi zahtijevalo znatno složeniju logiku), korištena je pojednostavljena provjera na nivou intervala. Pošto je horizontalna pozicija loptice fiksna ($x_0 = 640$, poluprečnik ≈ 16 piksela), do sudara

dolazi kada se prepreka horizontalno preklapa sa opsegom koji zauzima loptica, a loptica pritom nije u potpunosti unutar proreza:

Kod 25 Kombinaciona detekcija sudara.

```
1 always_comb begin: _collision_check
2     hit = (obst_x <= BALL_X + BALL_R)
3         & (obst_x + OBST_WIDTH > BALL_X - BALL_R)
4         & ~( (brojac >= gap_y + BALL_R) & (brojac + BALL_R <= gap_y +
5             GAP_HEIGHT) );
end: _collision_check
```

Rezultat sudara (signal `hit`) koristi se za signalizaciju korisniku putem posebne LED diode, opisano u narednoj sekciji.

Rezultat

Kombinacijom pomjeranja loptice (opisanog u prethodnoj sekciji) i pokretne prepreke sa slučajno raspoređenim prorezima, dobijena je jednostavna interaktivna igra u kojoj igrač dugmadima izbjegava sudar sa preprekama koje neprekidno pristižu sa desne strane ekrana.

3.3.6 Signalizacija sudara pomoću dvije nezavisne LED diode

Razvojna pločica raspolaže sa šest ugrađenih LED dioda (pinovi 10, 11, 13, 14, 15 i 16), od kojih je startni projekat koristio samo jednu (pin 11) za prikaz opšte aktivnosti sistema. Kako bi se jasno razdvojile dvije nezavisne informacije – da li sistem uopšte radi, i da li je trenutno došlo do sudara loptice sa preprekom – korištene su dvije odvojene diode, svaka zadužena za tačno jednu namjenu.

- `led_n` (pin 11) – treperi ravnomjerno jednom u svakih `MFRAM_CNT_MAX` (30) prikazanih okvira, nezavisno od bilo kakvog događaja u igri. Služi kao pokazatelj da sistem radi i da je takt aktivan.
- `led_hit_n` (pin 10) – upaljena isključivo onoliko dugo koliko traje preklapanje loptice i prepreke, računato direktno iz signala `hit` opisanog u prethodnoj sekciji.

Fizičko mapiranje novog signala dodaje se u fajl `pinout.cst`:

Kod 26 Dodatak u `pinout.cst` za drugu LED diodu.

```
1 IO_LOC   "led_hit_n" 10;
2 IO_PORT  "led_hit_n" PULL_MODE=UP DRIVE=8;
```

U SystemVerilog kodu, svaka dioda se upravlja u svom vlastitom, potpuno nezavisnom sekvencijalnom bloku, čime se izbjegava bilo kakvo međusobno ometanje njihovog ponašanja:

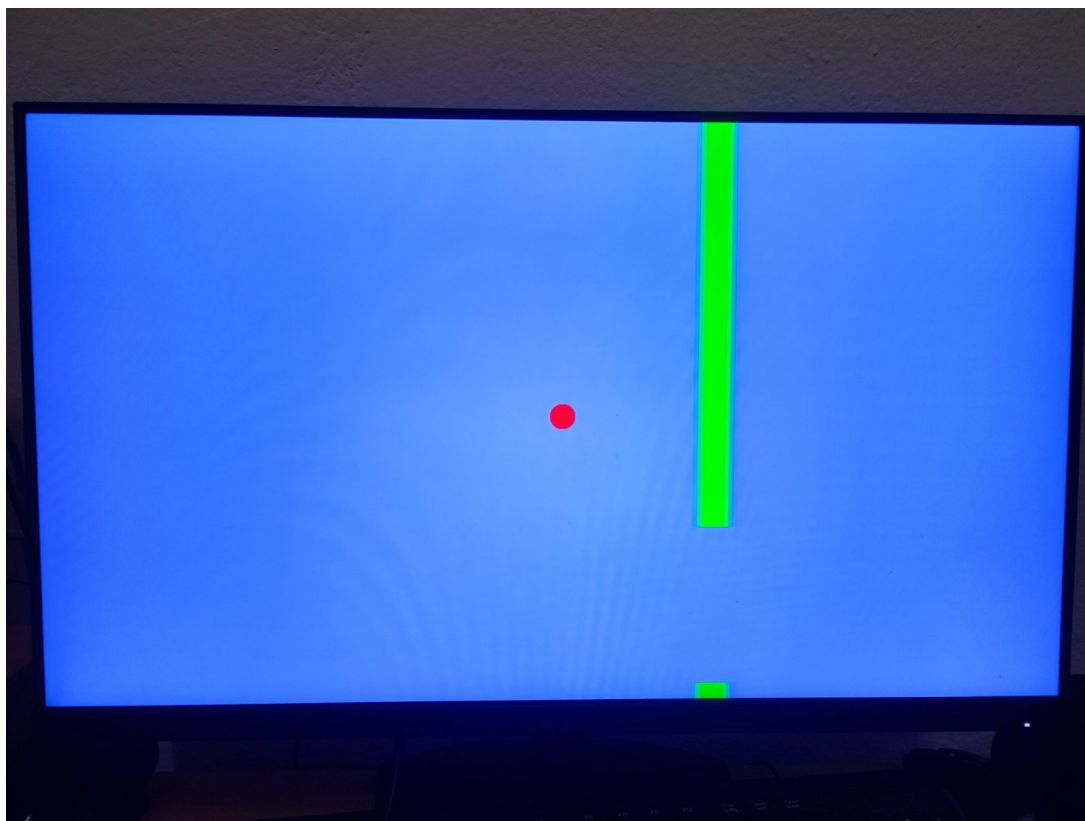
Kod 27 Nezavisno upravljanje dvjema LED diodama.

```

1 //-----
2 // Pomjeranje loptice dugmadima. LED (led_n) treperi ravnomjerno
3 // jednom u MFRAM_CNT_MAX prikazanih okvira, nezavisno od sudara.
4 //-----
5 always_ff @(negedge srst_n or posedge clk_pix) begin: _led_mon
6     if (srst_n == LO) begin
7         led_n  <= HI;
8         brojac <= BALL_Y_INIT;
9     end
10    else begin
11        if (mfram_done == HI) begin
12            led_n <= ~led_n;
13        end
14
15        if (frame_done == HI) begin
16            if (butnL_n == LO) begin
17                if (brojac > BALL_Y_MIN + BALL_STEP)
18                    brojac <= brojac - BALL_STEP;
19                else
20                    brojac <= BALL_Y_MIN;
21            end
22            else if (butnR_n == LO) begin
23                if (brojac < BALL_Y_MAX - BALL_STEP)
24                    brojac <= brojac + BALL_STEP;
25                else
26                    brojac <= BALL_Y_MAX;
27            end
28        end
29    end
30 end: _led_mon
31
32 //-----
33 // Posebna LED (led_hit_n) isključivo za signalizaciju sudara -
34 // upaljena tačno onoliko dugo koliko traje preklapanje loptice
35 // i prepreke, potpuno nezavisno od led_n treperenja.
36 //-----
37 always_ff @(negedge srst_n or posedge clk_pix) begin: _hit_led
38     if (srst_n == LO) begin
39         led_hit_n <= HI;
40     end
41     else begin
42         led_hit_n <= hit ? LO : HI;
43     end
44 end: _hit_led

```

Na slici 3.12 se nalazi prikaz kako izgleda interaktivna igrica nakon ubacivanja prepreke.



Slika 3.12: Crvena loptica i prepreka prikazani na monitoru putem HDMI izlaza.

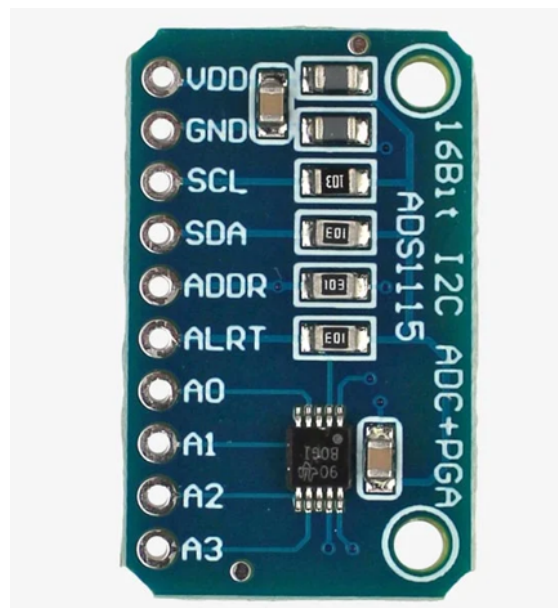
3.4 Laboratorijska vježba 4

3.4.1 Teorijski uvod – potencijometar i ADS1115 analogno-digitalni konvertor

U okviru ove laboratorijske vježbe FPGA sistem se prvi put povezuje sa vanjskom analognom veličinom – naponom dobijenim sa potencijometra (slika 3.13) – koju je, prije bilo kakve digitalne obrade, potrebno pretvoriti u digitalni oblik pomoću posebnog integrisanog kola, analogno-digitalnog konvertora (engl.: *Analog-to-Digital Converter*, ADC). Za tu svrhu korišten je ADS1115 (slika 3.14), 16-bitni ADC modul sa I2C sučeljem (engl.: *interface*).



Slika 3.13: Potenciometar korišten u laboratorijskoj vježbi.



Slika 3.14: ADS1115 modul korišten u laboratorijskoj vježbi.

Potenciometar kao djelilac napona

Potenciometar je pasivna elektronička komponenta sa tri priključka: dva krajnja priključka spojena su na suprotne krajeve otporničke staze fiksnog otpora, dok se treći priključak (engl.: *wiper*, klizač) mehanički pomjera duž te staze, dijeleći je na dva serijski spojena otpornika R_1 i R_2 , čiji zbir ostaje konstantan ($R_1 + R_2 = R_{uk}$) nezavisno od položaja klizača [16]. Kada se na krajnje priključke dovede napon napajanja V_{DD} , na klizaču se dobija napon:

$$V_{wiper} = V_{DD} \cdot \frac{R_2}{R_1 + R_2} \quad (3.12)$$

gdje R_2 predstavlja dio otporničke staze između klizača i uzemljenog kraja. Iz jednačine (3.12) vidljivo je da napon na klizaču linearno zavisi od položaja klizača (kod linearnog, engl.: *linear-taper*, potenciometra) i kreće se u granicama od 0 V (klizač na uzemljenom kraju) do V_{DD} (klizač na kraju spojenom na napajanje). Upravo se ovo svojstvo koristi u ovoj vježbi – mehanički položaj potenciometra pretvara se u naponski nivo koji potom čita ADC.

ADS1115 – 16-bitni analogno-digitalni konvertor

ADS1115 je 16-bitni analogno-digitalni konvertor sukcesivne aproksimacije (engl.: *Successive Approximation Register*, SAR), namijenjen preciznom mjerenju malih analognih napona, sa integrisanim programabilnim pojačalom (engl.: *Programmable Gain Amplifier*, PGA), internim naponskim referentnim izvorom i internim oscilatorom, koji komunicira sa mikrokontrolerom ili FPGA uređajem putem I2C sučelja [2]. Čip posjeduje četiri analogna ulaza (A0–A3) koji se, u zavisnosti od konfiguracije, mogu koristiti u diferencijalnom režimu (mjerenje razlike napona između dva ulaza) ili u jednostrukom režimu (engl.: *single-ended*, mjerenje napona jednog ulaza u odnosu na masu) – upravo je ovaj drugi režim korišten u ovoj vježbi, sa ulazom A0.

Rezultat konverzije smješta se u 16-bitni registar, u obliku broja u dvostrukom komplementu, pri čemu se stvarni izmjereni napon V_{IN} dobija iz digitalnog koda D prema izrazu:

$$V_{IN} = \frac{D}{2^{15}} \cdot FS \quad (3.13)$$

gdje je FS pun opseg mjerenja (engl.: *full-scale*), određen podešavanjem programabilnog pojačala (PGA). ADS1115 nudi šest mogućih opsega PGA-a, od $\pm 6.144\text{ V}$ do $\pm 0.256\text{ V}$ – veći opseg omogućava mjerenje viših napona, dok manji opseg daje finiju rezoluciju za niže napone [2].

Način rada ADC-a (koje se podešenje PGA-a koristi, koji se ulaz mjeri, brzina uzorkovanja i sl.) određuje se upisom u 16-bitni konfiguracioni registar, čija su polja data u tabeli 3.13.

Tabela 3.13: Polja konfiguracionog registra ADS1115 korištena u ovoj vježbi [2].

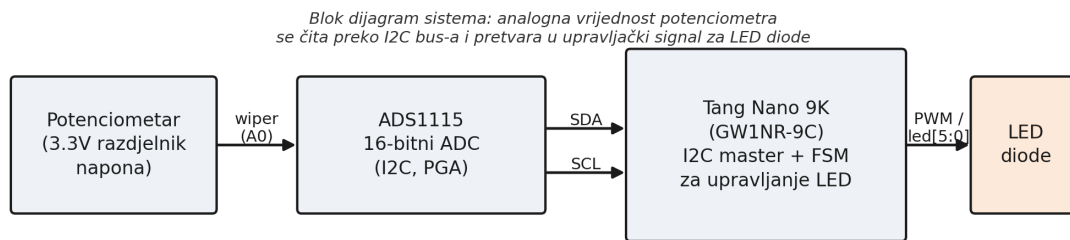
Bit(i)	Polje	Korištena vrijednost	Značenje
15	OS	1	pokreni novu (single-shot) konverziju
14:12	MUX	100	A0 naspram GND (single-ended)
11:9	PGA	010	opseg $FS = \pm 2.048\text{ V}$
8	MODE	1	<i>single-shot</i> režim rada
7:5	DR	100	brzina uzorkovanja 128 SPS
4:2	COMP_*	000	komparator nije korišten
1:0	COMP_QUE	11	komparator (ALERT/RDY) onemogućen

Adresa uređaja na I2C magistrali dodatno se određuje stanjem pina ADDR – kada je taj pin spojen na masu (GND), kao u ovoj vježbi, ADS1115 odgovara na standardnu adresu 0×48 [2].

I2C komunikacija

I2C (engl.: *Inter-Integrated Circuit*) je serijski komunikacioni protokol sa dvije linije – SDA (podaci) i SCL (takt) – koje rade u režimu otvorenog kolektora (engl.: *open-drain*), uz obavezne pull-up otpornike prema naponu napajanja, čime se omogućava da više uređaja dijeli istu magistralu [17]. Komunikaciju uvijek pokreće uređaj u ulozi glavnog uređaja (engl.: *master*, u ovom slučaju FPGA), koji generiše START uslov, šalje 7-bitnu adresu uređaja praćenu bitom smjera (upis/čitanje), nakon čega svaki prenešeni bajt podređeni uređaj (engl.: *slave*, ovdje ADS1115) potvrđuje bitom potvrde (engl.: *acknowledge*, ACK); komunikacija se završava STOP uslovom, ili se, radi promjene smjera prenosa bez oslobađanja magistrale, koristi REPEATED START uslov [17]. Upravo je ova sekvenca (upis konfiguracije, čekanje, REPEATED START i čitanje rezultata) implementirana u FSM-u opisanom u narednoj sekciji.

Blok dijagram cjelokupnog sistema, koji povezuje potencijometar, ADS1115 modul, FPGA razvojni sistem i LED diode, prikazan je na slici 3.15.



Slika 3.15: Blok dijagram sistema korištenog u laboratorijskoj vježbi 4.

3.4.2 Struktura projekta i način spajanja

Projekat korišten u ovoj vježbi sastoji se od sljedećih fajlova:

- `i2c_ctrl.sv` – modul koji implementira I2C *master* kao konačni automat (engl.: *Finite State Machine*, FSM), realizovan isključivo na nivou logičkih signala (engl.: *bit-banging* pristup, bez posebnog Gowin IP jezgra), koji upravlja linijama SCL i SDA preko izlaza sa tri stanja (engl.: *tri-state* izlaza, `scl_oe/sda_oe`) kako bi se ostvario zahtjevani režim otvorenog kolektora (engl.: *open-drain*). Modul kontinuirano ponavlja sekvencu: upis konfiguracionog registra (tabela 3.13), čekanje na završetak konverzije, te čitanje 16-bitnog rezultata sa pokazivačem na registar konverzije.
- `top.sv` – najviši nivo hijerarhije za zadatak 1, koji rezultat ADC konverzije mapira na 6 ugrađenih LED dioda u vidu "trčecg" bar-grafa.
- `top_pwm.sv` – najviši nivo hijerarhije za zadatak 2, koji rezultat ADC konverzije koristi kao PWM radni ciklus (engl.: *duty cycle*) za kontinualnu regulaciju jačine svjetla LED dioda.
- `tang_nano_9k.cst` – fizičko mapiranje signala na pinove Tang Nano 9K razvojnog sistema.

Fizičko mapiranje signala na pinove Tang Nano 9K razvojnog sistema dato je u Kodu 28. Onboard LED diode su aktivne-niske (engl.: *active-low*, upaljene kada je pin postavljen na logičku nulu), dok su pinovi za I2C (31 i 32) odabrani proizvoljno između slobodnih GPIO pinova pločice, s obzirom na to da FPGA ne posjeduje namjensku hardversku I2C perifernu jedinicu vezanu za tačno određene pinove.

Kod 28 Fizičko mapiranje pinova za laboratorijsku vježbu 4 (`tang_nano_9k.cst`).

```

IO_LOC   "clk" 52;
IO_PORT  "clk" IO_TYPE=LVC MOS33 PULL_MODE=UP;

IO_LOC   "areset_n" 4;
IO_PORT  "areset_n" IO_TYPE=LVC MOS18 PULL_MODE=UP;
  
```

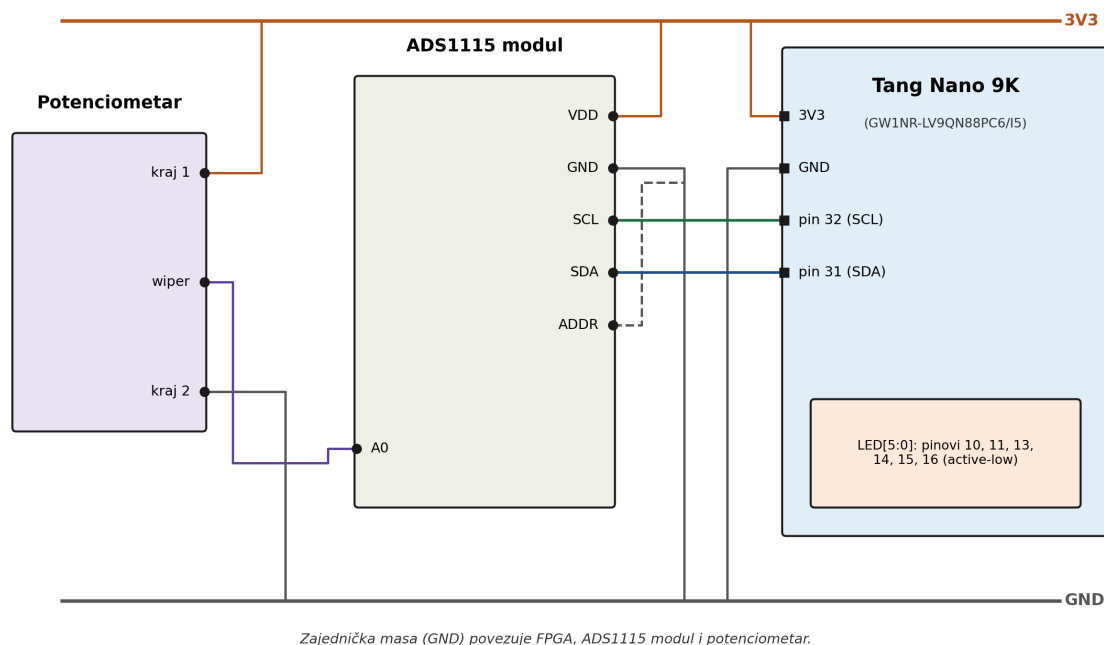
```

IO_LOC    "led[0]" 10;
IO_PORT   "led[0]" IO_TYPE=LVC MOS18 PULL_MODE=NONE;
IO_LOC    "led[1]" 11;
IO_PORT   "led[1]" IO_TYPE=LVC MOS18 PULL_MODE=NONE;
IO_LOC    "led[2]" 13;
IO_PORT   "led[2]" IO_TYPE=LVC MOS18 PULL_MODE=NONE;
IO_LOC    "led[3]" 14;
IO_PORT   "led[3]" IO_TYPE=LVC MOS18 PULL_MODE=NONE;
IO_LOC    "led[4]" 15;
IO_PORT   "led[4]" IO_TYPE=LVC MOS18 PULL_MODE=NONE;
IO_LOC    "led[5]" 16;
IO_PORT   "led[5]" IO_TYPE=LVC MOS18 PULL_MODE=NONE;

IO_LOC    "i2c_sda" 31;
IO_PORT   "i2c_sda" IO_TYPE=LVC MOS33;
IO_LOC    "i2c_scl" 32;
IO_PORT   "i2c_scl" IO_TYPE=LVC MOS33;

```

Fizičko spajanje komponenti prikazano je na slici 3.16. Potenciometar i ADS1115 modul napajaju se sa istog 3.3V izvora kao i sama FPGA pločica (a ne sa 5V), čime se osigurava da napon na analognom ulazu A0 nikada ne pređe naponski nivo VDD samog ADS1115 modula, što je neophodno kako mjerenje ne bi prekoračilo dozvoljene granice ADC-a.



Slika 3.16: Šema spajanja potenciometra, ADS1115 modula i Tang Nano 9K razvojnog sistema.

3.4.3 Zadatak 1 – LED bar-graf upravljan potenciometrom

U okviru prvog zadatka potrebno je ostvariti da broj upaljenih LED dioda prati napon očitao sa potenciometra – pri minimalnom naponu (potenciometar okrenut do kraja u jednom smjeru)

sve diode su ugašene, dok pri maksimalnom naponu (potenciometar okrenut do kraja u drugom smjeru) svih šest dioda svijetli, sa postepenim, "trčećim" paljenjem preostalih dioda za međuvrijednosti napona.

Kako je rezultat konverzije ADS1115 pri odabranom PGA opsegu ($FS = 2.048\text{ V}$, tabela 3.13) 15-bitni pozitivan broj (bit 15 služi kao znak, a pri jednostrukom mjerenju naspram mase on je uvijek 0 za očekivane pozitivne napone), taj opseg $[0, 2^{15} - 1]$ podijeljen je na šest jednakih dijelova, po jedan za svaku diodu, prema pragovima datim u tabeli 3.14. Dioda i se pali kada digitalni kod D (nakon ograničavanja eventualnih malih negativnih vrijednosti, nastalih usljed šuma blizu 0 V , na nulu) dostigne ili premaši prag TH_i :

$$\text{led_on}[i] = (D \geq TH_i), \quad TH_i = \left\lceil \frac{i}{6} \cdot (2^{15} - 1) \right\rceil, \quad i = 1, \dots, 6 \quad (3.14)$$

Tabela 3.14: Pragovi paljenja LED dioda (zadatak 1).

Dioda	Prag TH_i	Udio punog opsega
led[0]	5461	1/6
led[1]	10922	2/6
led[2]	16384	3/6
led[3]	21845	4/6
led[4]	27306	5/6
led[5]	32767	6/6 (puni opseg)

Ovakav način poređenja (umjesto npr. direktnog dekodera) daje efekat "trčećeg svjetla" – kako se ulazni napon povećava, diode se pale redom jedna za drugom, a kako se smanjuje, gase se istim redoslijedom obrnuto.

Implementacija ovog zadatka data je u Kodu 29. Modul `i2c_ctrl` (opisan u sekciji 3.4.2) instanciran je nepromijenjen; `top` modul dodatno generiše takt za I2C FSM, vrši tri-state upravljanje linijama SCL/SDA, i implementira logiku bar-grafa iz jednačine (3.14).

Kod 29 SystemVerilog implementacija zadatka 1 – LED bar-graf upravljan potencijetrom (`top.sv`).

```

1 module top (
2     input logic clk,           // Tang Nano 9K onboard 27 MHz
3     input logic areset_n,      // active-low reset button (S1)
4     inout wire i2c_scl,        // to ADS1115 SCL (needs an external/
5     inout wire i2c_sda,        // to ADS1115 SDA (needs an external/
6     output logic [5:0] led     // onboard LEDs, active-low -> pins
7                                 10,11,13,14,15,16
8 );
9
10
11 // -----

```

```

12 // ~400 kHz strobe that paces the I2C bit-banging FSM
13 // -----
14 logic [26:0] brojac;
15 logic strobe_400kHz;
16
17 always_ff @(negedge areset_n or posedge clk) begin
18     if (areset_n == 1'b0) begin
19         brojac      <= 27'd0;
20         strobe_400kHz <= 1'b0;
21     end else begin
22         brojac <= brojac + 27'd1;
23         if (brojac == 27'd68) begin
24             strobe_400kHz <= 1'b1;
25             brojac      <= 27'd0;
26         end else begin
27             strobe_400kHz <= 1'b0;
28         end
29     end
30 end
31
32 // -----
33 // ADS1115 read: single-ended AIN0 vs GND, PGA = +-2.048V, single
34 // -shot.
35 // start_transaction is tied high so it re-triggers continuously,
36 // i.e. it just keeps polling the potentiometer forever.
37 // -----
38 logic transaction_done;
39 logic [15:0] result_16;
40
41 logic scl_oe, scl_di, sda_oe, sda_di;
42
43 i2c_ctrl u_i2c_ads1115 (
44     .clk            (clk),
45     .strobe_400kHz  (strobe_400kHz),
46     .areset_n       (areset_n),
47     .start_transaction(1'b1),
48     .pause_duration  (16'hFFFF),
49     .transaction_done (transaction_done),
50     .scl_oe          (scl_oe),
51     .scl_di          (scl_di),
52     .sda_oe          (sda_oe),
53     .sda_di          (sda_di),
54     .result_16       (result_16)
55 );
56
57 assign i2c_scl = scl_oe ? 1'b0 : 1'bz;
58 assign scl_di  = i2c_scl;
59
60 assign i2c_sda = sda_oe ? 1'b0 : 1'bz;
61 assign sda_di  = i2c_sda;

```

```

62 // -----
63 // ADC code -> 6-LED bargraph
64 // -----
65 logic [15:0] adc_mag;
66 assign adc_mag = result_16[15] ? 16'd0 : result_16;
67
68 localparam logic [15:0] TH1 = 16'd5461;
69 localparam logic [15:0] TH2 = 16'd10922;
70 localparam logic [15:0] TH3 = 16'd16384;
71 localparam logic [15:0] TH4 = 16'd21845;
72 localparam logic [15:0] TH5 = 16'd27306;
73 localparam logic [15:0] TH6 = 16'd32767;
74
75 logic [5:0] led_on;
76
77 assign led_on[0] = (adc_mag >= TH1);
78 assign led_on[1] = (adc_mag >= TH2);
79 assign led_on[2] = (adc_mag >= TH3);
80 assign led_on[3] = (adc_mag >= TH4);
81 assign led_on[4] = (adc_mag >= TH5);
82 assign led_on[5] = (adc_mag >= TH6);
83
84 assign led = ~led_on; // onboard LEDs are active-low (0 = lit)
85
86 endmodule

```

U nastavku je detaljnije opisana uloga pojedinih dijelova koda datog u Kodu 29.

Ulazno-izlazni portovi modula `top` direktno odgovaraju fizičkim signalima sa slike 3.16: `clk` je onboard takt od 27 MHz, `areset_n` je aktivno-niski signal reseta sa tastera S1, `i2c_scl` i `i2c_sda` su dvosmjerni (inout) signali ka ADS1115 modulu, dok je `led[5:0]` izlaz ka šest ugrađenih, aktivno-niskih LED dioda.

Prvi blok koda (brojač `brojac` i registar `strobe_400kHz`) generiše periodični impuls u trajanju jednog takta, koji se ponavlja svakih 69 taktova 27 MHz oscilatora ($27\text{MHz}/69 \approx 391\text{kHz}$). Ovaj signal, nazvan `strobe_400kHz`, koristi se kao vremenska baza za modul `i2c_ctrl`: umjesto da FSM tog modula radi direktno na 27 MHz, on napreduje kroz svoja stanja tek kada je `strobe_400kHz` aktivan, čime se ostvaruje I2C brzina komunikacije reda veličine nekoliko stotina kHz, u skladu sa standardom [17].

Modul `i2c_ctrl` instanciran je kao `u_i2c_ads1115`, sa nepromijenjenom internom implementacijom u odnosu na opis iz sekcije 3.4.2. Ulaz `start_transaction` vezan je trajno na logičku jedinicu, čime se FSM tjera da nakon svakog završenog ciklusa (upis konfiguracije, čekanje, čitanje rezultata) odmah iznova započne novi – efektivno se ostvaruje kontinualno, neprekidno očitavanje potencijometra bez potrebe za eksternim okidanjem. Ulaz `pause_duration` određuje koliko dugo FSM čeka između upisa konfiguracionog registra i čitanja rezultata, kako bi se ADS1115 osiguralo dovoljno vremena da završi *single-shot* konverziju. Izlaz `transaction_done` generiše modul `i2c_ctrl` da signalizira kraj jedne transakcije, ali ga `top` modul u ovom zadatku ne koristi dalje (rezultat `result_16` se čita kontinualno, pa nije neophodno detektovati tačan trenutak ažuriranja). Signali `scl_oe/sda_oe` predstavljaju izlazne signale omogućavanja (aktivni kada modul želi liniju povući na masu), dok `scl_di/sda_di` predstavljaju digitalne ulaze kojima modul očitava trenutno stanje linija

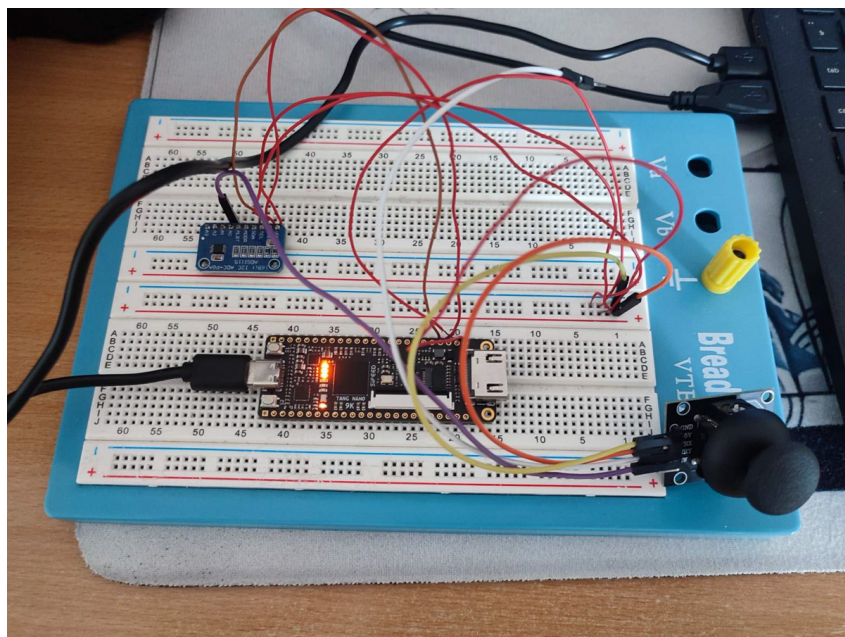
(neophodno, između ostalog, za očitavanje ACK bita i za detekciju eventualnog *clock stretching*-a od strane ADS1115).

Dvije linije koda oblika `assign i2c_scl = scl_oe ? 1'b0 : 1'bz;` realizuju traženi režim otvorenog kolektora: kada modul `i2c_ctrl` želi liniju postaviti na logičku nulu, izlaz se aktivno postavlja na 0; u suprotnom, izlaz se prebacuje u visoku impedansu (`1'bz`), čime linija ostaje slobodna da je, preko vanjskog pull-up otpornika, vrati na logičku jedinicu. Ista linija se odmah zatim (`assign scl_di = i2c_scl;`) vraća nazad kao ulaz u modul `i2c_ctrl`, tako da FSM u svakom trenutku može očitati stvarno stanje linije, bez obzira na to da li je trenutno postavlja on sam ili neki drugi uređaj na magistrali. Analogno je realizovana i linija `i2c_sda`.

Drugi dio koda obrađuje 16-bitni rezultat konverzije, `result_16`, koji stiže iz modula `i2c_ctrl` u obliku broja u dvostrukom komplementu. Linija `assign adc_mag = result_16[15] ? 16'd0 : result_16;` osigurava da, ukoliko bi zbog šuma blizu 0V konverzija povremeno dala vrlo mali negativan broj (bit 15 postavljen na 1), takva vrijednost bude odsječena na nulu, umjesto da se – usljed tumačenja bita 15 kao dijela magnitude – pogrešno protumači kao ogroman pozitivan broj, što bi trenutno upalilo sve LED diode. Konstante TH1–TH6 predstavljaju šest pragova iz tabele 3.14, izračunatih prema jednačini (3.14). Šest linija oblika `assign led_on[i] = (adc_mag >= THi);` nezavisno poredi trenutnu magnitudu `adc_mag` sa svakim od pragova, čime se dobija tzv. termometarski kod: kako `adc_mag` raste, biti u `led_on` se pale redom od najnižeg ka najvišem, nikada u proizvoljnom redoslijedu, što i proizvodi traženi efekat "trčećeg svjetla". Konačno, linija `assign led = ~led_on;` invertuje sve bitove, jer su ugrađene LED diode aktivno-niske – logička jedinica u `led_on` (dioda treba da svijetli) mora se pretvoriti u logičku nulu na samom izlazu `led` da bi dioda zaista bila upaljena.

Testiranje

S obzirom na to da bi vjerna simulacija modula `i2c_ctrl` zahtijevala izradu potpunog bihevioralnog modela ADS1115 slave uređaja (uključujući odziv na START/STOP uslove, ACK/NACK signalizaciju i internu logiku konverzije), što znatno prevazilazi okvir ove vježbe, ispravnost implementacije je – nakon provjere sinteze i internog FSM-a – verifikovana direktno na fizičkom sistemu, spojenom prema šemi datoj na slici 3.16: promjenom položaja potencijometra i posmatranjem broja upaljenih LED dioda potvrđeno je da se implementacija ponaša u skladu sa jednačinom (3.14). Rezultat koji se dobije nakon prebacivanja koda na fizički hardver je vidljiv na slici 3.17.



Slika 3.17: Broj upaljenih LED dioda kontinualno prati položaj potencimetra.

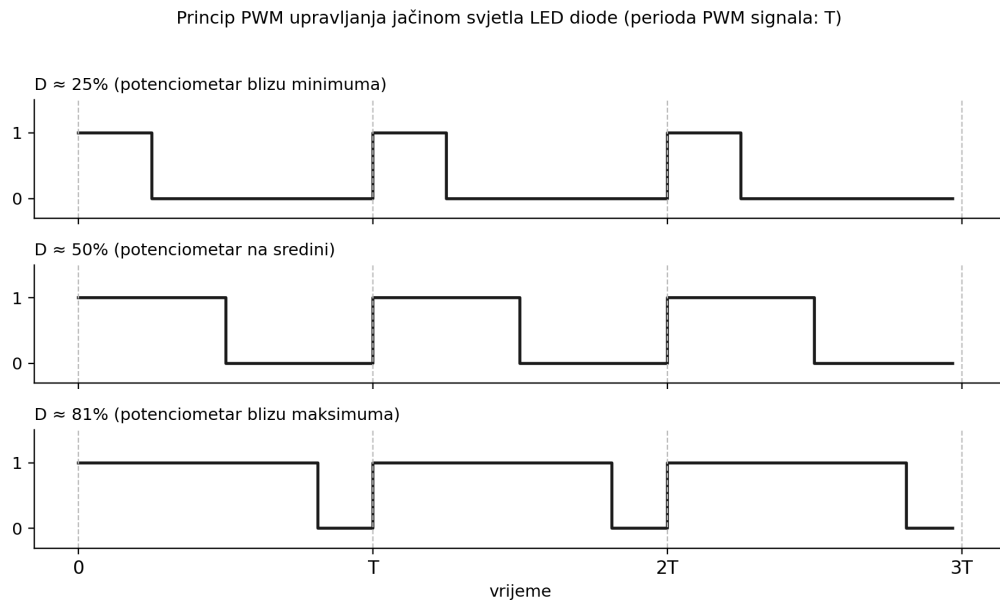
3.4.4 Zadatak 2 – Upravljanje jačinom svjetla LED diode pomoću PWM-a

Drugi zadatak zahtijeva da se potencimetrom kontinualno upravlja *jačinom* svjetla LED diode, a ne samo brojem upaljenih dioda – pri minimalnom naponu dioda je potpuno ugašena, pri maksimalnom svijetli punom jačinom, a za međuvrijednosti napona svijetli proporcionalno smanjenim intenzitetom.

FPGA izlazni pinovi su, za razliku od analognog izlaza, isključivo digitalni (dvije moguće vrijednosti napona), pa se kontinualna promjena jačine svjetla ne može ostvariti direktno. Umjesto toga koristi se pulsno-širinska modulacija (engl.: *Pulse Width Modulation*, PWM) – tehnika kojom se izlazni signal vrlo brzo prekida između logičke jedinice i nule, pri čemu se udio vremena provedenog u stanju "upaljeno" naziva radni ciklus (engl.: *duty cycle*) [18]:

$$D_{uty} = \frac{t_{ON}}{T} \times 100\% \quad (3.15)$$

Ukoliko je frekvencija prekidanja dovoljno visoka (u praksi, iznad otprilike 100Hz, daleko iznad praga treperenja koji ljudsko oko može percipirati), LED dioda se ne doima kao da treperi, već kao da svijetli kontinualnom, prividno analognom jačinom, proporcionalnom radnom ciklusu [18]. Princip je ilustriran na slici 3.18, za tri karakteristične vrijednosti radnog ciklusa.



Slika 3.18: Princip PWM upravljanja jačinom svjetla LED diode za tri različita radna ciklusa.

U implementaciji je iskorišten 8-bitni brojač koji se neprekidno inkrementira, čime se dobija period od 256 taktova ($\approx 105\text{ kHz}$ pri taktu od 27 MHz – daleko iznad granice vidljivog treperenja). Radni ciklus $duty \in [0, 255]$ dobija se uzimanjem gornjih 8 bita 15-bitne magnitute ADC rezultata (`adc_mag[14:7]`), čime se, zbog istog efekta saturacije ADC-a opisanog u zadatku 1, puna jačina svjetla ($duty \approx 255$) pouzdano dostiže tačno na fizičkom maksimumu potenciometra. LED dioda svijetli (aktivna-niska) onoliko taktova unutar perioda koliko iznosi $duty$:

$$pwm_on = (pwm_cnt < duty) \quad (3.16)$$

Implementacija ovog zadatka data je u Kodu 30. I2C dio koda (generisanje takta, instanciranje modula `i2c_ctrl`, tri-state upravljanje SCL/SDA linijama) identičan je zadatku 1; jedina razlika je u načinu na koji se rezultat ADC konverzije koristi za upravljanje LED diodama – umjesto poređenja sa šest pragova, generiše se jedan PWM signal koji istovremeno pokreće svih šest ugrađenih LED dioda.

Kod 30 SystemVerilog implementacija zadatka 2 – upravljanje jačinom svjetla LED diode pomoću PWM-a (`top_pwm.sv`).

```

1 module top (
2     input logic clk,           // Tang Nano 9K onboard 27 MHz
3     input logic areset_n,      // active-low reset button (S1)
4     inout wire i2c_scl,        // to ADS1115 SCL (needs an external/
5     inout wire i2c_sda,        // to ADS1115 SDA (needs an external/
6     // onboard pull-up)
7 
```

```

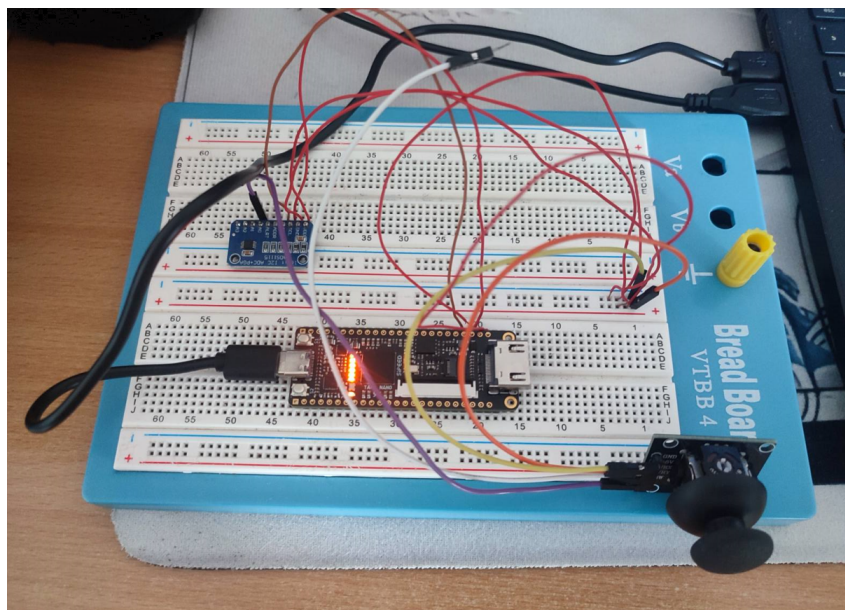
8      output logic [5:0] led      // onboard LEDs, active-low -> pins
9      10,11,13,14,15,16
10
11      // -----
12      // ~400 kHz strobe that paces the I2C bit-banging FSM (unchanged)
13      // -----
14      logic [26:0] brojac;
15      logic strobe_400kHz;
16
17      always_ff @(negedge areset_n or posedge clk) begin
18          if (areset_n == 1'b0) begin
19              brojac      <= 27'd0;
20              strobe_400kHz <= 1'b0;
21          end else begin
22              brojac <= brojac + 27'd1;
23              if (brojac == 27'd68) begin
24                  strobe_400kHz <= 1'b1;
25                  brojac      <= 27'd0;
26              end else begin
27                  strobe_400kHz <= 1'b0;
28              end
29          end
30      end
31
32      // -----
33      // ADS1115 read: single-ended AIN0 vs GND, PGA = +-2.048V, single
34      // -shot.
35      // Unchanged from the bargraph version -- same wiring (pot +
36      // ADS1115
37      // both on 3.3V, wiper straight into A0, no divider).
38      // -----
39      logic transaction_done;
40      logic [15:0] result_16;
41
42      logic scl_oe, scl_di, sda_oe, sda_di;
43
44      i2c_ctrl u_i2c_ads1115 (
45          .clk          (clk),
46          .strobe_400kHz (strobe_400kHz),
47          .areset_n      (areset_n),
48          .start_transaction(1'b1),
49          .pause_duration (16'hFFFF),
50          .transaction_done (transaction_done),
51          .scl_oe          (scl_oe),
52          .scl_di          (scl_di),
53          .sda_oe          (sda_oe),
54          .sda_di          (sda_di),
55          .result_16       (result_16)
56      );

```

```
56  assign i2c_scl = scl_oe ? 1'b0 : 1'bz;
57  assign scl_di  = i2c_scl;
58
59  assign i2c_sda = sda_oe ? 1'b0 : 1'bz;
60  assign sda_di  = i2c_sda;
61
62  // -----
63  // ADC code -> 8-bit PWM duty cycle (LED brightness)
64  // -----
65  logic [15:0] adc_mag;
66  assign adc_mag = result_16[15] ? 16'd0 : result_16;
67
68  logic [7:0] duty;
69  assign duty = adc_mag[14:7];
70
71  // Free-running 8-bit counter -> PWM period = 256 clk cycles
72  // (~105 kHz @ 27 MHz).
73  logic [7:0] pwm_cnt;
74
75  always_ff @(negedge areset_n or posedge clk) begin
76      if (areset_n == 1'b0) begin
77          pwm_cnt <= 8'd0;
78      end else begin
79          pwm_cnt <= pwm_cnt + 8'd1;
80      end
81  end
82
83  logic pwm_on;
84  assign pwm_on = (pwm_cnt < duty); // '1' for 'duty' out of
85  // every 256 cycles
86
87  // Drive all 6 onboard LEDs together (active-low).
88  assign led = {6{~pwm_on}};
89
90  endmodule
```

Testiranje

Kao i u zadatku 1, verifikacija je izvršena direktno na fizičkom hardveru, iz istog razloga navedenog u sekciji 3.4.3 (složenost potpunog bihevioralnog modela ADS1115 slave uređaja potrebnog za simulaciju). Pomjeranjem potenciometra od jednog kraja hoda do drugog, jačina svjetla svih šest LED dioda glatko raste od potpuno ugašenog do punog intenziteta, bez vidljivog treperenja, u skladu sa jednačinama (3.15)–(3.16). Rezultat je prikazan na slici 3.19.

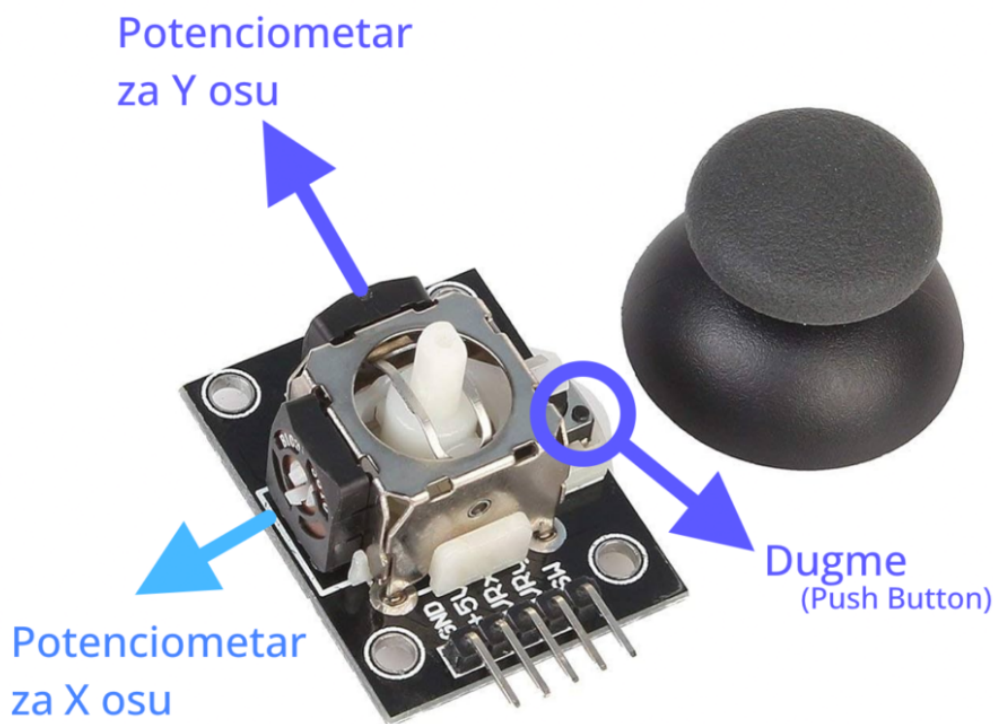


Slika 3.19: Jačina svjetla LED dioda kontinualno prati položaj potencijometra.

3.5 Laboratorijska vježba 5

3.5.1 Teorijski uvod – analogni džojstik i proširenje interaktivne HDMI igre

U laboratorijskoj vježbi 4 (sekcija 3.4) potencijometar je korišten kao izvor analognog napona koji se, preko ADS1115 modula i I2C magistrale, digitalizuje i koristi za upravljanje LED diodama. U ovoj vježbi isti princip očitavanja analognog veličine primjenjuje se na interaktivnu HDMI igru: umjesto tastera, položajem crvene loptice na ekranu upravlja se analognim džojstikom (slika 3.20), čime FPGA sistem prvi put istovremeno kombinuje generisanje video signala u realnom vremenu i obradu analognog upravljačkog signala.



Slika 3.20: Analogni džojstik modul korišten u laboratorijskoj vježbi.

Analogni džojstik kao dvoosni potenciometar

Analogni džojstik modul u suštini predstavlja dva nezavisna linearna potenciometra – jedan za X, drugi za Y osu – čiji su klizači mehanički spojeni na zajedničku polugu, tako da nagnjanje poluge u bilo kojem pravcu istovremeno mijenja položaj oba klizača, srazmjerno projekciji otklona na svaku od osa [19]. Svaka osa se, sa stanovišta električnog kola, ponaša identično potenciometru opisanom u laboratorijskoj vježbi 4 – napon na klizaču dat je istim izrazom (3.12) – s tom razlikom da se u mirovanju (kada korisnik ne dodiruje polugu) klizač mehanički vraća u sredinu otporničke staze, pa napon na izlazu svake ose u mirovanju iznosi približno polovinu napona napajanja V_{DD} [19]. U ovoj vježbi korištena je samo jedna osa džojstika (Y), čiji se izlaz dovodi na isti analogni ulaz ADS1115 modula (A0) koji je u LV4 korišten za potenciometar. Druga osa (X) ostaje nespojena, a horizontalna pozicija loptice je fiksna.

Generisanje HDMI video signala i logika igre

Prikaz na monitoru ostvaruje se generisanjem HDMI video signala rezolucije 1280×720 piksela pri frekvenciji osvježavanja od 60 Hz, uz TMDS (engl.: *Transition-Minimized Differential Signaling*) kodiranje i serijalizaciju piksela u FPGA-u – ova infrastruktura (hdmi_backend sa pripadajućim podmodulima) preuzeta je nepromijenjena iz ranije razvijenog projekta i u ovoj vježbi se koristi kao gotova cjelina, bez izmjena. Iznad generisanog video signala implementirana je jednostavna igra: crvena loptica kružnog oblika kreće se unutar igraćeg polja, dok se zeleni vertikalni zid sa horizontalnim prorezom kontinuirano pomjera zdesna nalijevo, sa novom (slučajno odabranom) visinom proreza pri svakom novom pojavljivanju sa desne strane ekrana. Zadatak igrača je da džojstikom pomjera lopticu gore/dole tako da ona u trenutku susreta sa zidom prođe kroz otvoreni prorez; u suprotnom dolazi do sudara, što se signalizira posebnom LED diodom. Fokus ove vježbe nije na samom generisanju HDMI signala (koje je

pretpostavljeno kao već savladano), već na proširenju postojeće igre novim, analognim načinom upravljanja.

Mrtva zona i prelazak između taktnih domena

Za razliku od tastera, koji imaju samo dva stanja (pritisnut/otpušten), analogni džojstik nikada ne miruje u savršeno istom položaju – mehaničke tolerancije i sitan šum uzrokuju da očitana vrijednost oko sredine oscilira za nekoliko desetina do stotina jedinica ADC koda, čak i kada korisnik uopšte ne dodiruje polugu. Kada bi se i najmanji otklon od teorijske sredine odmah pretvarao u pomjeranje loptice, ona bi neprestano, neželjeno "puzala" u jednom pravcu. Ovaj problem se rješava uvođenjem tzv. mrtve zone (engl.: *deadzone*) – opsega vrijednosti oko centra unutar kojeg se očitavanje tumači kao "džojstik u mirovanju", opisano detaljnije u sekciji 3.5.3.

Dodatna razlika u odnosu na LV4 jeste da se u ovoj vježbi po prvi put unutar jednog projekta koriste dva nezavisna taktna domena: I2C dio (kao i cijeli LV4) radi na frekvenciji vanjskog oscilatora od 27 MHz, dok HDMI dio, radi generisanja piksela odgovarajućom brzinom, koristi izvedeni takt od 74.25 MHz, dobijen internim PLL-om. Signal koji nastaje u jednom taktnom domenu (rezultat očitavanja džojstika) ne smije se direktno koristiti kao ulaz u logiku drugog taktnog domena (pomjeranje loptice) – takav prelazak, poznat kao prelazak između taktnih domena (engl.: *Clock Domain Crossing*, CDC), bez odgovarajuće sinhronizacije može dovesti do metastabilnosti: stanja u kojem izlaz flip-flopa privremeno poprimi neodređenu, ni logičku nulu ni jedinicu vrijednost, usljed kršenja vremena postavljanja/držanja (engl.: *setup/hold time*) signala koji se mijenja asinhrono u odnosu na takt koji ga uzorkuje [20]. Standardno i u praksi najčešće korišteno rješenje za spore, jednobitne (ili, kao ovdje, uskopojasne) signale jeste dvostruki flip-flop sinhronizator (engl.: *double flip-flop synchronizer*): signal se registruje sa dva uzastopna flip-flopa u odredišnom taktnom domenu, čime se vjerovatnoća da metastabilnost "procuri" dalje kroz kolo svodi na zanemarljivo malu vrijednost, po cijenu kašnjenja od jednog dodatnog perioda odredišnog takta [20]. Primjena ovog rješenja u konkretnoj implementaciji opisana je u sekciji 3.5.3.

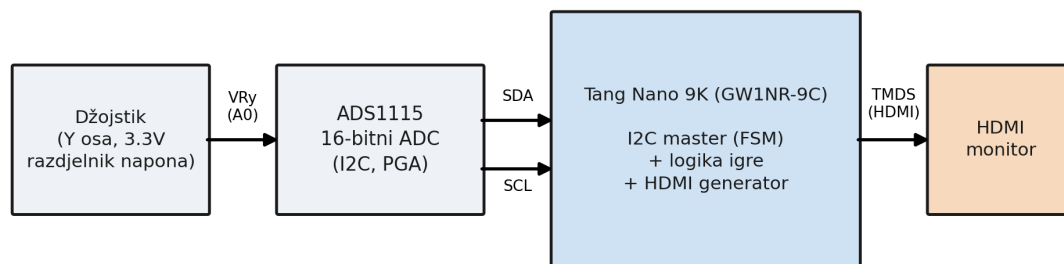
3.5.2 Struktura projekta i način spajanja

Projekat korišten u ovoj vježbi sastoji se od sljedećih fajlova:

- `i2c_ctrl sv` – modul I2C *master*-a, potpuno nepromijenjen u odnosu na LV4 (sekcija 3.4.2): isti FSM, ista konfiguracija ADS1115-a (jednostruko mjerenje na A0, PGA = ± 2.048 V).
- `hdmi_pkg sv`, `hdmi_backend sv`, `hdmi_tdms_enc sv`, `fpgatech_lib/GOWIN/*.sv` – infrastruktura za generisanje HDMI video signala (PLL, serijalizatori, TMDS enkoder), preuzeta nepromijenjena iz ranijeg projekta.
- `top sv` – najviši nivo hijerarhije: instancira HDMI generator i I2C *master*, implementira logiku igre (loptica, prepreka, detekcija sudara) i, novo u odnosu na raniju verziju igre, preslikava očitavanje džojstika u pomjeranje loptice po Y osi.
- `projekat cst` – fizičko mapiranje signala na pinove Tang Nano 9K razvojnog sistema.

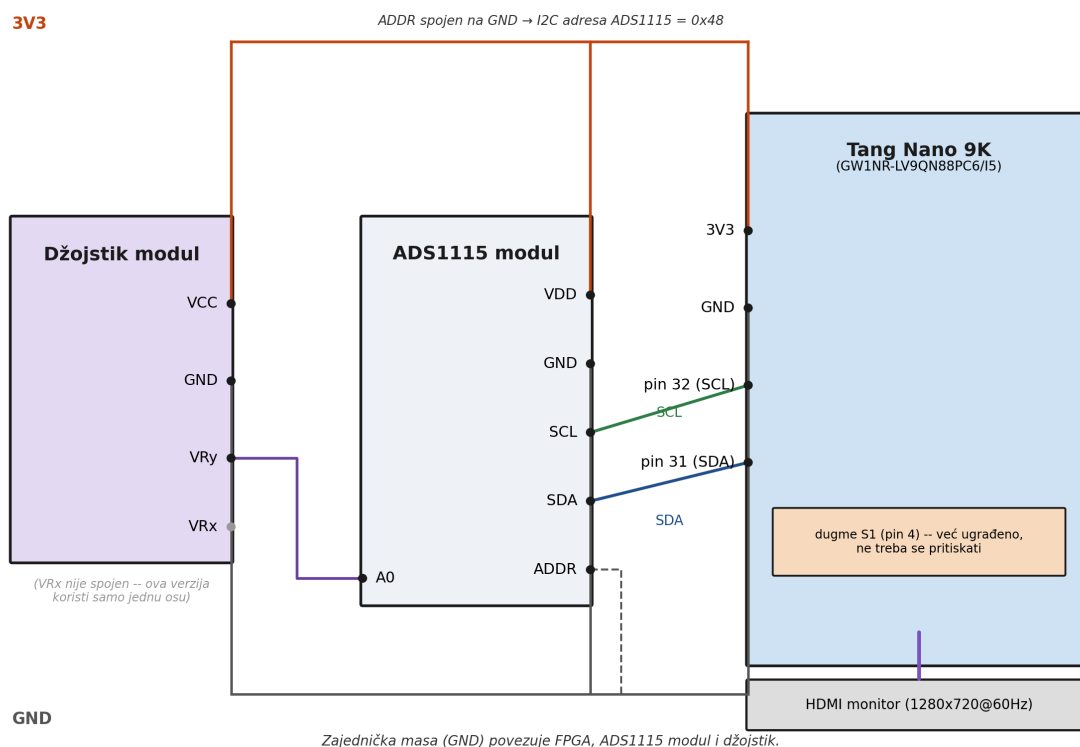
Blok dijagram sistema prikazan je na slici 3.21: džojstik (samo Y osa) spojen je na ADS1115 modul, koji FPGA sistem naizmjenično čita I2C protokolom; FPGA istovremeno generiše HDMI video signal koji nosi sliku igre ka monitoru.

Blok dijagram sistema: Y osa džojstika se čita preko I2C busa i koristi za pomjeranje crvene loptice u igri prikazanoj putem HDMI-a



Slika 3.21: Blok dijagram sistema korištenog u laboratorijskoj vježbi 5.

Fizičko spajanje džojstika i ADS1115 modula na Tang Nano 9K razvojni sistem prikazano je na slici 3.22. Spoj je identičan onom iz LV4 (ista I2C konfiguracija, isti pinovi 31/32, potencijometar zamijenjen Y osom džojstika na istom ulazu A0), uz jednu dodatnu, važnu sličnost: signal reseta I2C/džojstik dijela kola (`areset_n`) vezan je, kao i u LV4, na fizičko dugme S1 (pin 4) razvojne pločice – dio kola koji radi na 27MHz tako ima svoj sopstveni, nezavisan izvor reseta, potpuno odvojen od reseta HDMI dijela kola, čime se izbjegava potreba za dodatnim sinhronizatorom reset signala između dva taktna domena.



Slika 3.22: Šema spajanja džojstika, ADS1115 modula i Tang Nano 9K razvojnog sistema.

Fizičko mapiranje pinova dato je u Kodu 31. HDMI izlazni pinovi (diferencijalni parovi takta i tri kanala podataka) preuzeti su nepromijenjeni iz ranijeg projekta.

Kod 31 Fizičko mapiranje pinova za laboratorijsku vježbu 5 (projekat .cst).

```
IO_LOC   "clk_ext" 52;
IO_PORT  "clk_ext" PULL_MODE=UP   IO_TYPE=LVC MOS33 ;

IO_LOC   "areset_n" 4;
IO_PORT  "areset_n" IO_TYPE=LVC MOS18 PULL_MODE=UP;

IO_LOC   "i2c_sda" 31;
IO_PORT  "i2c_sda" IO_TYPE=LVC MOS33;
IO_LOC   "i2c_scl" 32;
IO_PORT  "i2c_scl" IO_TYPE=LVC MOS33;

IO_LOC   "led_n" 11;
IO_PORT  "led_n" PULL_MODE=UP DRIVE=8;

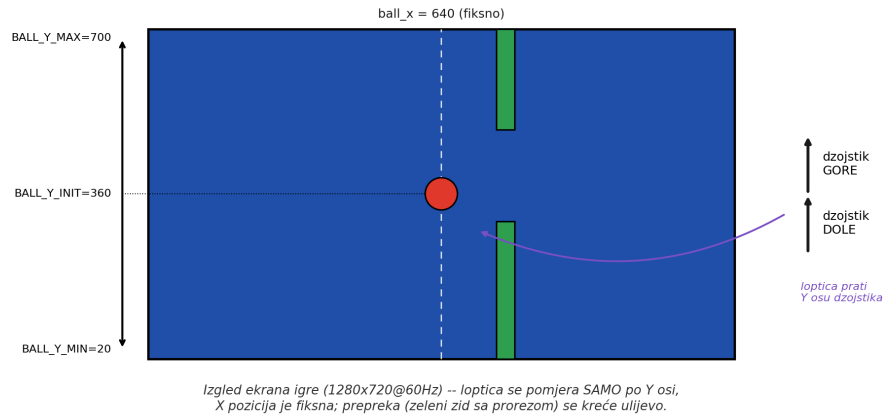
IO_LOC   "led_hit_n" 10;
IO_PORT  "led_hit_n" PULL_MODE=UP DRIVE=8;

IO_LOC   "led_dbg_n[0]" 13;
IO_PORT  "led_dbg_n[0]" PULL_MODE=UP DRIVE=8;
IO_LOC   "led_dbg_n[1]" 14;
IO_PORT  "led_dbg_n[1]" PULL_MODE=UP DRIVE=8;
IO_LOC   "led_dbg_n[2]" 15;
IO_PORT  "led_dbg_n[2]" PULL_MODE=UP DRIVE=8;

IO_LOC   "hdmi_clk_p"      69,68;
IO_PORT  "hdmi_clk_p"      PULL_MODE=NONE DRIVE=8;
IO_LOC   "hdmi_dat_p[0]" 71,70;
IO_PORT  "hdmi_dat_p[0]" PULL_MODE=NONE DRIVE=8;
IO_LOC   "hdmi_dat_p[1]" 73,72;
IO_PORT  "hdmi_dat_p[1]" PULL_MODE=NONE DRIVE=8;
IO_LOC   "hdmi_dat_p[2]" 75,74;
IO_PORT  "hdmi_dat_p[2]" PULL_MODE=NONE DRIVE=8;
```

3.5.3 Zadatak – upravljanje igrom putem Y ose analognog džojstika

Zadatak je proširiti postojeću igru (crvena loptica, pokretna prepreka sa prerezom, detekcija sudara) tako da se loptica po vertikalnoj (Y) osi pomjera analognim džojstikom umjesto tasterima, dok njena horizontalna (X) pozicija ostaje fiksna, na sredini ekrana. Izgled ekrana igre, sa označenim granicama kretanja loptice, prikazan je na slici 3.23.



Slika 3.23: Izgled ekrana igre – loptica se kreće isključivo po Y osi, između granica BALL_Y_MIN i BALL_Y_MAX.

Implementacija zadatka data je u Kodu 32.

Kod 32 SystemVerilog implementacija upravljanja igrom putem Y ose džojstika (top.sv).

```

1 module top
2   import hdmi_pkg::*;
3   (
4     input  logic  clk_ext,      // vanjski oscilator: 27 MHz (Tang Nano
                                9K)
5     input  logic  areset_n,     // fizicko dugme S1 (aktivno nisko) ->
                                pin 4
6
7     inout  wire   i2c_scl,      // ka ADS1115 SCL
8     inout  wire   i2c_sda,      // ka ADS1115 SDA
9
10    output logic  hdmi_clk_p,
11    output logic  hdmi_clk_n,
12    output bus3_t hdmi_dat_p,
13    output bus3_t hdmi_dat_n,
14
15    output logic  led_n,         // treperi ravnomjerno - igra je "ziva"
16    output logic  led_hit_n,     // pali se isključivo pri sudaru sa
                                preprekom
17
18    output logic [2:0] led_dbg_n // dijagnostički bar-graf (vidi
                                tekst)
19  );
20
21  logic  clk_pix, srst_n;
22  logic  frame_done;
23  logic  mfram_done;
24
25  bus12_t x, y;
26  pix_t  pix;

```

```

27 // Loptica: X pozicija fiksna, Y pozicija se pomjera dzojstikom.
28 localparam bus12_t BALL_R      = 12'd16;
29 localparam bus12_t BALL_X_INIT = 12'd640;
30 localparam bus12_t BALL_Y_MIN  = 12'd20;
31 localparam bus12_t BALL_Y_MAX  = 12'd700;
32 localparam bus12_t BALL_Y_INIT = 12'd360;
33
34
35 bus12_t ball_y;
36 localparam bus12_t ball_x = BALL_X_INIT;
37
38 // Prepreka: vertikalni zid sirine OBST_WIDTH, sa prorezom
39 //          GAP_HEIGHT,
40 // koji se kreće OBST_SPEED piksela po prikazanom okviru.
41 localparam bus12_t OBST_WIDTH  = 12'd40;
42 localparam bus12_t OBST_SPEED  = 12'd6;
43 localparam bus12_t GAP_HEIGHT  = 12'd200;
44
45 bus12_t obst_x;
46 bus12_t gap_y;
47 logic [1:0] gap_sel;
48
49 function automatic bus12_t gap_preset(input logic [1:0] sel);
50     case (sel)
51         2'd0: gap_preset = 12'd60;
52         2'd1: gap_preset = 12'd220;
53         2'd2: gap_preset = 12'd380;
54         default: gap_preset = 12'd500;
55     endcase
56 endfunction
57
58 logic obst_here, in_gap, hit;
59
60 always_comb begin: _my_game
61     obst_here = (x >= obst_x) && (x < obst_x + OBST_WIDTH);
62     in_gap     = (y >= gap_y) && (y < gap_y + GAP_HEIGHT);
63
64     if ((x-ball_x)*(x-ball_x)+(y-ball_y)*(y-ball_y)<=250) begin
65         pix.R = '1; pix.G = '0; pix.B = '0; // loptica
66     end else if (obst_here && !in_gap) begin
67         pix.R = '0; pix.G = '1; pix.B = '0; // prepreka
68     end else begin
69         pix.R = '0; pix.G = '0; pix.B = '1; // pozadina
70     end
71 end: _my_game
72
73 always_comb begin: _collision_check
74     hit = (obst_x <= ball_x + BALL_R)
75         & (obst_x + OBST_WIDTH > ball_x - BALL_R)
76         & ~( (ball_y >= gap_y + BALL_R) & (ball_y + BALL_R <= gap_y
77             + GAP_HEIGHT) );

```

```

76  end: _collision_check
77
78  hdmi_backend u_hdmi_backend (
79      .clk_ext(clk_ext), .clk_pix(clk_pix), .srst_n(srst_n),
80      .hcount(x), .vcount(y), .pix(pix),
81      .frame_done(frame_done), .mfram_done(mfram_done),
82      .hdmi_clk_p(hdmi_clk_p), .hdmi_clk_n(hdmi_clk_n),
83      .hdmi_dat_p(hdmi_dat_p), .hdmi_dat_n(hdmi_dat_n)
84  );
85
86  always_ff @(negedge srst_n or posedge clk_pix) begin:
    _obstacle_mov
87      if (srst_n == L0) begin
88          obst_x  <= 12'(HSCREEN) + OBST_WIDTH;
89          gap_y   <= gap_preset(2'd0);
90          gap_sel <= 2'd0;
91      end else if (frame_done == HI) begin
92          if (obst_x <= OBST_SPEED) begin
93              gap_sel <= gap_sel + 2'd1;
94              gap_y   <= gap_preset(gap_sel + 2'd1);
95              obst_x  <= 12'(HSCREEN) + OBST_WIDTH;
96          end else begin
97              obst_x <= obst_x - OBST_SPEED;
98          end
99      end
100  end: _obstacle_mov
101
102  always_ff @(negedge srst_n or posedge clk_pix) begin: _led_mon
103      if (srst_n == L0) led_n <= HI;
104      else if (mfram_done == HI) led_n <= ~led_n;
105  end: _led_mon
106
107  always_ff @(negedge srst_n or posedge clk_pix) begin: _hit_led
108      if (srst_n == L0) led_hit_n <= HI;
109      else led_hit_n <= hit ? L0 : HI;
110  end: _hit_led
111
112  // -----
113  // I2C / ADS1115 / dzojstik -- identicno LV4 (jedan kanal, AIN0,
114  // PGA=+-2.048V), radi u domenu takta clk_ext (27 MHz).
115  // -----
116  logic [26:0] brojac_i2c;
117  logic strobe_400kHz;
118
119  always_ff @(negedge areset_n or posedge clk_ext) begin
120      if (areset_n == 1'b0) begin
121          brojac_i2c    <= 27'd0;
122          strobe_400kHz <= 1'b0;
123      end else begin
124          brojac_i2c <= brojac_i2c + 27'd1;
125          if (brojac_i2c == 27'd68) begin

```

```

126         strobe_400kHz <= 1'b1;
127         brojac_i2c     <= 27'd0;
128     end else begin
129         strobe_400kHz <= 1'b0;
130     end
131 end
132 end
133
134 logic transaction_done;
135 logic [15:0] result_16;
136 logic scl_oe, scl_di, sda_oe, sda_di;
137
138 i2c_ctrl u_i2c_ads1115 (
139     .clk(clk_ext), .strobe_400kHz(strobe_400kHz), .areset_n(
140         areset_n),
141     .start_transaction(1'b1), .pause_duration(16'hFFFF),
142     .transaction_done(transaction_done),
143     .scl_oe(scl_oe), .scl_di(scl_di), .sda_oe(sda_oe), .sda_di(
144         sda_di),
145     .result_16(result_16)
146 );
147
148 assign i2c_scl = scl_oe ? 1'b0 : 1'bz;
149 assign scl_di  = i2c_scl;
150 assign i2c_sda = sda_oe ? 1'b0 : 1'bz;
151 assign sda_di  = i2c_sda;
152
153 logic [15:0] adc_mag;
154 assign adc_mag = result_16[15] ? 16'd0 : result_16;
155
156 // Dijagnosticki bar-graf: [0],[1] -- pragovi sirove magnitude;
157 // [2] -- "otkucaj srca" FSM-a (mijenja stanje pri svakoj
158 //      završenoj
159 // I2C transakciji), koristan za brzu provjeru da protokol uopšte
160 //      radi.
161 logic [2:0] led_dbg;
162 assign led_dbg[0] = (adc_mag >= 16'd10000);
163 assign led_dbg[1] = (adc_mag >= 16'd22000);
164
165 logic i2c_heartbeat;
166 always_ff @(negedge areset_n or posedge clk_ext) begin
167     if (areset_n == 1'b0) i2c_heartbeat <= 1'b0;
168     else if (transaction_done) i2c_heartbeat <= ~i2c_heartbeat;
169 end
170 assign led_dbg[2] = i2c_heartbeat;
171 assign led_dbg_n  = ~led_dbg;
172
173 // Preslikavanje ADC koda u brzinu/pravac pomjeranja loptice.
174 localparam logic signed [16:0] JOY_CENTER  = 17'sd26400;
175 localparam logic          [15:0] DEADZONE   = 16'd800;
176 localparam logic          [15:0] FAST_THRESH = 16'd4000;

```

```

173 localparam logic signed [ 4:0] STEP_SLOW    = 5'sd3;
174 localparam logic signed [ 4:0] STEP_FAST    = 5'sd9;
175
176 logic signed [16:0] diff_y;
177 assign diff_y = $signed({1'b0, adc_mag}) - JOY_CENTER;
178
179 logic [15:0] adiff_y;
180 assign adiff_y = diff_y[16] ? 16'(-diff_y) : 16'(diff_y);
181
182 logic signed [4:0] step_y_comb;
183 always_comb begin
184     if (adiff_y < DEADZONE) begin
185         step_y_comb = 5'sd0;
186     end else if (adiff_y < FAST_THRESH) begin
187         step_y_comb = diff_y[16] ? -STEP_SLOW : STEP_SLOW;
188     end else begin
189         step_y_comb = diff_y[16] ? -STEP_FAST : STEP_FAST;
190     end
191 end
192
193 logic signed [4:0] step_y_ext;
194 always_ff @(negedge areset_n or posedge clk_ext) begin
195     if (areset_n == 1'b0) step_y_ext <= 5'sd0;
196     else step_y_ext <= step_y_comb;
197 end
198
199 // CDC: dvostruki flip-flop sinhronizator iz clk_ext u clk_pix
200 // domen.
201 logic signed [4:0] step_y_sync0, step_y_sync;
202 always_ff @(negedge srst_n or posedge clk_pix) begin:
203     _cdc_sync_pix
204     if (srst_n == L0) begin
205         step_y_sync0 <= 5'sd0;
206         step_y_sync <= 5'sd0;
207     end else begin
208         step_y_sync0 <= step_y_ext;
209         step_y_sync <= step_y_sync0;
210     end
211 end: _cdc_sync_pix
212
213 always_ff @(negedge srst_n or posedge clk_pix) begin: _ball_mov
214     if (srst_n == L0) begin
215         ball_y <= BALL_Y_INIT;
216     end else if (frame_done == HI) begin
217         if (step_y_sync > 5'sd0) begin
218             if (ball_y < BALL_Y_MAX - 12'(step_y_sync))
219                 ball_y <= ball_y + 12'(step_y_sync);
220             else
221                 ball_y <= BALL_Y_MAX;
222         end else if (step_y_sync < 5'sd0) begin
223             if (ball_y > BALL_Y_MIN + 12'(-step_y_sync))

```

```
222         ball_y <= ball_y - 12' (-step_y_sync);
223     else
224         ball_y <= BALL_Y_MIN;
225     end
226 end
227 end: _ball_mov
228
229 endmodule: top
```

U nastavku je detaljnije opisana uloga pojedinih dijelova koda datog u Kodu 32.

Portovi i preuzeta HDMI/logika igre

Ulazno-izlazni portovi modula `top` prošireni su u odnosu na raniju verziju igre (koja je koristila dva tastera) sa signalima `areset_n`, `i2c_scl` i `i2c_sda`, dok su `hdmi_clk_p/n`, `hdmi_dat_p/n`, `led_n` i `led_hit_n` nepromijenjeni. Instanciranje `hdmi_backend` (koji generiše taktove `clk_pix` i `srst_n`, te trenutnu poziciju piksela `x/y`), blok `_my_game` (koji boji trenutni piksel u zavisnosti od toga da li se nalazi unutar loptice, prepreke ili pozadine), blok `_collision_check` (koji izračunava signal `hit`), blok `_obstacle_mov` (koji svakih 60-tak puta u sekundi pomjera prepreku ulijevo) i blokovi `_led_mon/_hit_led` (LED signalizacija) preuzeti su bez izmjena iz ranije razvijene igre – jedina razlika je zamjena fiksne konstante `BALL_X` promjenljivom `ball_y`, koja se, umjesto tasterima, sada ažurira iz I2C/džojstik dijela koda opisanog u nastavku.

Nezavisan reset I2C/džojstik dijela kola

Dio koda koji čita džojstik radi u domenu takta `clk_ext` (27 MHz) i, kao i u LV4, koristi `areset_n` – signal sa fizičkog dugmeta S1 – kao svoj asinhroni reset. Ovaj signal je namjerno potpuno nezavisan od signala `srst_n` koji resetuje HDMI dio kola (domen takta `clk_pix`): s obzirom na to da su u pitanju dva odvojena taktna domena, deljenje jednog te istog reset signala između njih bi samo po sebi predstavljalo još jedan slučaj prelaska između taktnih domena (jer bi se oslobađanje reseta u jednom domenu moralo bezbjedno prenijeti i u drugi domen), pa je jednostavnije i pouzdanije svakom taktnom domenu dodijeliti sopstveni, nezavisan izvor reseta.

Generisanje I2C takta i čitanje ADS1115 (identično LV4)

Blok koji generiše `strobe_400kHz` i instanciranje modula `i2c_ctrl` (`u_i2c_ads1115`) potpuno su identični odgovarajućem dijelu koda iz LV4 (Kod 29), uključujući i vrijednost `pause_duration=16'hFFFF` – detaljno objašnjenje ovog dijela dato je u sekciji 3.4.3. Isto tako, tri-state upravljanje linijama `i2c_scl/i2c_sda` i odsijecanje eventualnih negativnih kodova u signalu `adc_mag` preuzeti su nepromijenjeni iz LV4.

Dijagnostički bar-graf

Tri diode `led_dbg_n[2:0]` nisu neophodne za rad same igre, ali predstavljaju koristan alat za nezavisnu provjeru rada I2C dijela kola tokom razvoja i testiranja, potpuno odvojeno od HDMI slike: diode `led_dbg_n[0]` i `led_dbg_n[1]` pale se kada sirova magnituda očitavanja `adc_mag` pređe unaprijed određene pragove, dok `led_dbg_n[2]` mijenja stanje pri svakom signalu `transaction_done` – s obzirom na to da se taj signal generiše dva puta u

toku svakog potpunog ciklusa čitanja (jednom nakon upisa konfiguracije, jednom po završetku čitanja rezultata), ova dioda vidljivo treperi sve dok I2C komunikacija ispravno funkcionira, a ostaje trajno u jednom stanju ukoliko FSM iz bilo kojeg razloga (npr. izostanak ACK signala od ADS1115 modula) ne uspije završiti transakciju. Ovakav pristup – ugrađivanje jednostavnih, nezavisnih dijagnostičkih signala u sâm dizajn – česta je i korisna praksa pri otklanjanju grešaka u digitalnim sistemima, jer omogućava lociranje problema (u ovom slučaju: da li greška potiče iz same I2C komunikacije ili iz logike koja obrađuje već ispravno očitane vrijednosti) bez potrebe za spoljašnjom mjernom opremom.

Mrtva zona i preslikavanje u brzinu pomjeranja

Rezultat `adc_mag` se prvo upoređuje sa teorijskom vrijednošću sredine, `JOY_CENTER`. Budući da je odabrani PGA opseg $FS = \pm 2.048 \text{ V}$ (isti kao u LV4), a džojstik se, iz istog sigurnosnog razloga kao potencijometar u LV4, napaja sa 3.3 V FPGA sabirnice, teorijski napon sredine iznosi $V_{sredina} = 3.3/2 = 1.65 \text{ V}$, što prema jednačini (3.13) odgovara digitalnom kodu:

$$JOY_CENTER = \frac{V_{sredina}}{FS} \cdot 2^{15} = \frac{1.65}{2.048} \cdot 32768 \approx 26400 \quad (3.17)$$

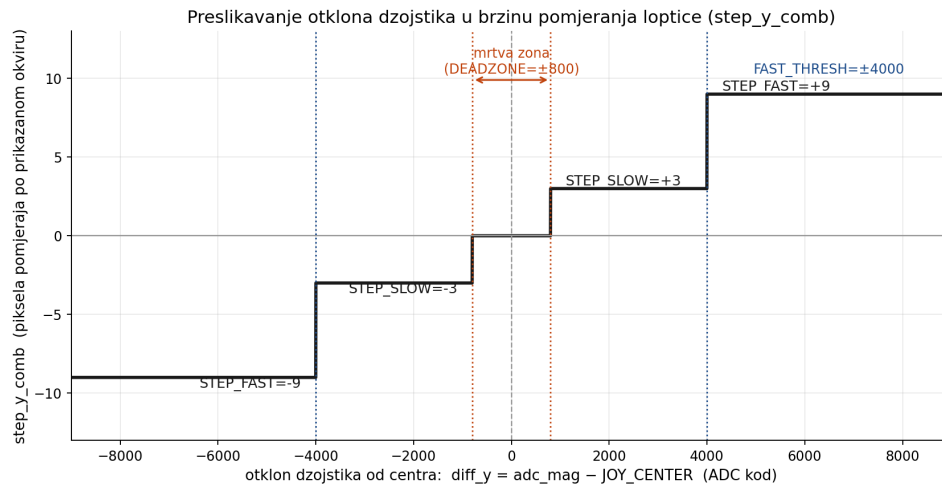
Razlika `diff_y = adc_mag - JOY_CENTER` (računata u dvostrukom komplementu, sa dodatnim bitom radi ispravnog predznaka) predstavlja potpisani otklon trenutnog očitavanja od te teorijske sredine, dok `adiff_y` predstavlja njegovu apsolutnu vrijednost. Na osnovu `adiff_y` i predznaka `diff_y`, brzina i smjer pomjeranja loptice po Y osi (`step_y_comb`) određuju se prema pravilu:

$$\text{step_y_comb} = \begin{cases} 0, & |\text{diff_y}| < DEADZONE \\ \pm STEP_SLOW, & DEADZONE \leq |\text{diff_y}| < FAST_THRESH \\ \pm STEP_FAST, & |\text{diff_y}| \geq FAST_THRESH \end{cases} \quad (3.18)$$

gdje predznak + odgovara otklonu džojstika iznad sredine (`diff_y[16]=0`), a predznak – otklonu ispod sredine (`diff_y[16]=1`). Konkretni vrijednosti parametara korištene u implementaciji date su u tabeli 3.15, a sama funkcija je grafički prikazana na slici 3.24.

Tabela 3.15: Parametri preslikavanja otklona džojstika u brzinu pomjeranja loptice.

Parametar	Vrijednost	Značenje
JOY_CENTER	26400	teorijska sredina (jednačina (3.17))
DEADZONE	800	poluširina mrtve zone oko centra
FAST_THRESH	4000	prag otklona za prelazak na veću brzinu
STEP_SLOW	3 piksela/okvir	brzina za mali otklon (mrtva zona $\leq diff < FAST_THRESH$)
STEP_FAST	9 piksela/okvir	brzina za veliki otklon ($ diff \geq FAST_THRESH$)



Slika 3.24: Preslikavanje otklona džojstika od centra u brzinu pomjeranja loptice, sa označenom mrtvom zonom i pragom prelaska na veću brzinu.

Vrijednost `step_y_comb` se registruje u signal `step_y_ext` u istom taktom domenu (`clk_ext`) prije prelaska u domen `clk_pix` – ovo je standardna CDC praksa: sinhronizuje se već registrovan, a ne "sirov" kombinacioni signal, čime se izbjegavaju dodatni hazardi (kratkotrajne, nevažeće međuvrijednosti kombinacione logike) na ulazu sinhronizatora [20].

Prelazak u domen `clk_pix` i pomjeranje loptice

Blok `_cdc_sync_pix` implementira dvostruki flip-flop sinhronizator opisan u sekciji 3.5.1: signal `step_y_ext`, nastao u domenu `clk_ext`, registruje se dva puta uzastopno na takt `clk_pix` (`step_y_sync0`, zatim `step_y_sync`), čime se rizik od metastabilnosti svodi na zanemarljiv nivo prije nego što se signal upotrijebi u logici koja pomjera lopticu [20]. Konačno, blok `_ball_mov` – koji se, kao i pomjeranje prepreke, izvršava jednom po svakom prikazanom okviru (signal `frame_done`, 60 puta u sekundi) – ažurira `ball_y` za iznos `step_y_sync` u odgovarajućem smjeru, uz provjeru granica `BALL_Y_MIN/BALL_Y_MAX`: ukoliko bi pomjeranje za puni korak izašlo van dozvoljenog opsega, loptica se umjesto toga postavlja tačno na najbližu granicu, čime se sprječava da izađe van vidljivog dijela ekrana.

Testiranje

Implementacija je testirana direktno na fizičkom sistemu, spojenom prema šemi na slici 3.22. Pomjeranjem džojstika gore/dole potvrđeno je da se loptica pomjera u očekivanom smjeru i očekivanom (proporcionalnom otklonu) brzinom, da ostaje na granicama igračeg polja kada se džojstik drži u krajnjem položaju, te da se, kada je otpušten, loptica zaustavlja i ostaje nepomična (zahvaljujući mrtvoj zoni iz jednačine (3.18)). Dijagnostička dioda `led_dbg_n[2]` vidljivo treperi tokom rada, potvrđujući da I2C komunikacija kontinuirano i uspješno funkcioniše, dok diode `led_dbg_n[1:0]` prate otklon džojstika od centra. Sudar loptice sa preprekom ispravno aktivira diodu `led_hit_n`, dok dioda `led_n` tokom cijelog rada ravnomjerno treperi, potvrđujući da HDMI dio kola radi neprekidno i nezavisno od I2C/džojstik dijela.

3.6 Laboratorijska vježba 6

3.6.1 Teorijski uvod – diskretni PID regulator

Za razliku od prethodnih vježbi, u kojima je FPGA korišten za obradu i prikaz izmjerene analogne veličine, u ovoj vježbi se FPGA po prvi put koristi za implementaciju algoritma automatskog upravljanja – diskretnog PID (proporcionalno-integraciono-derivacijskog) regulatora. Kontinualni oblik PID regulatora, koji povezuje grešku $e(t)$ (razliku između željene i trenutne vrijednosti regulisane veličine) sa upravljačkom veličinom $u(t)$, dat je izrazom:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.19)$$

gdje je K_p koeficijent proporcionalnog djelovanja, K_i koeficijent integracijskog djelovanja, a K_d koeficijent derivacijskog djelovanja. Proporcionalni član reaguje na trenutnu grešku, integracijski član akumulira prošle vrijednosti greške i eliminiše stacionarnu grešku, dok derivacijski član reaguje na brzinu promjene greške.

Diskretizacija i inkrementalni (rekurzivni) algoritam

FPGA, za razliku od analognog kola, radi isključivo u diskretnim vremenskim trenucima određenim taktim signalom, pa je izraz (3.19) potrebno diskretizovati. Integral se aproksimira sumom, a izvod razlikom unazad, u trenucima $t = kT$, gdje je T perioda odabiranja. Direktna (pozicijska) diskretizacija zahtijeva pamćenje greške u svim prethodnim trenucima zbog sumacionog (integracijskog) člana, što je za praktičnu hardversku realizaciju nepogodno. Ovaj problem se rješava prelaskom na inkrementalni (rekurzivni) oblik algoritma, u kojem se nova upravljačka vrijednost računa kao zbir prethodne vrijednosti i priraštaja $\Delta u(kT)$ [21]:

$$\Delta u(kT) = K_p [e(kT) - e((k-1)T)] + K_i T e(kT) + \frac{K_d}{T} [e(kT) - 2e((k-1)T) + e((k-2)T)] \quad (3.20)$$

$$u(kT) = u((k-1)T) + \Delta u(kT) \quad (3.21)$$

Grupisanjem članova uz $e(kT)$, $e((k-1)T)$ i $e((k-2)T)$ u izrazu (3.20), inkrementalni algoritam se može zapisati u tzv. paralelnom obliku, pogodnom za direktnu hardversku implementaciju [21]:

$$u(kT) = u((k-1)T) + K_1 e(kT) + K_2 e((k-1)T) + K_3 e((k-2)T) \quad (3.22)$$

$$K_1 = K_p + K_i T + \frac{K_d}{T}, \quad K_2 = -K_p - 2\frac{K_d}{T}, \quad K_3 = \frac{K_d}{T} \quad (3.23)$$

Prednost izraza (3.22) je u tome što se u svakom taktu pamte samo tri prethodne vrijednosti ($u((k-1)T)$, $e((k-1)T)$, $e((k-2)T)$), umjesto cjelokupne historije greške, što ga čini pogodnim za implementaciju u konačnom automatu sa ograničenom memorijom – kao što je FPGA.

Pojednostavljenje na PI regulator

Ukoliko se u sistemu ne koristi derivacijsko djelovanje (npr. zbog osjetljivosti D člana na šum mjerenja), postavlja se $K_d = 0$, čime se dobija proporcionalno-integracioni (PI) regulator. Iz izraza (3.23) tada slijedi $K_3 = 0$, pa treći član u (3.22) u potpunosti otpada, a preostala dva koeficijenta se svode na:

$$K_1 = K_p + K_i T, \quad K_2 = -K_p \quad (3.24)$$

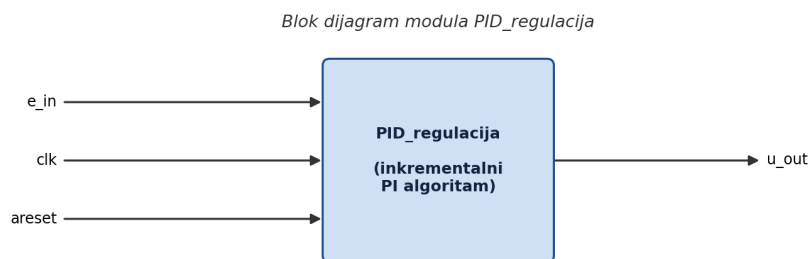
$$u(kT) = u((k-1)T) + K_1 e(kT) + K_2 e((k-1)T) \quad (3.25)$$

U ovoj vježbi implementira se upravo ovaj, pojednostavljeni PI slučaj, uz period odabiranja $T = 1$ s (odabran radi jednostavnije cjelobrojne aritmetike u hardveru – vremenska dimenzija se u tom slučaju "krije" unutar odabranih vrijednosti K_1 i K_2).

3.6.2 Struktura projekta

Za razliku od prethodnih vježbi, koje su zahtijevale fizičko povezivanje perifernih modula (ADS1115, džojstik) na Tang Nano 9K razvojni sistem, ova vježba je u potpunosti simulaciona: ispravnost implementiranog algoritma se provjerava isključivo softverskom simulacijom digitalnog kola, bez potrebe za fizičkim hardverom. Projekat se sastoji od jednog modula `PID_regulacija.sv`.

Blok dijagram modula, sa njegova tri ulaza i jednim izlazom, dat je na slici 3.25.



Slika 3.25: Blok dijagram modula `PID_regulacija`.

Ulaz `e_in` predstavlja trenutnu grešku $e(kT)$ (razliku između željene i izmjerene vrijednosti regulisane veličine, npr. brzine vrtnje motora dobijene iz nekog senzora), a izlaz `u_out` predstavlja izračunatu upravljačku vrijednost $u(kT)$ koja bi se u potpunom sistemu dalje proslijedivala, na primjer, PWM modulu radi upravljanja izvršnim elementom.

3.6.3 Zadatak – implementacija i simulaciona verifikacija diskretnog PI regulatora

Zadatak je implementirati modul `PID_regulacija` prema algoritmu iz izraza (3.25), te njegovu ispravnost provjeriti simulacijom prije bilo kakve eventualne fizičke realizacije. Implementacija je data u Kodu 33.

Kod 33 SystemVerilog implementacija diskretnog PI regulatora (PID_regulacija.sv).

```
1 module PID_regulacija (  
2     output unsigned [14:0] u_out,  
3     input  signed   [15:0] e_in,  
4     input   clk,  
5     input   areset  
6 );  
7  
8     parameter k1 = 10'b1010;  
9     parameter k2 = 10'b001;  
10  
11     logic signed [15:0] u_prev;  
12     logic signed [15:0] e_prev;  
13     logic signed [15:0] e_prev1;  
14     logic signed [35:0] d_reg;  
15     logic signed [35:0] d_wire;  
16  
17     always_ff @(posedge clk or negedge areset) begin  
18         if (areset == 0) begin  
19             u_prev  <= 0;  
20             e_prev  <= 0;  
21             e_prev1 <= 0;  
22         end else if ($signed(e_in) < $signed(16'b0)) begin  
23             u_prev <= 0;  
24         end else begin  
25             e_prev1 <= e_prev;  
26             e_prev  <= e_in;  
27             u_prev  <= u_out;  
28         end  
29     end  
30  
31     assign d_wire = u_prev + k1 * e_in - k2 * e_prev1;  
32  
33     always_ff @(posedge clk) begin  
34         d_reg <= d_wire;  
35         if ($signed(e_in) < $signed(16'b0)) begin  
36             d_reg <= 36'b0;  
37         end else if ($signed(d_wire) > $signed(36'h7FFF)) begin  
38             d_reg <= 36'h7FFF;  
39         end else if ($signed(d_wire) < $signed(36'b0)) begin  
40             d_reg <= 36'b0;  
41         end  
42     end  
43  
44     assign u_out = d_reg[14:0];  
45  
46 endmodule : PID_regulacija
```

Portovi i parametri

Modul ima jedan izlaz, `u_out` (15-bitna neoznačena vrijednost $u(kT)$), ograničena na opseg $[0, 32767]$, i tri ulaza: `e_in` (16-bitna označena vrijednost greške $e(kT)$), `clk` (takt) i `areset` (asinhroni reset, aktivan kada je jednak nuli). Greška je predstavljena označenim brojem jer regulisana veličina može biti i veća i manja od željene vrijednosti. Parametri `k1` i `k2` odgovaraju koeficijentima K_1 i K_2 iz izraza (3.24), sa numeričkim vrijednostima $K_1 = 10$ i $K_2 = 1$.

Pamćenje historije (registri `u_prev`, `e_prev`, `e_prev1`)

Pošto izraz (3.25) zahtijeva prethodnu upravljačku vrijednost $u((k-1)T)$ i prethodnu grešku $e((k-1)T)$, modul mora pamtit i te vrijednosti između dva takta. To se ostvaruje registrima `u_prev`, `e_prev` i `e_prev1`, koji se ažuriraju u prvom `always_ff` bloku. Pri aktivnom asinhronom resetu svi registri se postavljaju na nulu. Ukoliko je trenutna greška negativna, `u_prev` se prisilno postavlja na nulu – ovo predstavlja dodatnu, u osnovnom izrazu (3.25) nepostojeću zaštitnu granu, čija je svrha vjerovatno sprječavanje neograničenog rasta integralnog djelovanja (tzv. *integral windup*) u slučaju kada je regulisana veličina već iznad željene vrijednosti. U svim ostalim slučajevima, registri kliznu za jedno mjesto: `e_prev1` preuzima staru vrijednost `e_prev`, `e_prev` preuzima trenutnu grešku `e_in`, a `u_prev` preuzima prethodno izračunati izlaz `u_out`.

Proračun i zasićenje (`d_wire`, `d_reg`)

Signal `d_wire` predstavlja neograničeni (nesaturisani) rezultat proračuna po formuli (3.25): `u_prev + k1*e_in - k2*e_prev1`. Širina ovog signala (36 bita) namjerno je znatno veća od širine izlaza (15 bita), kako bi se tokom samog proračuna izbjeglo prekoračenje (*overflow*) prije nego što se izvrši eksplicitno zasićenje. Drugi `always_ff` blok tu vrijednost registruje u `d_reg`, uz tri mogućnosti: ako je trenutna greška negativna, izlaz se prisilno postavlja na nulu; ako proračunata vrijednost prelazi maksimalnu dozvoljenu ($36'h7FFF = 32767$, najveći broj predstavljiv u 15-bitnom neoznačenom izlazu), odsijeca se na tu maksimalnu vrijednost; ako bi ispala negativna, odsijeca se na nulu. Konačno, `u_out` se dobija uzimanjem donjih 15 bita iz `d_reg`, što je ispravno upravo zahvaljujući prethodnom zasićenju koje garantuje da je vrijednost uvijek u opsegu $[0, 32767]$.

Testbench i simulacija

Ispravnost implementacije provjerava se simulacijom, korištenjem testbench-a datog u Kodu 34. Za razliku od simulatora koji direktno izvršavaju SystemVerilog, ovdje se koristi Verilator – alat koji SystemVerilog opis modula prevodi u ekvivalentan C++ model, koji se zatim povezuje sa ručno napisanim C++ testbench-om. Ovakav pristup omogućava da se test-scenario (redoslijed i trajanje vrijednosti na ulazu `e_in`) opiše u vidu običnog C++ programa, dok se signali tokom simulacije snimaju u FST zapis talasnih oblika koji se naknadno pregleda u programu GTKWave – pristup koji je znatno brži za otkrivanje grešaka nego programiranje i testiranje direktno na razvojnoj pločici, i koji se preporučuje kao prvi korak provjere svake digitalne implementacije prije njene fizičke realizacije.

Kod 34 Verilator C++ testbench za simulaciju modula `PID_regulacija`.

```
#include <stdlib.h>
```

```
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "VPID_regulacija.h"

#define MAX_SIM_TIME 2200
#define CLK_PERIOD 100 // period takta u ns (posedge clk na svakih
    100 ns)
vluint64_t sim_time = 0;

double sc_time_stamp() {
    return sim_time;
}

int main(int argc, char** argv, char** env) {
    VPID_regulacija *dut = new VPID_regulacija;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;

    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");

    while (sim_time < MAX_SIM_TIME) {

        if (sim_time % CLK_PERIOD == 0) dut->clk = 1;
        if (sim_time % CLK_PERIOD == CLK_PERIOD / 2) dut->clk = 0;

        if (sim_time == 0) {
            dut->areset = 0;
            dut->e_in = 0;
        }
        if (sim_time == 20) {
            dut->areset = 1;
        }

        if (sim_time == 100) {
            dut->e_in = 100; // mala pozitivna greska -> u_out treba
                rasti
        }
        if (sim_time == 600) {
            dut->e_in = 5000; // velika pozitivna greska -> provjera
                zasicenja na 32767
        }
        if (sim_time == 1100) {
            dut->e_in = (uint16_t)(int16_t)(-50); // negativna greska ->
                u_out treba pasti na 0
        }
        if (sim_time == 1400) {
            dut->e_in = 10; // mala pozitivna greska -> provjera
                oporavka
        }
    }
}
```

```

    }
    if (sim_time == 1700) {
        dut->e_in = (uint16_t)(int16_t)(-1000); // veća negativna
        greska
    }
    if (sim_time == 2000) {
        dut->e_in = 0; // vraćanje na neutralnu gresku
    }

    dut->eval();
    m_trace->dump(sim_time);
    sim_time++;
}

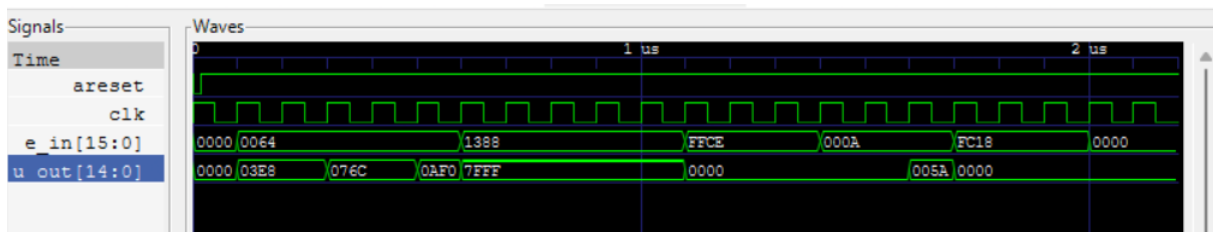
m_trace->close();
delete dut;
exit(EXIT_SUCCESS);
}

```

Testbench redom dovodi na ulaz `e_in` sekvencu vrijednosti $0 \rightarrow 100 \rightarrow 5000 \rightarrow -50 \rightarrow 10 \rightarrow -1000 \rightarrow 0$, svaku dovoljno dugo da se na izlazu ustali odgovarajuća reakcija.

Analiza rezultata simulacije

Rezultat simulacije, dobijen pokretanjem testbench-a iz Koda 34 i prikazan u programu GTKWave, dat je na slici 3.26. Vrijednosti signala `e_in` i `u_out` prikazane su u heksadecimalnom zapisu.



Slika 3.26: GTKWave prikaz rezultata simulacije modula `PID_regulacija`: signali `areset`, `clk`, `e_in[15:0]` i `u_out[14:0]`.

Heksadecimalne vrijednosti sa slike 3.26 odgovaraju sljedećim decimalnim brojevima (uz napomenu da je `e_in` označen broj, pa se npr. `FFCE` čita kao -50 , a `FC18` kao -1000 , u zapisu dvojnog komplementa): `e_in`: $0 \rightarrow 100 \rightarrow 5000 \rightarrow -50 \rightarrow 10 \rightarrow -1000 \rightarrow 0$; `u_out`: $0 \rightarrow 1000 \rightarrow 1900 \rightarrow 2800 \rightarrow 32767 \rightarrow 0 \rightarrow 90 \rightarrow 0$.

Ponašanje modula u svakoj fazi testa sažeto je u tabeli 3.16.

Tabela 3.16: Odziv modula `PID_regulacija` po fazama testa.

e_in	u_out	Komentar
0	0	Stanje neposredno nakon reseta.
100	0 → 1000 → 1900 → 2800	Konstantna pozitivna greška – izlaz linearno raste za $(K_1 - K_2) \cdot e = (10 - 1) \cdot 100 = 900$ po taktu, kao što se i očekuje od integralnog djelovanja na konstantnu grešku.
5000	→ 32767	Velika pozitivna greška – proračunata vrijednost premašuje maksimum, pa se zasićenjem odsijeca na 32767 i tu ostaje.
-50	→ 0	Greška postaje negativna – izlaz se odmah prisilno vraća na nulu, nezavisno od prethodne (maksimalne) vrijednosti.
10	0 (jedan takt kašnjenja), zatim → 90	Greška je ponovo pozitivna, ali se oporavak dešava tek nakon što registri historije (<code>e_prev</code> , <code>e_prev1</code>) "proguraju" novu, malu vrijednost greške kroz sebe.
-1000	→ 0	Ponovna negativna greška – izlaz se, kao i ranije, momentalno vraća na nulu.

Vidljivo je da se svih sedam prelaza sa slike 3.26 tačno poklapa sa vrijednostima navedenim u tabeli, čime je potvrđeno da modul u simulaciji reaguje onako kako je opisano.

Iz tabele 3.16 se vidi da modul, u pogledu vidljivog ponašanja na izlazu, ispunjava osnovni zahtjev: dok je greška pozitivna, upravljačka vrijednost raste (integralno djelovanje), veliku grešku ograničava zasićenje, a čim greška postane negativna, izlaz se bezbjedno vraća na nulu.

Testiranje

Verifikacija u ovoj vježbi izvedena je isključivo simulacijom, korištenjem testbench-a iz Koda 34, bez fizičke realizacije na FPGA razvojnom sistemu – povezivanje modula sa stvarnim izvršnim elementom (npr. DC motorom preko PWM signala) izlazi izvan okvira ove vježbe. Simulacijom je potvrđeno da modul ispravno akumulira upravljačku vrijednost pri konstantnoj pozitivnoj grešci, da zasićenje ograničava izlaz na propisani maksimum, te da se pri negativnoj grešci izlaz bezbjedno vraća na nulu.

*

*

*

Kroz šest laboratorijskih vježbi prikazanih u ovom poglavlju, razvoj digitalnih sistema na FPGA platformi Tang Nano 9K vođen je od najjednostavnijih ka sve složenijim projektima. Vježba 1 uvodi osnove kombinacione logike i uspostavlja metodologiju simulacije putem Verilator/GTKWave alata koja se dosljedno koristi u svim narednim vježbama. Vježba 2 prelazi na sekvencijalnu logiku kroz realizaciju Mealyjevog i Mooreovog automata, i to i u ponašajnom i u strukturnom obliku. Vježba 3 uvodi generisanje video signala u realnom vremenu (HDMI/TMDs) i njegovu primjenu kroz interaktivnu igricu, dok Vježba 4 proširuje sistem integracijom vanjske analogne periferije putem I2C komunikacije. Vježba 5 spaja prethodno stečena znanja – video izlaz, analogni ulaz i rad sa više taktnih domena – u jedinstven interaktivni sistem upravljan džojstikom. Konačno, Vježba 6 se odmiče od direktne interakcije

sa hardverom ka implementaciji i, prije svega, rigoroznoj simulacionoj verifikaciji diskretnog algoritma upravljanja, čime se naglašava značaj provjere ispravnosti implementacije naspram teorijski izvedene formule, a ne samo posmatranja "razumnog" ponašanja izlaza.

Ove vježbe zajedno pokrivaju širok spektar vještina digitalnog projektovanja – od kombinacione i sekvencijalne logike, preko generisanja signala u realnom vremenu i komunikacije sa perifernim uređajima, do implementacije algoritama upravljanja – i predstavljaju praktičnu osnovu na kojoj se zasniva realizacija opisana u prethodnim poglavljima ovog rada. U narednom, završnom poglavlju rada dat je sažetak ostvarenih rezultata i doprinosa, te zaključna razmatranja o mogućim pravcima daljeg razvoja opisanog sistema.

Zaključak

Cilj ovog završnog rada bio je izrada praktikuma iz logičkog dizajna i upravljanja zasnovanog na FPGA razvojnom sistemu Gowin 9k, koji bi na sistematičan i postepen način proveo studenta kroz osnovne koncepte digitalnog dizajna, sve do implementacije naprednijeg upravljačkog algoritma. Kroz tri uvodna poglavlja i šest laboratorijskih vježbi, rad je pokušao odgovoriti na potrebu za strukturiranom literaturom na lokalnom jeziku koja bi olakšala usvajanje FPGA tehnologije studentima Elektrotehničkog fakulteta u Sarajevu.

Sažetak ostvarenih rezultata

Prvo poglavlje rada postavilo je teorijski temelj potreban za razumijevanje unutrašnje arhitekture FPGA uređaja – od konfigurabilnih logičkih blokova i LUT elemenata, preko interkonekcijske mreže, do procesa konfigurisanja čipa putem bitstream-a. Drugo poglavlje je ovu teoriju povezal sa praktičnom pripremom radnog okruženja, kroz detaljan opis instalacije i konfiguracije Gowin EDA softvera, alata za simulaciju (Verilator i GTKWave) te samog Tang Nano 9K razvojnog sistema koji je korišten kroz sve vježbe.

Treće poglavlje, kao srž rada, kroz šest laboratorijskih vježbi rastuće složenosti pokrilo je širok spektar tema digitalnog dizajna. Prva i druga vježba uvele su osnove kombinacione i sekvencijalne logike, kroz implementaciju logičkih funkcija i strukturnu sintezu Mealyevog i Mooreovog konačnog automata, sa verifikacijom putem simulacije za svaki od zadataka. Treća vježba je uvela generisanje video signala u realnom vremenu putem HDMI/TMDs protokola i njegovu primjenu kroz interaktivnu aplikaciju, dok je četvrta vježba proširila sistem integracijom vanjske analogne periferije – potencijometra i ADS1115 analogno-digitalnog konvertora – putem I2C komunikacije. Peta vježba je objedinila prethodno stečena znanja kroz upravljanje interaktivnom aplikacijom analognim džojstikom, uz rješavanje praktičnih problema poput mrtve zone signala i sinhronizacije između različitih taktnih domena. Konačno, šesta vježba se odmakla od direktne interakcije sa fizičkim hardverom ka implementaciji diskretnog PI regulatora i njegovoj simulacionoj verifikaciji, pri čemu je poseban naglasak stavljen na provjeru ispravnosti implementacije naspram teorijski izvedene formule, a ne samo na posmatranje "razumnog" ponašanja izlaznog signala.

Doprinosi rada

Osnovni doprinos ovog rada jeste izrada kompletnog, testiranog i dokumentovanog praktikuma na bosanskom jeziku, koji studenta vodi od najjednostavnijih kombinacionih kola do implementacije algoritma upravljanja na FPGA platformi, uz dosljednu primjenu jedinstvene metodologije rada kroz sve vježbe. U tom smislu, posebno se izdvajaju sljedeći doprinosi:

- uspostavljena je jedinstvena i ponovljiva metodologija simulacije zasnovana na Verilator i

GTKWave alatima, koja se dosljedno primjenjuje kroz sve laboratorijske vježbe, omogućavajući studentu da svaku implementaciju samostalno provjeri prije (ili umjesto) fizičke realizacije na razvojnom sistemu;

- praktikum objedinjuje teme koje se u literaturi obično obrađuju odvojeno – kombinacija i sekvencijalna logika, generisanje video signala u realnom vremenu, komunikacija sa analognim perifernim uređajima putem I2C protokola, rad sa više taktnih domena, te implementacija diskretnog algoritma upravljanja – u jedinstvenu, međusobno nadovezujuću cjelinu;
- kroz šestu vježbu demonstriran je kompletan tok simulacione verifikacije algoritma upravljanja – od teorijskog izvođenja formule diskretnog PI regulatora, preko implementacije odgovarajućeg SystemVerilog modula, do potvrde njegovog ponašanja kroz analizu rezultata simulacije u GTKWave alatu – čime je praktikum zaokružen primjerom koji povezuje teoriju upravljanja sa njenom hardverskom realizacijom.

Ograničenja i mogući pravci daljeg razvoja

Rad ima i određena ograničenja, koja ujedno predstavljaju osnovu za moguće buduće proširenje praktikuma. Verifikacija u četvrtoj i petoj vježbi izvedena je direktno na fizičkom razvojnom sistemu, bez prethodne simulacije analognih perifernih komponenti (ADS1115 modula), zbog složenosti modeliranja I2C komunikacije sa stvarnim slave uređajem u simulacionom okruženju. Šesta vježba, obrnuto, verifikovana je isključivo simulacijom, bez povezivanja modula sa stvarnim izvršnim elementom, čime praktična validacija algoritma upravljanja u zatvorenoj petlji sa realnim aktuatorom ostaje otvorena za dalji rad.

Mogući pravci daljeg razvoja praktikuma uključuju: fizičku realizaciju regulatora iz šeste vježbe uz povezivanje na stvaran izvršni element (npr. DC motor upravljan putem PWM signala) i mjerenje stvarnog odziva sistema u zatvorenoj petlji; proširenje simulacione metodologije na četvrtu i petu vježbu kroz izradu pojednostavljenog modela I2C slave uređaja; te dodavanje novih laboratorijskih vježbi koje bi obradile druge komunikacijske protokole (npr. SPI ili UART) ili složenije algoritme upravljanja (npr. pun PID regulator sa derivacijskim djelovanjem, umjesto trenutno implementiranog PI oblika).

Zaključna riječ

Kroz razvoj i dokumentovanje šest međusobno povezanih laboratorijskih vježbi, ovaj rad je pokazao da se na pristupačnoj FPGA platformi poput Gowin 9k sistema, uz odgovarajuću metodologiju i alate otvorenog koda za simulaciju, može izgraditi cjelovit i praktično primjenjiv edukacijski praktikum koji pokriva put od osnovnih principa digitalnog dizajna do implementacije i verifikacije algoritma upravljanja. Time je ostvaren osnovni cilj rada, a rezultat predstavlja osnovu koja se, kroz predložene pravce daljeg razvoja, može dalje nadograđivati i prilagođavati potrebama budućih generacija studenata.

Lista skraćenica

- ADC** (engl.: Analog-to-Digital Converter, bos.: Analogno-digitalni konvertor)
- CLB** (engl.: Configurable Logic Block, bos.: Konfigurabilni logički blok)
- DC** (engl.: Direct Current, bos.: Istosmjerna struja)
- EDA** (engl.: Electronic Design Automation, bos.: Automatizacija elektroničkog dizajna)
- FPGA** (engl.: Field-Programmable Gate Array, bos.: Programabilni logički sklopovi)
- FSM** (engl.: Finite State Machine, bos.: Konačni automat)
- GPIO** (engl.: General-Purpose Input/Output, bos.: Ulaz/Izlaz opće namjene)
- HDL** (engl.: Hardware Description Language, bos.: Jezik za opis hardvera)
- I/O** (engl.: Input/Output, bos.: Ulaz/Izlaz)
- I2C** (engl.: Inter-Integrated Circuit, bos.: Inter-integrisano kolo)
- LED** (engl.: Light-Emitting Diode, bos.: Svjetleća dioda)
- LUT** (engl.: Look-Up Table, bos.: Pregledna tabela)
- MUX** (engl.: Multiplexer, bos.: Multiplekser)
- PGA** (engl.: Programmable Gain Amplifier, bos.: Programabilno pojačalo)
- PWM** (engl.: Pulse-Width Modulation, bos.: Širinsko-impulsna modulacija)
- SCL** (engl.: Serial Clock Line, bos.: Serijska takt linija)
- SDA** (engl.: Serial Data Line, bos.: Serijska podatkovna linija)
- UART** (engl.: Universal Asynchronous Receiver/Transmitter, bos.: Univerzalni asinhroni prijemnik/predajnik)
- USB** (engl.: Universal Serial Bus, bos.: Univerzalna serijska sabirnica)

Literatura

- [1] Betz, V., Rose, J., Marquardt, A., Architecture and CAD for Deep-Submicron FPGAs. Springer Science & Business Media, 1999, Vol. 502.
- [2] Texas Instruments, “ADS111x ultra-small, low-power, I2C-compatible, 860-SPS, 16-bit ADCs with internal reference, oscillator, and programmable comparator”, Texas Instruments, Datasheet SBAS444E, Dec. 2024, dostupno na: <https://www.ti.com/lit/ds/symlink/ads1115.pdf>
- [3] Kuon, I., Tessier, R., Rose, J., “FPGA architecture: Survey and challenges”, Foundations and Trends in Electronic Design Automation, Vol. 2, No. 2, 2007, str. 135–253.
- [4] Sulaiman, N., Obaid, Z. A., Marhaban, M. H., Hamidon, M. N., “Design and implementation of FPGA-based systems – a review”, Australian Journal of Basic and Applied Sciences, Vol. 3, No. 4, 2009, str. 3575–3596, dostupno na: <https://ajbasweb.com/old/ajbas/2009/3575-3596.pdf>
- [5] Sali, S. M., Meribout, M., Majeed, A. A., “Real-time fpga-based cnns for detection, classification, and tracking in autonomous systems: State-of-the-art designs and optimizations”, arXiv preprint arXiv:2509.04153, 2025.
- [6] Maxfield, C., The Design Warrior’s Guide to FPGAs. Elsevier, 2004.
- [7] IEEE, “IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language”, IEEE, Tech. Rep. IEEE Std 1800-2017, 2018.
- [8] Gülgönül, , “Sparking curiosity in digital system design lectures with take home labs”, arXiv preprint arXiv:2503.16625, 2025, opisuje korištenje Tang Nano 9K razvojnog sistema kao edukacijske platforme u nastavi digitalnog dizajna.
- [9] Sipeed Technology Co., Ltd., “Sipeed tang nano 9k datasheet”, Sipeed, Tech. Rep., 2023, dostupno na: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-9K/Nano-9K.html>
- [10] All About Circuits, “Purpose and internal functionality of fpga look-up tables”, dostupno na: <https://www.allaboutcircuits.com/technical-articles/purpose-and-internal-functionality-of-fpga-look-up-tables/> 2017.
- [11] Iqbal, O., “Understanding fpga lookup tables (luts) in xilinx artix-7 – a complete guide”, dostupno na: <https://medium.com/@owaisiqbal2021/understanding-fpga-lookup-tables-luts-in-xilinx-artix-7-a-complete-guide-429469e8d065> 2025.

- [12] Sutherland, S., Mills, D., “Synthesizing systemverilog: Busting the myth that systemverilog is only for verification”, SNUG Silicon Valley, Tech. Rep., 2013, dostupno na: https://sutherland-hdl.com/papers/2013-SNUG-SV_Synthesizable-SystemVerilog_paper.pdf
- [13] Chu, P. P., “Systemverilog vs verilog in rtl design”, Cleveland State University, Tech. Rep., 2018, dostupno na: <https://academic.csuohio.edu/chu-pong/wp-content/uploads/sites/64/2023/02/SystemVerilog-vs-Verilog-in-RTL-design.pdf>
- [14] DisplayModule, “Introduction to hdmi protocol”, dostupno na: <https://www.displaymodule.com/blogs/knowledge/introduction-to-hdmi-protocol> 2023.
- [15] fpga4fun.com, “Hdmi – fpga to hdmi interfacing”, dostupno na: <https://www.fpga4fun.com/HDMI.html> 2023.
- [16] SparkFun Electronics, “Voltage dividers”, SparkFun Learn, dostupno na: <https://learn.sparkfun.com/tutorials/voltage-dividers/all>
- [17] NXP Semiconductors, “UM10204: I2C-bus specification and user manual”, NXP Semiconductors, User manual UM10204, Oct. 2021, dostupno na: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [18] SparkFun Electronics, “Pulse width modulation”, SparkFun Learn, dostupno na: <https://learn.sparkfun.com/tutorials/pulse-width-modulation/all>
- [19] Last Minute Engineers, “In-depth: How a 2-axis joystick works? interfacing with arduino & processing”, lastminuteengineers.com, dostupno na: <https://lastminuteengineers.com/joystick-interfacing-arduino-processing/>
- [20] Cummings, C. E., “Clock domain crossing (cdc) design & verification techniques using SystemVerilog”, Synopsys Users Group (SNUG), Boston, MA, Conference paper, 2008, dostupno na: http://www.sunburst-design.com/papers/CummingsSNUG2008Boston_CDC.pdf
- [21] Haugen, F., “Tidsdiskret pid-regulator (kapittel 6, sesm3401 – styring av mekatroniske systemer)”, TechTeach / Høgskolen i Buskerud, dostupno na: https://techteach.no/fag/sesm3401/h08/pid_diskret/tidsdiskret_pid_reg.pdf